

1 Explainability in Reinforcement Learning

2 Maxime Alaarabiou

3 Monday 21st September, 2026

Chapter 1

Deep Learning

This chapter introduces the fundamental concepts of Deep Learning (DL), the training paradigm used to optimize deep Neural Network (NN). The objective is to provide the necessary foundation in DL for the ideas developed throughout the rest of this manuscript. Its structure is as follows:

Section 1.1: An overview of the historical developments that led to the emergence of the field.

Section 1.2: A formalization of the objective underlying DL algorithms.

Section 1.3: An examination of the structure of artificial NN.

Section 1.4: An explanation of how the learning process shapes NN.

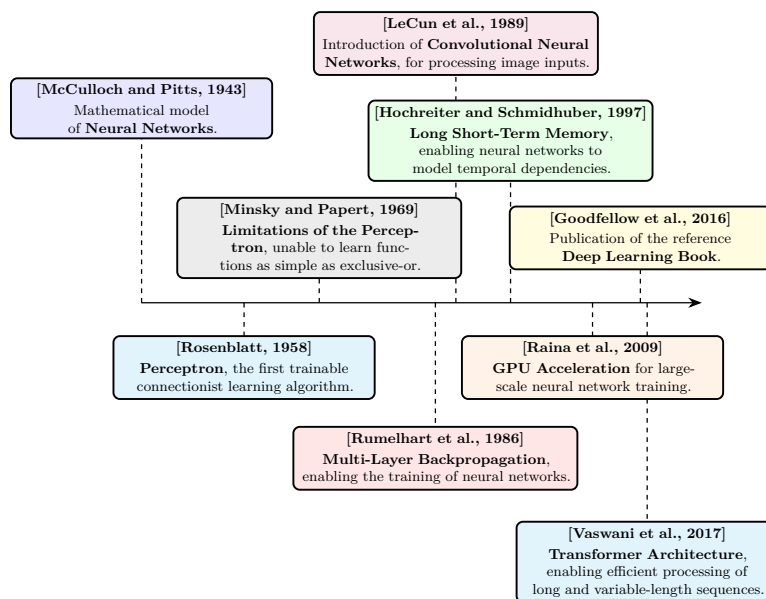
Section 1.5: A discussion of how such models are evaluated.

Section 1.6: A summary of the concepts introduced in the chapter and a discussion of the theoretical limitations of DL.

1.1 Historical Developments

NN and DL emerged from an ambition to study the expressive capabilities of organic brain. The first major milestone came in 1943 with the proposal of a mathematical model of an artificial neuron, with a formulation of networked interactions [McCulloch and Pitts, 1943]. This work established the key idea that networks of simple units can give rise to complex computations. The focus was on representational power, not on the ability of such systems to learn. As a result, this work remained largely theoretical and had limited practical applications.

A major step toward operationalizing the idea of NNs was achieved with the perceptron algorithm in 1958 [Rosenblatt, 1958]. It was designed to adjust its parameters from input-output pairs in order to approximate a target function. The perceptron fundamental limitations were exposed in 1969, showing that it could not learn functions as simple as the exclusive-or [Minsky and Papert, 1969]. At the same time, it was demonstrated that stacking multiple perceptrons could, in principle, represent such functions. However, no effective training procedure for these multi-layer architectures was available at the time. It was only in 1986



that researchers introduced gradient descent as an effective method for training deep NN [Rumelhart et al., 1986]. From this point onward, the emergence of DL is retrospectively situated, even though the term itself appeared later. DL is defined as the training of deep NN using gradient-based optimization methods.

It took several decades before DL reached the level of prominence it has today. One major obstacle was the absence of theoretical guarantees regarding the convergence of learning algorithms. Although the universal approximation theorem shows that NN can represent a wide class of functions [Hornik et al., 1989], it did not ensure that such representations could be effectively learned through optimization methods like gradient descent. This lack of rigorous guarantees initially reduced interest within the academic community. As a result, early progress relied largely on empirical breakthroughs.

These empirical advances were made possible by two key developments: the availability of large-scale datasets and significant improvements in computational infrastructure. DL methods require vast amounts of data to perform well, and the rise of the internet alongside the digitization of information enabled the creation of such datasets [Hilbert and López, 2011]. The use of graphics processing units for large-scale training, demonstrated in 2009, greatly increased computational capacity and made it feasible to train large-scale models [Raina et al., 2009, Krizhevsky et al., 2012].

At the same time, researchers developed architectures specifically designed for different types of data, allowing DL to address an increasingly wide range of problems. Convolutional Neural Network (CNN) were introduced in 1989 to process images, using local, weight-shared filters that exploit the spatial structure of pixel grids [LeCun et al., 1989]. Long Short-Term Memory (LSTM) networks were later proposed in 1997 to process temporal sequences, introducing gating mechanisms able to retain relevant information over long time horizons

[Hochreiter and Schmidhuber, 1997]. More recently, the Transformer architecture, relying entirely on the attention mechanism, was introduced in 2017 to process long, variable-length sequences of strongly interconnected elements, allowing every element to attend directly to every other element regardless of their distance within the sequence [Vaswani et al., 2017]. This growing body of knowledge was consolidated in 2016 in a reference textbook that formalized DL as a coherent field of study [Goodfellow et al., 2016].

These changes paved the way for the rise of DL as it is known today. DL has achieved remarkable success across a wide range of domains, including computer vision [Krizhevsky et al., 2012], complex games [Silver et al., 2017a], content recommendation [Covington et al., 2016], bio-chemistry [Jumper et al., 2021], as well as the now essential digital assistants based on Large Language Model (LLM) [Brown et al., 2020]. The diversity of these applications explains the enthusiasm for this technology, which continues to redefine the boundaries of what was once considered automatable.

Figure 1.1 provides a timeline of the key milestones that shaped deep learning.

1.2 Objective

Now that the main developments that led to the rise of DL have been outlined, it is necessary to formalize what is actually being achieved when performing DL. To ensure clarity, the supervised learning setting will be considered, specifically applied to a regression task. This framework serves as a standard reference in the field, as all building blocks of DL applications are built upon this fundamental formalism.

In this setting, learning consists of progressively shaping a NN that serves as an approximation of a target function. At initialization, the network does not yet provide a meaningful representation of this function. Since a NN is a parametric function, its parameters are progressively updated during training so as to make it a better approximation of the target function. It gradually becomes capable of capturing the relationship between inputs and outputs through successive parameter updates during training. For this reason, DL is naturally situated within the framework of machine learning. A standard definition of machine learning is:

“A computer program is said to learn from experience E with respect to some class of tasks T and performance measure P , if its performance at tasks in T , as measured by P , improves with experience E .”
[Mitchell, 1997]

98 When this formulation is adapted to DL, the following correspondences hold:

- 99 • The computer program corresponds to the learning algorithm, which is
100 responsible for adjusting the network's parameters.
- 101 • The task T consists of learning a NN that provides a good approximation
102 of the target function.
- 103 • The performance measure P corresponds to a function used to evaluate
104 the quality of this approximation.
- 105 • The experience E takes the form of a dataset composed of input–output
106 pairs derived from the function to be approximated.

107 The function to be approximated is denoted by f . As with any function, it
108 defines a mapping from an input to an output: $x \in \mathcal{X}$ denotes an input and
109 $y \in \mathcal{Y}$ the corresponding output. The function f is therefore characterized by an
110 input space $\mathcal{X}$ and an output space $\mathcal{Y}$, and can be formally written as $f : \mathcal{X} \rightarrow \mathcal{Y}$.
111 For simplicity of exposition, both $\mathcal{X}$ and $\mathcal{Y}$ are assumed to be vector spaces
112 throughout this manuscript. This assumption is adopted solely to facilitate the
113 presentation, as the framework can be readily extended to more general settings.
114 The notation $\hat{\cdot}$ indicates that a given object is an approximation of another.
115 Since the goal of the trained NN is to approximate f , this approximation is
116 denoted by $\hat{f} : \mathcal{X} \rightarrow \mathcal{Y}$. Given an input $x \in \mathcal{X}$, the output of the approximating
117 function $\hat{f}(x)$ is denoted by $\hat{y}$.

118 It is important to emphasize that the target function f is generally not known
119 explicitly. If it were, there would be no need to learn an approximation, since
120 the true function would already provide the exact mapping. In practice, only a
121 finite set of observations generated by f is available, organized in what is called
122 a dataset. To distinguish between the different samples, an index i is introduced,
123 allowing input–output pairs of the form $(x^{(i)}, y^{(i)})$ to be defined. This leads to
124 the following formulation for a dataset $\mathcal{D}$ of size N , where N denotes the number
125 of available examples:

$$\mathcal{D} = \{(x^{(i)}, y^{(i)})\}_{i=1}^N. \quad (1.1)$$

126 Now that the representation of the target function f has been established,
127 the next question is what makes a function $\hat{f}$ a good approximation of it. For
128 a given input $x^{(i)}$, this means that the NN output $\hat{y}^{(i)}$ should be close to the
129 true output $y^{(i)}$. To quantify the quality of this approximation, a function $\mathcal{L}$ is
130 introduced that measures the discrepancy between predictions and target values.
131 This function called the loss function plays a central role as it guides the learning
132 process. In regression settings, a common choice is the Mean Squared Error
133 (MSE), defined as:

$$\mathcal{L}(\hat{y}, y) = (\hat{y} - y)^2. \quad (1.2)$$

134 Thanks to this loss, it is now possible to rigorously define the performance
135 measure P . This performance measure corresponds to the average loss over the
136 dataset and can therefore be written as:

$$P = \frac{1}{N} \sum_{i=1}^N \mathcal{L}(\hat{f}(x^{(i)}), y^{(i)}). \quad (1.3)$$

137 This formulation captures the mathematical core of the learning objective,
138 namely finding a NN $\hat{f}$ that minimizes the prediction error over the dataset $\mathcal{D}$.

Since the target function f may take many different forms, $\hat{f}$ must be expressive enough to approximate a broad range of such functions. NN architectures are well suited to this requirement, as they can represent a wide range of functional mappings, whose composition is detailed in the following section.

1.3 Neural Network Structure

NN are called connectionist models. Rather than modeling information processing through a set of explicit rules, they rely on a flow of signals distributed within a computational structure. Their architecture is composed of a large number of simple computational units, which operate on partial representations and exchange signals across the network. A complex pattern of weighted connections between these units enables the transformation and propagation of information.

To simplify the presentation, the focus is placed on the MultiLayer Perceptron (MLP) architecture. Although more specialized architectures exist, the MLP forms a fundamental building block of DL. All the essential mechanisms are present in it, making it a particularly suitable framework for introducing key concepts.

This section therefore begins by examining the artificial neuron, the fundamental computational unit underlying the MLP. It then explains how these neurons are organized into layers. Finally, it concludes with a general description of the NN as a whole.

1.3.1 Neurons

The artificial neuron is the basic building block of NN. It is through these units that all computations within the network are carried out. A neuron receives a set of input signals from other neurons, where these inputs are represented as real-valued numbers. It produces an output, also a real-valued number, which is transmitted to subsequent neural units. Its internal state, called the activation, reflects its level of response and encodes the information processed for a given input. This activation, or lack thereof, is then used as input information by other neurons to determine their own activations, thereby propagating information through the network.

A neuron is characterized by:

- a set of incoming connections from other neurons,
- a weight associated with each of these connections,
- a bias term,
- a differentiable nonlinear activation function,
- a scalar activation state.

The activation of a neuron is determined by a weighted combination of its input signals, to which a bias is added, followed by the application of a nonlinear activation function. Mathematically, for a neuron receiving inputs $(a_1, \dots, a_n)$, its activation a is given by:

$$a = \sigma \left(\sum_{i=1}^n w_i a_i + b \right), \quad (1.4)$$

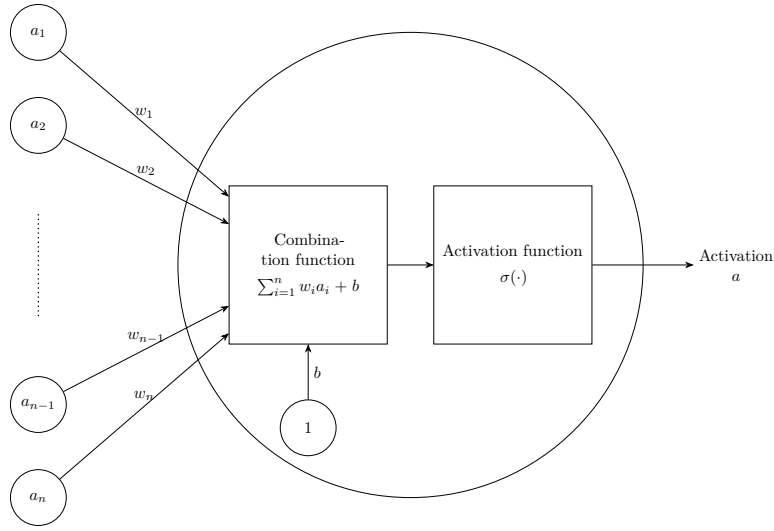


Figure 1.2: Artificiale neurone.

179 where:

- 180 • w_i is the weight associated with the i -th incoming connection,
- 181 • b is the bias,
- 182 • σ is the activation function.

183 Figure 1.2 shows a schematic view of this basic unit. The set of weights
 184 $w_1, \dots, w_i$ with the bias b constitute the parameters of the neuron, and they
 185 are the elements that evolve during training. The magnitude of a weight w ,
 186 relative to the others, indicates how strongly a neuron depends on the activation
 187 of the neurons to which it is connected. A large positive weight means that a
 188 high activation in the connected neuron tends to increase the activation of the
 189 receiving neuron, whereas a negative weight implies the opposite effect. In this
 190 way, the weights modulate the strength and nature of the connections between
 191 neurons, thereby defining the overall pattern of interaction within the network.

192 The activation function σ models how a neuron transforms incoming signals
 193 into its output activation. Originally, this function was inspired by biological
 194 neurons, with the goal of mimicking the firing behavior of organic cells. However,
 195 modern DL no longer relies on this biological interpretation. In practice, any
 196 function that is nonlinear, differentiable, and computationally efficient can be
 197 used. Nonlinearity is a crucial property, since without it a NN would reduce
 198 to a composition of linear transformations, and would therefore be unable to
 199 approximate complex functions.

200 1.3.2 Layers

201 In MLP architectures, neurons are organized into successive layers, as illustrated
 202 in Figure 1.3. This layered structure determines how neurons are connected. In
 203 this setting, each neuron receives inputs from all neurons in the preceding layer

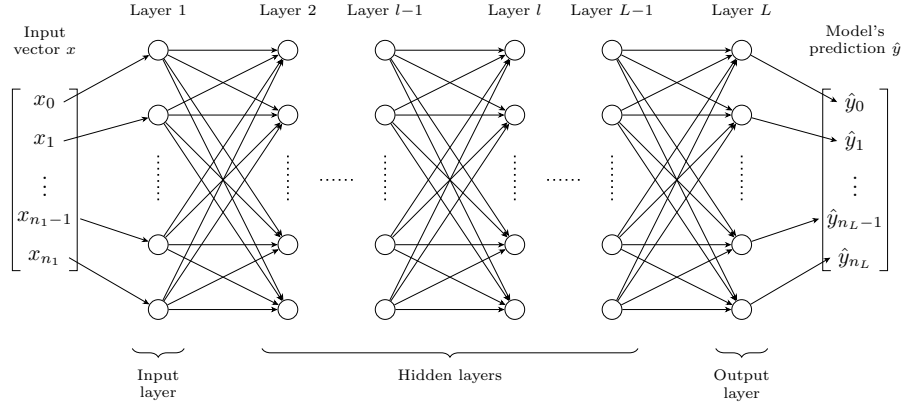


Figure 1.3: Multilayer Perceptron Architecture.

and sends its output to all neurons in the following layer. As a result, neurons within the same layer do not interact with each other.

Three types of layers are distinguished:

- an input layer, which directly receives input,
- one or more hidden layers (also referred to as intermediate layers), which perform intermediate transformations,
- an output layer, which produces the final prediction.

The input and output layers play a specific role. The input layer has no preceding layer, as its neurons directly encode the components of the input vector x . Conversely, the output layer has no subsequent layer, and its neurons produce the model's prediction $\hat{y}$. This structure imposes a direct correspondence between the dimensionality of the input space and the number of neurons in the input layer, and similarly between the output space and the number of neurons in the output layer.

When describing NN, especially in MLP, it is more common to reason at the level of layers rather than individual neurons. This perspective allows interactions between layers to be modeled in a matrix form. Such a representation enables efficient parallel computation and forms the basis of modern DL implementations [Krizhevsky et al., 2012]. Within this framework, it is useful to introduce the notion of layer activations. The activation of a layer corresponds to the activations of all its neurons, organized into a vector. For a layer l composed of n_l neurons, the activations are given by:

$$\mathbf{h}^{(l)} = [a_1^{(l)}, \dots, a_{n_l}^{(l)}]. \quad (1.5)$$

Using this formulation, the interaction between two consecutive layers can be written compactly as:

$$\mathbf{h}^{(l)} = \sigma^{(l)} \left(\mathbf{W}^{(l)} \mathbf{h}^{(l-1)} + \mathbf{b}^{(l)} \right), \quad (1.6)$$

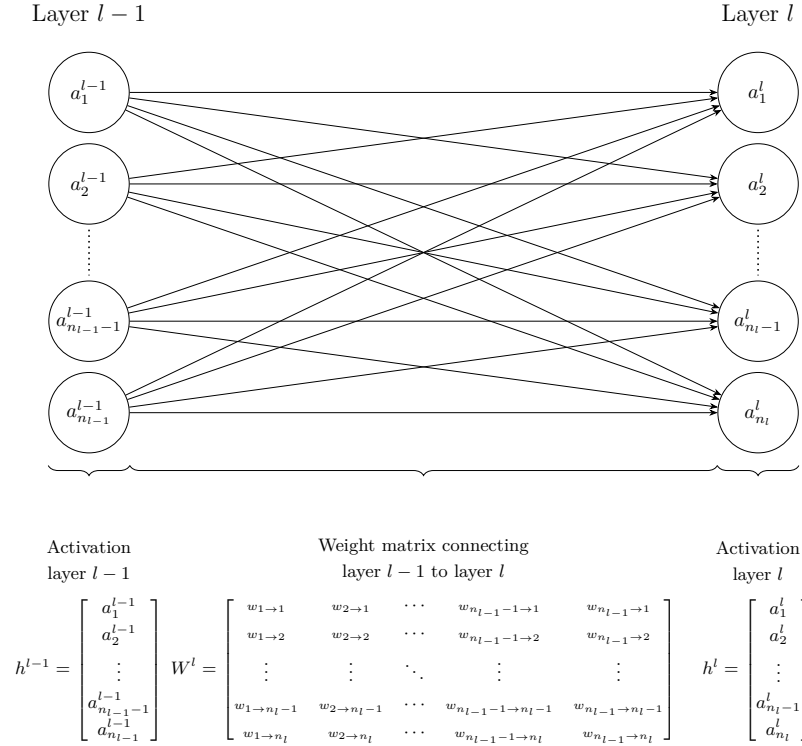


Figure 1.4: Interaction between layers.

where:

- $\mathbf{h}^{(l-1)}$ represents the activations vector of the previous layer,
- $\mathbf{W}^{(l)} \in \mathbb{R}^{n_l \times n_{l-1}}$ is the weight matrix connecting layer $l - 1$ to layer l ,
- $\mathbf{b}^{(l)} \in \mathbb{R}^{n_l}$ is the bias vector associated with layer l ,
- $\sigma^{(l)}$ is the activation function applied element-wise to the neurons of layer l .

This matrix-vector representation is visualised in Figure 1.4, which highlights how each neuron in layer l receives weighted signals from all neurons in the previous layer.

It is important to examine the role played by the intermediate layers of a NN. At least one hidden layer is required to represent sufficiently complex functions, and in practice, modern architectures often include many more. Theoretical results have shown that, for a fixed number of parameters, increasing the number of layers enhances the expressive capacity of NN [Telgarsky, 2016]. Each layer can be viewed as an additional stage in the computation performed by the network. This suggests that deeper networks can represent increasingly complex functions by composing a larger number of transformations. The precise relationship between depth and function complexity remains primarily supported by empirical observations rather than formal theoretical justification [Goodfellow et al., 2016].

1.3.3 Network

With a layer-wise representation, the propagation of information can be described as a sequence of transformations applied from the input layer (indexed 1) to the output layer (indexed L):

$$\hat{f}(x) = \mathbf{h}^{(L)} = \sigma^{(L)} \left(\mathbf{W}^{(L)} \sigma^{(L-1)} \left(\dots \sigma^{(1)} \left(\mathbf{W}^{(1)} x + \mathbf{b}^{(1)} \right) \dots \right) + \mathbf{b}^{(L)} \right) = \hat{y}. \quad (1.7)$$

For a more compact description of information propagation, the layer activations are defined recursively as:

$$\mathbf{h}^{(l)} = \begin{cases} x & \text{if } l = 1, \\ \sigma^{(l)} \left(\mathbf{W}^{(l)} \mathbf{h}^{(l-1)} + \mathbf{b}^{(l)} \right) & \text{if } 2 \leq l \leq L. \end{cases} \quad (1.8)$$

To simplify notation, it is convenient to group all parameters of the network into a single set. The model parameters are thus denoted by:

$$\theta = \{\mathbf{W}^{(l)}, \mathbf{b}^{(l)}\}_{l=1}^L. \quad (1.9)$$

In this manuscript, θ is a vector in $\mathbb{R}^P$, where P denotes the total number of parameters, such that $\theta = [w_1, \dots, w_P]$. The notation $\hat{f}_\theta$ expresses the dependence of the NN on its parameters. Increasing the number of parameters, either by adding more neurons per layer or by increasing the number of layers, can enhance the expressive power of the model, allowing it to approximate more complex functions. However, this also increases computational cost and training time.

It is important to distinguish between the architecture and the parameters of a NN. When referring to the architecture of $\hat{f}$, the number of layers, the number of neurons per layer, and the choice of activation functions are considered. The parameters correspond to θ , which controls the strength of the connections between neurons. The following sections examine how these parameters can be adjusted to obtain a better approximation of the target function.

1.4 Learning

Now that the structure of a NN has been detailed, the next step is to examine how its parameters are adjusted to accomplish the assigned task. This brings the discussion to the core of DL, namely the learning process itself. As a reminder, the objective of training a NN is to adjust its parameters so that the network provides a good approximation of the target function. Mathematically, this objective is expressed as:

$$\min_{\theta} \frac{1}{N} \sum_{i=1}^N \mathcal{L} \left(\hat{f}_\theta(x^{(i)}), y^{(i)} \right). \quad (1.10)$$

As shown in Equation 1.10, the loss for example i depends on the input-target pair $(x^{(i)}, y^{(i)})$, the loss function $\mathcal{L}$, the network architecture $\hat{f}$, and the parameters θ . Since the dataset, loss function, and network architecture are fixed

during training, only the parameters θ are optimized. To simplify the notation, only the dependence on θ is kept, and the loss for example i is written as

$$\mathcal{L}^{(i)}(\theta) = \mathcal{L}\left(\hat{f}_{\theta}(x^{(i)}), y^{(i)}\right). \quad (1.11)$$

Now that the learning objective has been defined, the next question is how to modify the parameters θ to minimize the loss. The NN architecture $\hat{f}$ used to approximate the target function f is highly complex. It involves a large number of parameters as well as many nonlinear components. As a result, the optimum cannot be determined analytically by directly studying the properties of the function.

Instead, approximate optimization methods are required to find suitable parameter values for NN. The class of algorithms used for this optimization is known as gradient descent methods. Numerous variants of gradient descent have been developed for NN training. For simplicity, the focus here is on Stochastic Gradient Descent (SGD) with a batch size of 1. As in previous cases, this choice is made because this setting captures the essential intuition behind the training method.

This section presents the learning process in three steps. The first derives the gradient of the loss using the chain rule. The second shows how this gradient is used to update the network parameters. The third discusses the convergence properties of the resulting algorithm.

1.4.1 Computing the Gradient

The first step of SGD is to randomly initialize the parameter vector $\theta \in \mathbb{R}^P$ for a given network architecture $\hat{f}$. This initial state is denoted by $\theta^{(1)}$. The goal is to iteratively update this vector so that the network gets better at approximating the target function. To do so, it is necessary to quantify the impact of each parameter w_k on the loss $\mathcal{L}^{(i)}(\theta)$. This is done using the partial derivative of the loss with respect to each parameter, denoted by $\frac{\partial \mathcal{L}^{(i)}(\theta)}{\partial w_k}$.

Computing this derivative is not straightforward because a NN is composed of a sequence of functions, and the parameter w_k can belong to any layer of the network. Its influence on the loss must therefore be propagated through all subsequent layers. To describe these dependencies, each layer can be viewed as a function, denoted by $\phi^{(l)}$ for layer l . This function takes the activation vector $\mathbf{h}^{(l-1)} \in \mathbb{R}^{n_{l-1}}$ produced by the previous layer as input. It produces the activation vector $\mathbf{h}^{(l)} \in \mathbb{R}^{n_l}$ of the current layer. This can be expressed as

$$\phi^{(l)}: \mathbb{R}^{n_{l-1}} \rightarrow \mathbb{R}^{n_l}, \quad \mathbf{h}^{(l)} = \phi^{(l)}(\mathbf{h}^{(l-1)}). \quad (1.12)$$

Each layer function can have multiple inputs and multiple outputs, so the ordinary derivative is not sufficient to describe how its outputs change with respect to its inputs. The Jacobian matrix extends the derivative to functions with multiple inputs and outputs, quantifying how each output component changes with respect to each input component. Consider the function $\phi^{(l)}$ and its Jacobian $J_{\phi^{(l)}}$ evaluated at $\mathbf{h}^{(l-1)}$. The Jacobian collects the partial derivatives

of each output component with respect to each input component:

$$J_{\phi^{(l)}}(\mathbf{h}^{(l-1)}) = \begin{bmatrix} \frac{\partial h_1^{(l)}}{\partial h_1^{(l-1)}} & \cdots & \frac{\partial h_1^{(l)}}{\partial h_{n_{l-1}}^{(l-1)}} \\ \vdots & \ddots & \vdots \\ \frac{\partial h_{n_l}^{(l)}}{\partial h_1^{(l-1)}} & \cdots & \frac{\partial h_{n_l}^{(l)}}{\partial h_{n_{l-1}}^{(l-1)}} \end{bmatrix}. \quad (1.13)$$

Since a NN is composed of several layers, a change in an early layer can affect the outputs of all subsequent layers. It is therefore necessary to propagate this change through the sequence of layer functions until it reaches the network output. The chain rule provides the mathematical tool required to describe this propagation. To illustrate this, consider two consecutive layers, $\phi^{(l)}$ and $\phi^{(l+1)}$. The chain rule states that the Jacobian of their composition is given by the product of their Jacobians. Each Jacobian is evaluated at the input received by the corresponding layer:

$$J_{\phi^{(l+1)} \circ \phi^{(l)}}(\mathbf{h}^{(l-1)}) = J_{\phi^{(l+1)}}(\mathbf{h}^{(l)}) J_{\phi^{(l)}}(\mathbf{h}^{(l-1)}). \quad (1.14)$$

The network is composed of a sequence of layer functions $\phi^{(1)}, \dots, \phi^{(L)}$. Each layer depends only on the output of the previous layer. Consider a parameter w_k belonging to layer l . A change in w_k first affects the output $\mathbf{h}^{(l)}$ of its own layer. This change then propagates through the subsequent layers, affecting $\mathbf{h}^{(l+1)}$, and so on. The effect reaches the output $\mathbf{h}^{(L)}$, from which the loss $\mathcal{L}^{(i)}(\theta)$ is computed. This layer-by-layer propagation is referred to as backpropagation:

$$\frac{\partial \mathcal{L}^{(i)}(\theta)}{\partial w_k} = J_{\mathcal{L}}(\mathbf{h}^{(L)}, y^{(i)}) J_{\phi^{(L)}}(\mathbf{h}^{(L-1)}) \cdots J_{\phi^{(l+1)}}(\mathbf{h}^{(l)}) J_{\phi^{(l)}}(w_k), \quad (1.15)$$

where:

- $J_{\mathcal{L}}(\mathbf{h}^{(L)}, y^{(i)})$ denotes the Jacobian of $\mathcal{L}$ with respect to its first argument, evaluated at $(\mathbf{h}^{(L)}, y^{(i)})$,
- each subsequent $J_{\phi^{(j+1)}}(\mathbf{h}^{(j)})$ denotes the Jacobian of the function $\phi^{(j+1)}$ with respect to its input $\mathbf{h}^{(j)}$,
- $J_{\phi^{(l)}}(w_k)$ denotes the Jacobian of the function $\phi^{(l)}$ with respect to the parameter w_k .

Once $\frac{\partial \mathcal{L}^{(i)}(\theta)}{\partial w_k}$ can be computed for every parameter, all of these values are stacked into one vector, called the gradient:

$$\nabla_{\theta} \mathcal{L}^{(i)}(\theta) = \left[\frac{\partial \mathcal{L}^{(i)}(\theta)}{\partial w_1}, \dots, \frac{\partial \mathcal{L}^{(i)}(\theta)}{\partial w_P} \right]. \quad (1.16)$$

Each component answers the same question for a different parameter, indicating whether a small increase in that parameter increases or decreases the loss, and by how much. A positive value means increasing the parameter increases the loss, whereas a negative value means the opposite. The gradient thus indicates, for every parameter, which direction to move in order to reduce the loss.

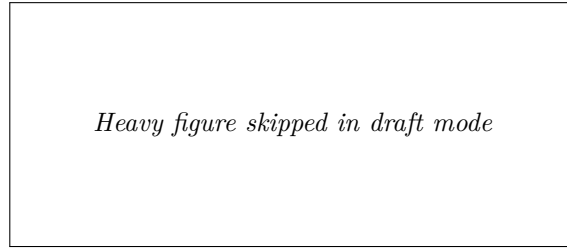


Figure 1.5: *Figure skipped in draft mode (set `\draftfiguresfalse` to render it).*

1.4.2 Update rule

Now that information is available on how changes in the parameters affect the loss, the objective is to minimize it. To do so, the parameters are updated in the direction opposite to the gradient. Indeed, the gradient is followed in the opposite direction because it indicates the direction in which the loss increases, whereas the objective is to reduce it. This leads to the gradient descent update rule:

$$\theta^{(n+1)} = \theta^{(n)} - \eta \nabla_{\theta} \mathcal{L}^{(i)}(\theta^{(n)}), \quad (1.17)$$

where $\eta > 0$ is the learning rate, which controls the step size, and $\nabla_{\theta} \mathcal{L}^{(i)}(\theta^{(n)})$ denotes the gradient of the loss with respect to the parameters, evaluated at the current values $\theta^{(n)}$.

The parameter η must remain small, as the gradient only provides reliable information in a local neighborhood of the current parameters. As a result, a single step is not sufficient to significantly reduce the loss, so the process is iterated many times. Producing a sequence of parameter vectors $\theta^{(1)}, \theta^{(2)}, \dots$ that progressively improve the model. In practice, training is stopped when successive updates no longer produce a significant decrease in the loss.

Algorithm 1 summarizes this whole training procedure.

To develop an intuition for this update rule, Figure 1.5 illustrates it applied to the simple convex loss function $\mathcal{L}(w_1, w_2) = w_1^2 + w_2^2$, where the red path shows the parameter updates converging to the minimum. The point $\theta^{(1)}$ represents the initial parameter values, with w_1 and w_2 randomly initialized, and the subsequent points, from $\theta^{(2)}$ to $\theta^{(6)}$, represent the successive updates produced by applying Equation 1.17, progressively moving the trajectory toward the minimum of the loss function.

1.4.3 Convergence Properties

Having established how SGD updates the parameters, a natural question is whether this procedure is guaranteed to converge. It has been mathematically shown that gradient descent applied to a convex loss with respect to the parameters converges to a global optimum, independently of the initialization. However, NN training involves highly non-convex loss surfaces, so the optimization trajectory becomes strongly dependent on the initial conditions, and no such guarantee holds. Figure 1.6 illustrates the effect of non-convex functions, different starting points lead to distinct local minima.

Algorithm 1: Stochastic Gradient Descent**Input:**Dataset $\mathcal{D} = \{(x^{(i)}, y^{(i)})\}_{i=1}^N$ Architecture $\hat{f}$ Loss function $\mathcal{L}$ Learning rate $\eta > 0$ Initial parameters θ (optional)**Output:**Trained parameters θ

```

1 if  $\theta$  is given then
2   |  $\theta^{(1)} \leftarrow \theta$ ;
3 else
4   | Initialize  $\theta^{(1)}$  randomly;
5  $n \leftarrow 1$ ;
6 while stopping criterion not satisfied do
7   | Select a random example  $(x^{(i)}, y^{(i)}) \in \mathcal{D}$ ;
8   | Model prediction for the selected example
   |  $\hat{y}^{(i)} \leftarrow \hat{f}_{\theta^{(n)}}(x^{(i)})$ ;
9   | Per-example loss between the prediction and the ground truth
   |  $\mathcal{L}^{(i)}(\theta^{(n)}) \leftarrow \mathcal{L}(\hat{y}^{(i)}, y^{(i)})$ ;
10  | Gradient of the per-example loss, since it is differentiable
   |  $g \leftarrow \nabla_{\theta} \mathcal{L}^{(i)}(\theta^{(n)})$ ;
11  | Error minimization via a gradient descent step
   |  $\theta^{(n+1)} \leftarrow \theta^{(n)} - \eta g$ ;
12  |  $n \leftarrow n + 1$ ;
13 Return  $\theta^{(n)}$ 

```

380 Despite the lack of theoretical guarantees of convergence to a global optimum,
381 the simple procedure of random initialization followed by iterative gradient-based
382 updates is sufficient in practice to train complex NN.

383 1.5 Evaluation

384 Now that the process of training a model has been described, the question of
385 how to evaluate it must be addressed. In the supervised learning setting, this
386 evaluation relies on two aspects:

- 387 • its performance on the data it was trained on,
- 388 • its performance on data it was not trained on.

389 The first aspect measures how well the loss minimization process has suc-
390 ceeded. The second measures how well the model generalizes. Indeed, the dataset
391 $\mathcal{D}$ represents a small subset of all possible inputs. To estimate how the model

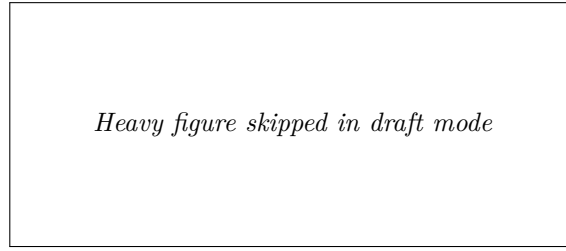


Figure 1.6: *Figure skipped in draft mode (set `\draftfiguresfalse` to render it).*

will perform in general, it is important to evaluate how it performs on data it has not seen during training.

In order to quantify these two aspects, the dataset is split into two subsets:

- $\mathcal{D}_{\text{train}}$ is the set on which the model minimizes its loss during the training.
- $\mathcal{D}_{\text{test}}$ is kept aside during training in order to evaluate the model's ability to generalize.

When training is completed, the first step is to verify that the model performs well on the data it was trained on. To do so, the average loss on the training set, denoted $\bar{\mathcal{L}}_{\text{train}}$, is studied. This is an instance of the performance measure P defined in Equation (1.3), restricted to $\mathcal{D}_{\text{train}}$:

$$\bar{\mathcal{L}}_{\text{train}} = \frac{1}{|\mathcal{D}_{\text{train}}|} \sum_{(x^{(i)}, y^{(i)}) \in \mathcal{D}_{\text{train}}} \mathcal{L}(\hat{f}_{\theta}(x^{(i)}), y^{(i)}). \quad (1.18)$$

If $\bar{\mathcal{L}}_{\text{train}}$ is not sufficiently low, this means that the model has not been able to learn the task. It is not necessary to go further in the analysis of the model's quality. Before restarting the training process, various design choices are then adjusted. Based on what has been discussed so far in this chapter, these include modifications to the number of layers, the number of neurons per layer, activation functions, the learning rate. However, in practice, there are many other possible variations. There is no universal rule for choosing the appropriate value for each of them. They are therefore chosen empirically, based on models that perform well on similar tasks, combined with intuition. Their selection is often costly and remains a challenging problem in DL.

Assuming now that the model achieves satisfactory performance on the training dataset $\mathcal{D}_{\text{train}}$. Its ability to generalize must now be studied. To do so, the corresponding average loss on $\mathcal{D}_{\text{test}}$ is computed:

$$\bar{\mathcal{L}}_{\text{test}} = \frac{1}{|\mathcal{D}_{\text{test}}|} \sum_{(x^{(i)}, y^{(i)}) \in \mathcal{D}_{\text{test}}} \mathcal{L}(\hat{f}_{\theta}(x^{(i)}), y^{(i)}). \quad (1.19)$$

If $\bar{\mathcal{L}}_{\text{test}} \gg \bar{\mathcal{L}}_{\text{train}}$, the model fails to generalize. This situation most often arises due to a phenomenon known as overfitting. In such cases, the model does not learn general patterns but instead memorizes the training examples. This typically occurs when the number of parameters is too large relative to

the amount of training data. The simplest remedy is therefore to reduce the number of parameters in the model. However, it is still necessary to ensure that the model remains sufficiently expressive to perform well on the data.

If $\bar{\mathcal{L}}_{\text{test}} \approx \bar{\mathcal{L}}_{\text{train}}$, this indicates that the training procedure has successfully produced a NN that generalizes well beyond the training data [Kawaguchi et al., 2022]. The resulting model can therefore be considered suitable for the task for which it was trained, marking the end of the NN training process.

1.6 Conclusion

This chapter has introduced the fundamental concepts underlying DL, including the general objective of NN models, their structure, the training procedure, and their evaluation. For clarity of presentation, several simplifying assumptions have been made throughout this chapter. The problem has been considered in a regression setting, using a MLP trained with SGD and a batch size of one. These choices were made deliberately to expose the core mechanisms of DL as clearly as possible.

Despite these simplifications, the framework introduced here captures mechanisms that are shared across a broad range of DL methods: A sequence of parameter updates, each based on information about the loss around the current parameters. The apparent simplicity of this learning procedure therefore contrasts with the complexity of the functions that NN can represent. Understanding how such complex behavior emerges from a relatively simple optimization process remains an important challenge in the study of DL.

Mathematical tools allow us to describe the functions represented by NN and the procedures used to train them. However, they do not yet fully explain their remarkable empirical performance. Understanding the origin of this success therefore requires looking beyond the optimization procedure itself.

A common feature across NN architectures is the presence of intermediate representations. Whether considering a MLP, a CNN, an LSTM, or an architecture based on attention, the input is progressively transformed into intermediate representations before producing the final output. Their presence provides a common perspective for understanding how NN transform information. The collection of these intermediate representations is referred to as the Latent Space (LS). Studying this LS can provide insights into how NNs learn and generalize, which motivates the following chapter.

Chapter 2

Latent Space Analysis

In classical machine learning approaches, practitioners had to manually design data representations in order to make them usable by simple models. Deep Learning (DL) introduces a paradigm shift: rather than relying on hand-crafted features to meaningfully represent data, it automatically learns these representations. Through the successive activation of hidden layers, Neural Network (NN) gradually construct representations that become increasingly relevant to the task under consideration [Rumelhart et al., 1986].

These representations are not meaningful at initialization, they emerge progressively during training as the model optimizes its objective. It is through these learned representations that a trained network is able to map an input to an output, including for examples never encountered during training. Since these representations are learned internally by the network and are not directly observable from the input or output spaces, they are commonly referred to as Latent Representation (LR). The collection of these LR learned throughout the network is referred to as a Latent Space (LS).

These LS are now at the core of a growing number of practical applications. Neural video compression exploits the compactness of LS to encode information efficiently [Lu et al., 2019]. In Reinforcement Learning (RL), agent internal representations are used for efficient exploration in extremely open environments [Burda et al., 2018]. Recommendation systems model users and content within a shared LS to compute meaningful similarities [He et al., 2017]. Retrieval-augmented generation systems index and query knowledge bases through LR [Lewis et al., 2021]. Finally, Large Language Model (LLM) and multimodal architectures rely on the quality of learned representations to align heterogeneous modalities such as text, images, and audio [Radford et al., 2021]. These examples represent only some of the most widespread applications of LR.

Given their extensive use, the study of LR has become a central topic in DL research. A major objective is to understand how information is organized within these spaces, which concepts are encoded, and how these LR influence model behavior.

The remainder of this chapter is organized as follows:

- Section 2.1 introduces the hypothesis that underlies much of the research on LS analysis, which we refer to as the *Concept Hypothesis* (Hyp. 1).
- Sections 2.2, 2.3, 2.4, and 2.5 examine four families of LS analysis methods, respectively based on probing, interpretable directions, feature superposition, and clustering.
- Section 2.6 concludes with a comparative synthesis of these methods and a discussion of the open questions that remain.

2.1 Concept Hypothesis

In this section, we introduce the main working hypothesis underlying much of the literature on LS analysis. Although rarely stated explicitly, this hypothesis is implicitly present in the recurring use of notions such as abstraction, representation, LS, concept, and semantics. Throughout this manuscript, we refer to this hypothesis as *Concept Hypothesis* (Hyp. 1). The goal of this section is to make this conceptual framework explicit by defining its main components.

The *Concept Hypothesis* (Hyp. 1) originates from the need to explain the generalization ability of deep NN observed after training, as discussed in Section 1.5. A model is said to generalize when, for an input $x \in \mathcal{X}$ not seen during training, it still produces an output $\hat{y}$ close to the true output y . Such behavior cannot be explained by simple memorization of training samples. Instead, the model must extract and exploit underlying regularities in the target function. This requires a process of abstraction, as defined in Definition 2.1.1, whereby raw inputs are transformed into internal structures that are meaningful for the task [Bengio et al., 2014].

Definition 2.1.1: Abstraction

Abstraction is the process by which a model transforms an input into a task-relevant internal form. It captures and restructures information in order to represent relationships that are useful for prediction.

The ability of NN to perform abstraction arises from their intermediate activations. A NN does not directly map x to $\hat{y}$, it computes a sequence of intermediate activations across hidden layers. These activations are numerical encodings of the input referred to as embeddings. Once training is complete, these embeddings are structured to support the performance of a specific task. To mark this distinction, we will refer to these trained embeddings as LR, as defined in Definition 2.1.2.

Definition 2.1.2: Latent Representation

A LR is the internal encoding of an input given by an intermediate layer of a trained NN.

Each hidden layer refines the LR of the previous layer. Each transformation progressively reshapes the LR into a form more directly aligned with the output

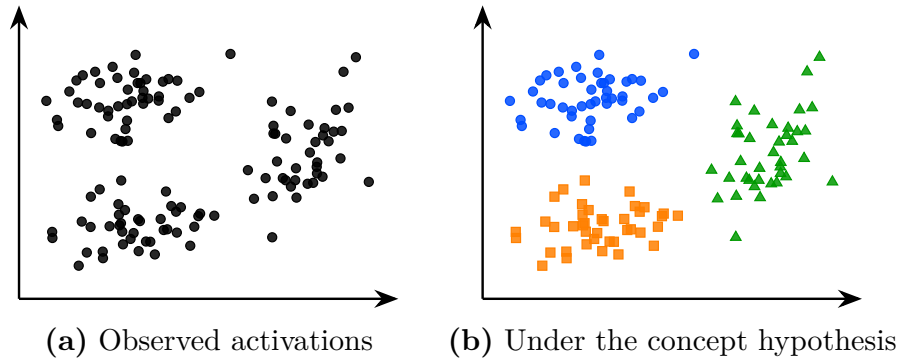


Figure 2.1: The same latent space activations seen without (a) and under (b) the *Concept Hypothesis* (Hyp. 1).

space [Rumelhart et al., 1986]. This process can be described as a sequence of transformations where each $\mathbf{h}_i$ is a LR:

$$x \rightarrow \mathbf{h}_1 \rightarrow \mathbf{h}_2 \rightarrow \cdots \rightarrow \mathbf{h}_{L-1} \rightarrow \hat{y}$$

The set of all such LR produced by a layer over a dataset defines its LS, as defined in Definition 2.1.3.

Definition 2.1.3: Latent Space

The LS is the set of all LR produced by a layer over a dataset. It is shaped by learning and organizes LR according to task-relevant properties.

Under the *Concept Hypothesis* (Hyp. 1), the analysis of the LS is assumed to provide access to abstract structures learned by the model, as illustrated in Figure 2.1. The same set of LR can be viewed either as an unstructured point cloud or, under this hypothesis, as a collection of points organized by concept membership, in which case the structure of the LS becomes meaningful and amenable to analysis. In Figure 2.1, the colors represent hypothetical concept memberships assigned by the model. In this manuscript, a concept is defined as an abstract representation that organizes information and supports reasoning (see Definition 2.1.4).

Definition 2.1.4: Concept

A concept is a task-dependent abstraction that captures regularities in the input space relevant for predicting the target output.

Our definition of concepts is consistent with both the pragmatic tradition in philosophy and the information-processing view of cognition. Within these frameworks, concepts arise from the need to simplify an otherwise intractable reality. Because the environment contains more information than any cognitive system can process, intelligent behavior relies on abstractions, called concepts, that discard irrelevant details while preserving the information required to achieve a given objective [Peirce, 1878, Misak, 2013]. These concepts are realized as

representations that reduce complexity and enable reasoning under limited computational resources [Newell, 1980, Simon, 1996]. Although these theories were originally developed to explain human cognition, the *Concept Hypothesis* (Hyp. 1) assumes that the same principles can be extended to deep NN.

In order to influence the model’s computations, concepts must necessarily admit a concrete realization within the LS. We refer to this realization as the Concept Structure (CS). The definition of the CS is provided in Definition 2.1.5.

Definition 2.1.5: Concept Structure

A CS is the specific form for how a concept is instantiated within the LS.

The methods reviewed in this chapter all build on the *Concept Hypothesis* (Hyp. 1) formalized in Hypothesis 1, but differ in how they characterize the CS of a concept. This choice directly determines how concepts can be identified and extracted from the LS. As a result, several families of LS analysis methods have emerged in the literature, each adopting a different view of how concepts are structured within the LS.

Hypothesis 1: Concept Hypothesis

NN generalize by performing abstraction. Through training, their LS becomes organized around concepts. Concepts are task-dependent abstractions that capture regularities relevant to the task. Analyzing the LS therefore provides access to the CS through which these concepts are instantiated in the model.

Now that the definition of a *Concept Hypothesis* (Hyp. 1) has been established, we can consider its relationship to human. According to our definition, we can imagine concepts that are useful for the machine but do not make sense to humans. This relationship is captured by the notion of semantics [Marconato et al., 2023], defined in Definition 2.1.6. Some concepts developed in LS can be readily associated with human concepts and are said to have high semantic value. Others may contribute to task performance while remaining difficult for humans to understand, and are said to have low semantic value.

Definition 2.1.6: Semantics

Semantics refers to the alignment between concepts in the model LS and concepts used by a human observer. A concept is semantic if it can be associated with a human interpretable concept.

It is important to note that this conceptual framework is not supported by a complete theoretical understanding of deep NN. Current mathematical tools do not provide a rigorous characterization of their internal representations and do not formally prove that concepts emerge as identifiable CS within LS. Nevertheless, this perspective constitutes a widely adopted working hypothesis in the field of representation and LS analysis.

The remainder of this chapter is devoted to reviewing methods for analyzing LS and extracting the concepts they encode. We begin with probing, one of the most direct approaches for assessing whether concepts are represented in the LS.

2.2 Probing

Probing methods aim to determine whether a given semantic concept is encoded within the LS of a NN [Li et al., 2024, Karvonen, 2024]. The idea behind these methods is that if a concept can be reliably decoded from LR, it is likely relevant to the problem being solved. Studying such concepts can therefore provide insight into how the network internally represents the problem.

A probing analysis begins by identifying a set of semantic concepts that the network could potentially use to accomplish its task. Each example in the dataset is then annotated to indicate the presence or absence of the studied concepts. We denote by c_j the presence or absence of concept j in example x . The dataset can therefore be written as follows:

$$\mathcal{D} = \{(x^{(i)}, y^{(i)}, c_1^{(i)}, \dots, c_n^{(i)})\}_{i=1}^N.$$

The data are subsequently projected into the LS of the pretrained model. For an input x , we write $\mathbf{h}_l(x) \in \mathbb{R}^{n_l}$, or simply $\mathbf{h}_l$, the LR obtained at layer l . A classifier, referred to as a probe, is then trained to predict the presence of a given concept from this LR. It is denoted by $p_\theta^{l,j}$, where θ represents its parameters, l indicates the layer from which the LR is extracted, and j identifies the concept being predicted. The probe maps the LR to a scalar value in $[0, 1]$, which can be interpreted as the probability that the concept is present. The parameters θ are learned by minimizing a loss function over the training examples using Stochastic Gradient Descent (SGD). This gives the following mathematical formulation:

$$p_\theta^{l,j} : \mathbb{R}^{n_l} \rightarrow [0, 1], \quad \min_{\theta} \frac{1}{N} \sum_{i=1}^N \mathcal{L} \left(p_\theta^{l,j} \left(\mathbf{h}_l(x^{(i)}) \right), c_j^{(i)} \right). \quad (2.1)$$

The architecture of the probe must be kept intentionally simple. A probe with excessive expressive capacity may be able to predict the concept by exploiting complex patterns within the representation. These patterns do not necessarily indicate that the concept is explicitly encoded in the LS. Instead, they may result from the ability of the probe itself to combine and transform the available information. Consequently, a highly expressive probe makes it difficult to distinguish between information that is directly accessible in the representation and information that is reconstructed by the classifier. Restricting the capacity of the probe therefore helps ensure that its predictions primarily reflect the information already present in the LS.

For this reason, probing methods often rely on linear classifiers. A linear separation suggests that the information is easily accessible to the NN and potentially usable during decision-making. If the classifier achieves performance significantly above chance level, this suggests that the LS contains exploitable information related to the considered concept.

Figure 2.2 illustrates the decision boundaries of such linear probes for three concepts, where each colored region indicates the area classified positively by the corresponding probe.

An application of this technique can be found in studies investigating whether LLM develop internal representations of their environment [Li et al., 2024, Karvonen, 2024]. In these works, researchers train LLM architecture to play games such as chess or Othello in a purely textual setting. The model is provided

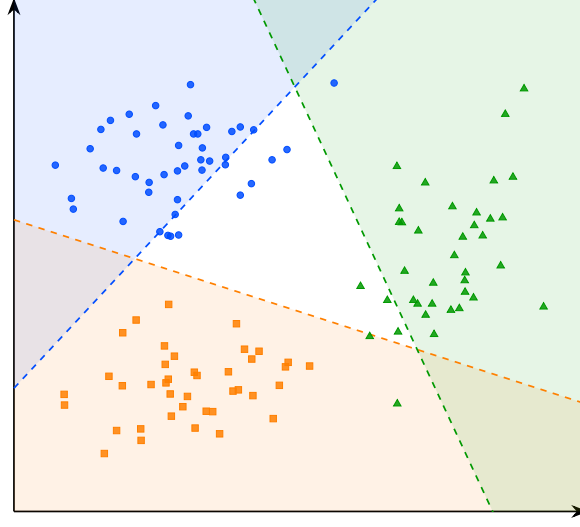


Figure 2.2: Decision boundaries of the linear probes for concepts A, B, and C.

only with sequences of moves and information about the actions it can take, all expressed in textual form. The central question is whether the LLM develops an internal representation of the game state. To address this question, the probed concepts correspond to the presence or absence of different game pieces on specific board positions.

As illustrated in Figure 2.3, probes are able to reconstruct the full board state from the model’s LR, even though the only available information to the model is a sequence of text. In this figure, each row corresponds to a piece type, comparing the ground truth with the probe’s predictions and confidence. The close agreement between the ground truth and predicted positions suggests that the model develops a structured representation of the underlying environment from which the data are generated.

However, the fact that a concept can be decoded from LR does not necessarily imply that the model actually uses this information to perform its task. To assess whether LR play a causal role in the model’s output, it is possible to perform interventions on them. An intervention consists of modifying a LR in a controlled manner by activating or deactivating a given concept, and then studying its impact on the model’s output. If the resulting change in the model’s output is consistent with the applied perturbation, this suggests that the representation had a causal role.

To this end, the LR is minimally modified such that the probe prediction for the target concept c_j changes state, while the predictions associated with other concepts remain unchanged. This goal can be formulated as an optimization problem over the LR itself. The optimization is performed directly on the LR, using SGD. The solution to this problem is the optimal modified LR, defined as:

$$\mathbf{h}_l^* = \arg \min_{\mathbf{h}_l'} \underbrace{\gamma \mathcal{L}_{\text{flip}}(p_{\theta}^{l,j}(\mathbf{h}_l'), c_j)}_{\text{flip target concept } c_j} + \underbrace{\lambda \sum_{k \neq j} \mathcal{L}_{\text{preserve}}(p_{\theta}^{l,k}(\mathbf{h}_l'), c_k)}_{\text{preserve other concepts } c_{k \neq j}} + \underbrace{\beta \|\mathbf{h}_l' - \mathbf{h}_l\|_2^2}_{\text{minimality}}$$

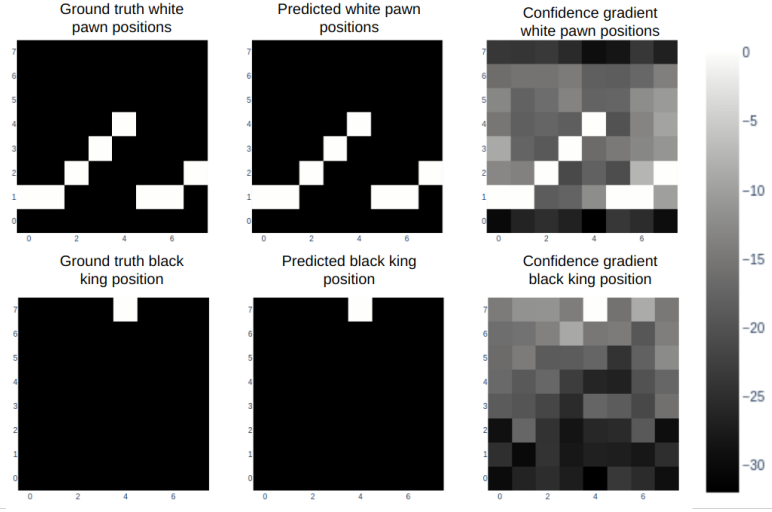


Figure 2.3: Probing analysis of a large language model trained to play chess [Karvonen, 2024].

where:

- $\mathbf{h}_l$ is the original LR of the input at layer l ,
- $\mathbf{h}'_l$ is the modified LR being optimized,
- $\mathbf{h}^*_l$ is the optimal modified LR,
- $\mathcal{L}_{\text{flip}}$: loss encouraging the prediction of the target concept c_j to change state (i.e., flipping the predicted label),
- $\mathcal{L}_{\text{preserve}}$: loss enforcing invariance of all non-target concepts c_k for $k \neq j$,
- $\|\mathbf{h}'_l - \mathbf{h}_l\|_2^2$: regularization term ensuring the modified LR remains close to the original one,
- γ, λ, β : weighting coefficients controlling the trade-off between the intervention strength, the preservation of other concepts, and the minimality of the perturbation.

In the context of LLM studied previously, such interventions have been used to remove specific pieces from the model’s internal representation of the game state [Li et al., 2024, Karvonen, 2024]. The main observation is that, in the vast majority of cases, the model behaves as if the piece had effectively disappeared. The LLM continues to generate legal moves while correctly accounting for the absence of the removed piece. This provides evidence for the causal role of these internal representations in the model’s behavior.

A main limitation of the probing approach is that the concepts under investigation must be predefined before any analysis can be performed. This introduces a strong bias in what can be studied. Moreover, annotating a dataset for each concept is extremely time-consuming and requires significant human effort. Interventions in LS are also difficult to interpret, as it is not always clear

what constitutes a meaningful change in the network’s output under a given perturbation.

Moreover, probing methods provide a binary view of concepts, focusing on whether a given concept is present or absent in a LR. This binary perspective can be overly restrictive, as concepts may be represented in a more continuous manner. To obtain a more detailed characterization of these representations, the next section turns to methods that describe a concept as a direction in LS along which it varies continuously.

2.3 Directions

Probing describes a concept as a binary attribute, either present or absent in a LR. The methods in this section instead associate each concept with a direction in the LS. Moving a LR along this direction is expected to strengthen or weaken the concept smoothly. This provides information that a binary probe cannot express.

Empirically, it has been observed that shifting a LR along such a direction can induce coherent and semantically meaningful changes in the model’s output [Haas et al., 2024, Voynov and Babenko, 2020]. Such transformations can be mathematically express as:

$$\mathbf{h}' = \mathbf{h} + \alpha \mathbf{v}_i,$$

where:

- $\mathbf{h}$ denotes the original LR,
- $\mathbf{v}_i$ is a unit vector associated with concept i ,
- α controls the strength of the intervention,
- $\mathbf{h}'$ is the resulting perturbed LR.

In many cases, the resulting change in the model’s output varies smoothly with α , suggesting an approximately linear CS with respect to certain semantic factors. This observation has led to the hypothesis that neural representations may organize concepts in a partially linear manner within the LS.

Vectors of this form are referred to as interpretable directions, as they correspond to transformations that can be associated with specific semantic attributes. The objective of this line of work is therefore to identify directions in LS that align with meaningful variations in the encoded concepts. This is illustrated schematically in Figure 2.4, where arrows represent semantic shifts from one concept toward others, and moving along these directions induces a coherent change in the model’s behavior, suggesting that the transition between concepts is continuously encoded along these axes.

This phenomenon has been extensively studied in generative image models. As illustrated in Figure 2.5, where each row corresponds to a distinct direction $\mathbf{v}_i$ and each column to an increasing value of α , controlled perturbations of LR along interpretable directions can induce specific semantic transformations, such as stroke thickness modification in digit images, head rotation in facial images, or texture and background changes in natural images [Voynov and Babenko, 2020]. These observations have led to the development of the field of LS editing, whose

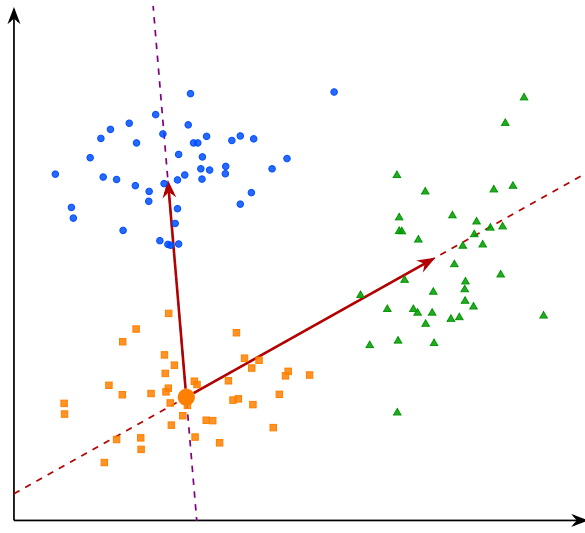


Figure 2.4: Interpretable directions in the latent space.

objective is to manipulate generated content directly through modifications in LS rather than through changes in the input prompt.

A central challenge remains identifying such interpretable directions. Most existing approaches rely on auxiliary models that encode human defined semantics, such as CLIP [Haas et al., 2024], to relate latent variations to textual concepts. Consequently, these methods inherit a limitation similar to that of probing techniques, since concept discovery depends on a pre-existing semantic framework defined by humans.

To overcome this limitation, a research direction known as unsupervised semantic discovery has emerged. Its goal is to uncover interpretable directions in LS without explicit semantic supervision. In these approaches, one model applies perturbations along random directions in LS, while a second model attempts to recognize the applied transformation. If a stable correspondence emerges between the perturbation and its prediction, this suggests that the considered direction corresponds to a coherent CS [Voynov and Babenko, 2020]. In this framework, human intervention occurs only a posteriori, to verify whether the discovered directions indeed correspond to interpretable semantic variations.

Despite these advances, several challenges remain. It appears that not all concepts encoded by NN are linearly identifiable in LS. In practice, interpretable direction methods typically discover only a few dozen directions per dataset, which intuitively seems insufficient to account for the full diversity of semantic variations that a generative model is capable of producing.

In addition, LR often exhibit a phenomenon known as entanglement, illustrated in Figure 2.6, where multiple semantic attributes are mixed within the same directions of the LS. For instance, in a generative face model, moving along the “glasse” direction may simultaneously modify age and gender, whereas a disentangled LR would let each axis control a single semantic attribute while leaving the others unchanged. This lack of separability suggests that semantic information is not cleanly organized in the LS. This observation motivates the *Superposition Hypothesis* (Hyp. 2), which proposes a different view of how NN



Figure 2.5: Interpretable directions in the latent space of a generative model [Voynov and Babenko, 2020].

739 organize information internally. The next section introduces this hypothesis.

740 2.4 Superposition

741 The previous section highlighted a limitation of interpretable directions. Only a
 742 limited number of concepts can be represented by a linear direction, and these
 743 directions are often entangled with multiple semantic concepts. To account for
 744 this, an extension of the *Concept Hypothesis* (Hyp. 1) has been proposed under
 745 the name *Superposition Hypothesis* (Hyp. 2), formalized in Hypothesis 2 [Elhage
 746 et al., 2022].

Hypothesis 2: Superposition Hypothesis

Concepts are preferentially represented by distinct linear directions in LS.
 When the number of concepts exceeds the available dimensions, multiple
 concepts are encoded in superposition within the same directions.

747

748 According to this hypothesis, in an unconstrained LS, each concept is repre-
 749 sented in its own independent linear direction. The idealized non-superposed

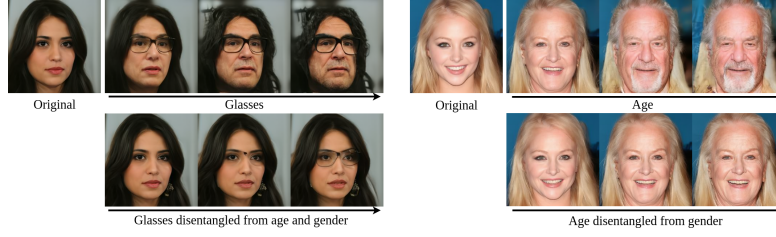


Figure 2.6: Illustration of entanglement in the latent space of a generative face model [Haas et al., 2024].

LR can be mathematically expressed as follows:

$$\mathbf{h} = \sum_{i=1}^n \alpha_i \mathbf{v}_i, \quad \forall i \neq j, \quad \mathbf{v}_i^\top \mathbf{v}_j \approx 0, \quad \|\boldsymbol{\alpha}\|_0 \ll n,$$

where:

- $\mathbf{h}$ denotes a LR,
- n is the total number of concepts,
- $\mathbf{v}_i$ represents the direction associated with concept c_i ,
- α_i corresponds to the activation strength of concept c_i in the LR $\mathbf{h}$,
- $\mathbf{v}_i^\top \mathbf{v}_j \approx 0$ indicates that concept directions are approximately orthogonal, meaning each concept independently controls a single factor of variation,
- $\|\boldsymbol{\alpha}\|_0 \ll n$ indicates that only a small fraction of concepts are active for a given LR.

However, NN operate in finite-dimensional spaces, so when the number of concepts exceeds the available dimensions, such a one-to-one correspondence becomes impractical. The network must therefore reuse its representational dimensions, allowing multiple concepts to be encoded using the same directions. This shared encoding is referred to as superposition [Bricken et al., 2023]. This phenomenon becomes an obstacle for any method that attempts to analyze LR directly in the original activation space.

Building on this observation, several works based on the *Superposition Hypothesis* (Hyp. 2) have developed methods that aim to recover less superposed LR. A common approach is to map LS into a higher-dimensional space while imposing additional constraints that encourage individual concepts to be represented separately. This transformation is illustrated in Figure 2.7, while LR are entangled in the initial LS, the sparse representation aligns each concept with a dedicated feature axis.

SAE provide a practical implementation of this approach and can be defined as follows:

$$z = f_{\text{enc}}(\mathbf{h}), \quad \hat{\mathbf{h}} = f_{\text{dec}}(z)$$

$$\mathcal{L}_{\text{auto}} = \gamma \|\mathbf{h} - \hat{\mathbf{h}}\|_2^2 + \lambda \|z\|_1$$

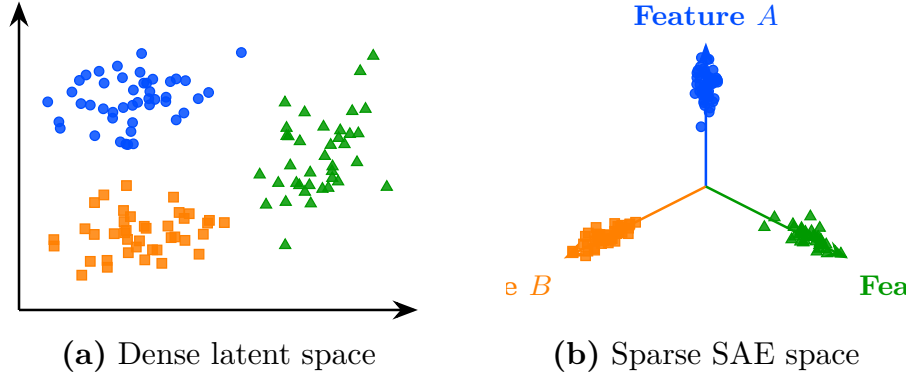


Figure 2.7: Transformation from a dense latent space (a) to a sparse Sparse Autoencoder (SAE) representation (b).

where:

- $f_{\text{enc}}(\mathbf{h})$ is the encoding function that projects the original LR $\mathbf{h}$ into a higher-dimensional sparse space,
- $f_{\text{dec}}(z)$ is the decoding function that reconstructs the original LR from the sparse code,
- $z \in \mathcal{Z}$ is the sparse representation of $\mathbf{h}$, with $\dim(z) \gg \dim(\mathbf{h})$,
- $\hat{\mathbf{h}}$ is the reconstructed LR,
- $\|\mathbf{h} - \hat{\mathbf{h}}\|_2^2$ is the reconstruction loss ensuring that the original LR can be faithfully recovered,
- $\|z\|_1$ is the sparsity regularization term that encourages most components of z to be zero,
- γ, λ are weighting coefficients controlling the trade-off between reconstruction fidelity and sparsity.

The parameters of these projection functions f_{enc} and f_{dec} are learned by minimizing $\mathcal{L}_{\text{auto}}$ using SGD. Under this formulation, each dimension of $\mathcal{Z}$ is interpreted as a dedicated concept direction.

In practice, both f_{enc} and f_{dec} are implemented as linear projections between the original LS and the high-dimensional space $\mathcal{Z}$. This choice is motivated by the same principle underlying probing methods. The transformation applied to the LR should remain as simple as possible. Using more expressive projection functions could introduce additional structures into the transformed LR. In this case, it would become difficult to determine whether an observed concept was already encoded in the model’s LR, or whether it was introduced by the projection itself. Restricting f_{enc} and f_{dec} to linear projections therefore helps ensure that the concepts identified in $\mathcal{Z}$ can be attributed to the original LR, rather than to the transformation used to analyze it.

The method proves to be powerful when successfully applied to language models [Templeton et al., 2024]. In these applications, SAE are used to desuperpose the LR of LLM. Once this LR is obtained, it has been shown that many



Figure 2.8: A concept direction corresponding to the Golden Gate Bridge, discovered by a sparse autoencoder applied to a large language model [Templeton et al., 2024].

concept vectors exhibit semantic properties. A example is the Golden Gate Bridge concept direction, illustrated in Figure 2.8, which activates systematically whenever the input text or image refers to the Golden Gate Bridge, regardless of the language or modality. This demonstrates that the learned concept is not tied to a specific surface form but captures a deeper semantic abstraction shared across modalities.

SAE also provide a way to intervene directly on the LR learned by the model. A concept can be artificially activated by modifying its corresponding feature in the sparse representation z , even when the input contains no reference to it. The modified LR is then projected back into the original activation space, after which the model proceeds with standard next-token generation from $\mathbf{h}^*$:

$$\mathbf{h}^* = f_{\text{dec}}(z^*), \quad \text{where} \quad z^* = z + \delta e_i,$$

where:

- z is the original sparse representation of the input,
- z^* is the modified sparse representation,
- e_i is the basis vector of $\mathcal{Z}$ associated with the target concept direction,
- δ controls the strength of the artificial activation,
- $\mathbf{h}^*$ is the resulting edited LR in the original activation space.

With this method, a reference to the Golden Gate Bridge can be injected into the LLM’s internal representation. This causes the model’s output to shift toward discussing the Golden Gate Bridge, even when the prompt contains no reference to it. This provides evidence that the learned feature plays a causal role in the model’s behavior. Such interventions enable control over specific model behaviors through LS editing.

Despite their empirical success, *Superposition Hypothesis* (Hyp. 2) exhibit several important limitations that question their assumptions. These limitations can be summarized as follows:

- **Instability of the decomposition.** Repeating the SAE training process multiple times on the same model does not yield the same decomposition of the LS. Different runs produce different concept directions in $\mathcal{Z}$, even when achieving comparable reconstruction performance, which raises questions about the reliability of the discovered CS.
- **Lack of semantic coverage.** In the high-dimensional space $\mathcal{Z}$, a large fraction of dimensions do not correspond to any interpretable semantic concept. This abundance of non-semantic directions raises questions about whether the decomposition truly reflects the model’s CS.
- **Sparsity–semantics trade-off.** Increasing the sparsity coefficient λ beyond a certain threshold degrades the semantic quality of the learned features [Jermyn et al., 2024]. Suggesting that enforcing excessive sparsity comes at the cost of interpretability.
- **Failure of scaling to remove superposition.** Under the *Superposition Hypothesis* (Hyp. 2), increasing the dimensionality of the LS should reduce superposition. However, this is not observed in practice. Even in very high-dimensional LS, representations remain superposed, indicating that scaling alone does not resolve the entanglement.

These limitations suggest that *Superposition Hypothesis* (Hyp. 2), while powerful, do not fully resolve the problem of CS. Convincing empirical results do not necessarily validate the underlying theoretical assumptions.

So far, every method described, whether based on probing, interpretable directions, or superposition, relies on a common assumption. The CS can be recovered from LR through a linear decomposition. This assumption has enabled substantial progress in understanding neural representations, but it also imposes a strong constraint on how concepts are expected to be represented. The following section examines clustering as an alternative approach that does not rely on this assumption.

2.5 Clustering

As the previous section noted, every LS analysis method presented so far rests on a linear view of the CS. For probing methods, concepts must be at least partially linearly separable for simple probes to detect them. Interpretable direction methods posit that variations associated with a concept can be captured by moving along a linear axis of the LS. The *Superposition Hypothesis* (Hyp. 2) proposes that concepts are represented through linear directions that become entangled because of representational capacity constraints. Although these assumptions have led to empirical successes, several observations suggest that the linearity hypothesis may not fully capture the CS.

For instance, some concepts can only be recovered using non-linear probes. Likewise, methods based on interpretable directions typically identify only a limited number of semantic factors. Finally, in sparse autoencoder approaches,

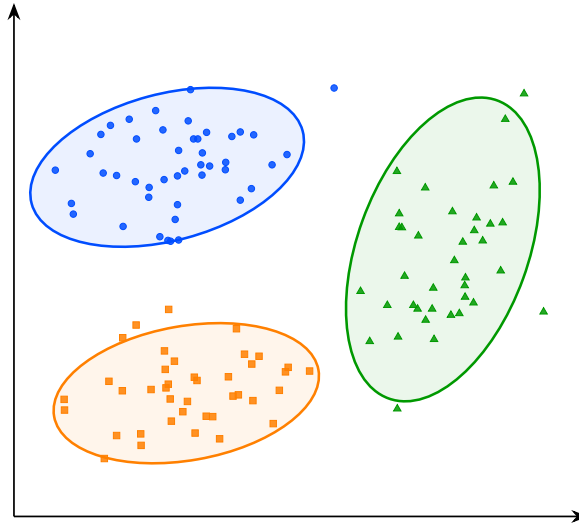


Figure 2.9: Visualization of the latent space structure.

enforcing excessive sparsity degrades interpretability. This suggests that the underlying representation may not be perfectly described by a sparse linear decomposition. Taken together, these observations indicate that semantic concepts may not always exhibit a linear CS. This motivates the exploration of alternative approaches that rely on weaker assumptions regarding the geometry of LS.

To the best of our knowledge, clustering-based analysis is the only family of LS analysis methods that does not rely on any assumption of a linear CS [Ren et al., , Wei et al.,]. Rather than assuming that concepts correspond to directions, clustering methods only assume that conceptual similar inputs produce nearby LR. Conversely, inputs that differ significantly from a concept perspective are expected to be located farther apart in the LS. This is a considerably weaker assumption, in the sense that it is empirically verified in a wide range of settings.

Under this assumption, the discovery of concepts can be reformulated as the identification of groups of LR that occupy the same region of the LS. This is shown schematically in Figure 2.9, where the points represent LR associated with concepts, and the ellipses indicate the approximate extent of each group and highlight their geometric separation.

This idea forms the basis of the research field known as deep clustering. In these approaches, a collection of inputs is projected into a LS using a NN. The resulting LR form a point cloud on which standard clustering algorithms can be applied. The obtained clusters are then interpreted as representing distinct concepts.

Deep clustering has been particularly successful in unsupervised image classification. For this type of task, a specific NN architecture called an autoencoder is commonly used. An autoencoder is composed of an encoder f_{enc} and a decoder f_{dec} . The encoder maps an input $x \in \mathcal{X}$ to a low-dimensional LR $\mathbf{z} \in \mathbb{R}^d$, the LR produced by the encoder. The decoder then maps this LR back to the input space to produce a reconstruction $\hat{x}$. The model is trained by SGD to minimize the reconstruction error between the original input and its reconstruction using a Mean Squared Error (MSE) loss. This objective encourages the LS to

903 preserve the most informative features of the input while enforcing a compact
 904 representation. This can be expressed mathematically as follows:

$$x \xrightarrow{f_{\text{enc}}} \mathbf{z} \xrightarrow{f_{\text{dec}}} \hat{x}, \quad d \ll \dim(\mathcal{X}), \quad \mathcal{L}_{\text{rec}} = \frac{1}{N} \sum_{i=1}^N \|x^{(i)} - \hat{x}^{(i)}\|_2^2.$$

Once the autoencoder has been trained sufficiently well, the reconstruction loss $\mathcal{L}_{\text{rec}}$ should be low enough for the learned representation to be useful. A clustering algorithm can then be applied to the compressed representations of the images. Let $\mathcal{D}$ denote a dataset of inputs, through the encoder f_{enc} , this dataset is mapped to a point cloud in the LS:

$$\mathcal{Z} = \{\mathbf{z}^{(i)} \in \mathbb{R}^d \mid \mathbf{z}^{(i)} = f_{\text{enc}}(x^{(i)}), x^{(i)} \in \mathcal{D}\}.$$

905 A classical clustering algorithm is applied to the set $\mathcal{Z}$ of LR to identify
 906 groups of similar LR. Each resulting cluster can therefore be interpreted as a
 907 group of images that the model considers similar.

908 An example of this methodology can be observed in Figure 2.10, where the
 909 approach is applied to the STL-10 dataset [Coates et al., 2011] using k -means
 910 clustering [MacQueen, 1967] with a number of clusters arbitrarily fixed to $K = 10$
 911 [Xie et al.,]. For each cluster, the corresponding line shows the 10 samples
 912 closest to the associated centroid in the LS.

913 We observe that the method captures coherent semantic categories such
 914 as animals, vehicles, and flying entities. This result is particularly striking
 915 given that the only available information to the system consists of raw images.
 916 It suggests that the compression process induces a LS in which semantically
 917 meaningful CS emerge.

918 More advanced approaches combine representation learning and clustering
 919 within a single optimization process. In this setting, additional loss terms
 920 encourage LR to organize themselves around cluster centroids during training.
 921 This often produces LS that are easier to analyze and improves the quality of
 922 the discovered semantic CS [Guo et al., , Miklautz et al.,].

923 Despite its effectiveness, clustering-based analysis presents several important
 924 limitations.

925 First, as with most unsupervised semantic discovery techniques, interpreting
 926 the resulting clusters remains partly subjective. Benchmark datasets often
 927 provide class labels for evaluation, but the discovered clusters may not match
 928 the categories that humans consider relevant.

929 Second, clustering methods are highly sensitive to design choices, as the
 930 number of clusters, the distance metric, and the clustering algorithm itself can
 931 all significantly influence the results, making parameter selection challenging
 932 and often requiring substantial empirical tuning.

933 Finally, to the best of our knowledge, no intervention technique has yet been
 934 proposed in the context of clustering-based analysis. Unlike probing or SAE
 935 analysis, which allow targeted modifications of LR to study their causal role,
 936 clustering methods currently offer no mechanism to alter the model’s behavior
 937 by acting directly on the discovered CS.

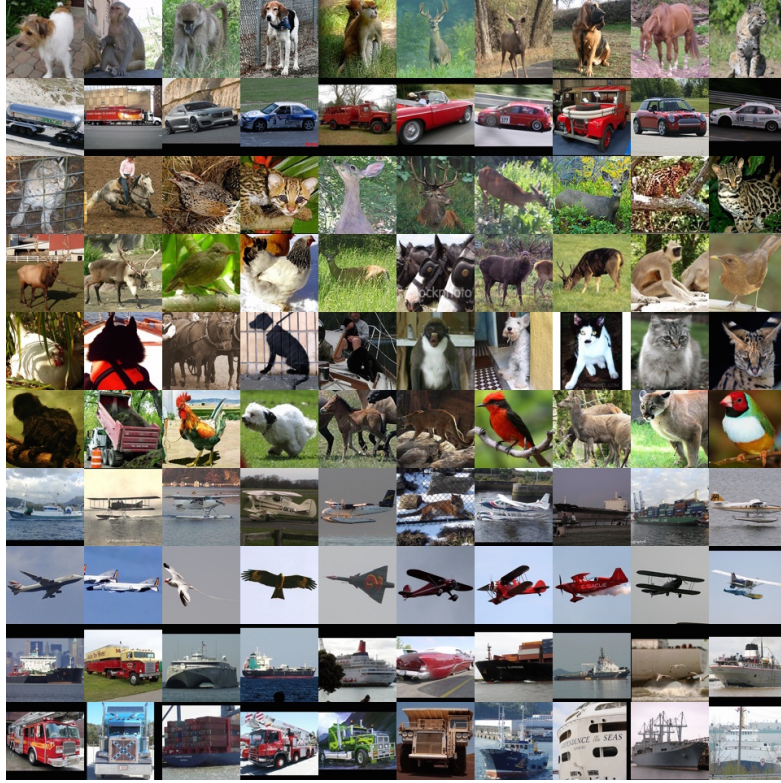


Figure 2.10: Clusters discovered by a deep clustering model applied to natural images [Xie et al.,].

2.6 Conclusion

The four families of methods reviewed in this chapter are summarized in Figure 2.11. They include probing, interpretable directions, sparse autoencoders, and clustering, all of which build upon the *Concept Hypothesis* (Hyp. 1) introduced in Section 2.1. Each family approaches the analysis of LR from a different angle. However, they share a common goal of understanding how LR are structured and what information they contain.

However, a fundamental difficulty remains. There is currently no consensus on how to rigorously evaluate whether these methods truly capture the internal concepts of a model. Most existing metrics assess reconstruction quality or linear separability, but these do not directly measure whether the discovered CS correspond to meaningful abstractions from the model’s perspective. This leaves open the question of what it means, formally, for a concept to be represented in a NN.

Addressing this question may require contributions from disciplines beyond computer science and mathematics. As the notion of semantics is inherently tied to human interpretation, a purely formal or computational account may be insufficient. Insights from philosophy, cognitive science, or psychology could prove valuable in grounding the notion of concept in a way that is both theoretically rigorous and empirically tractable. Despite these open questions, the methods

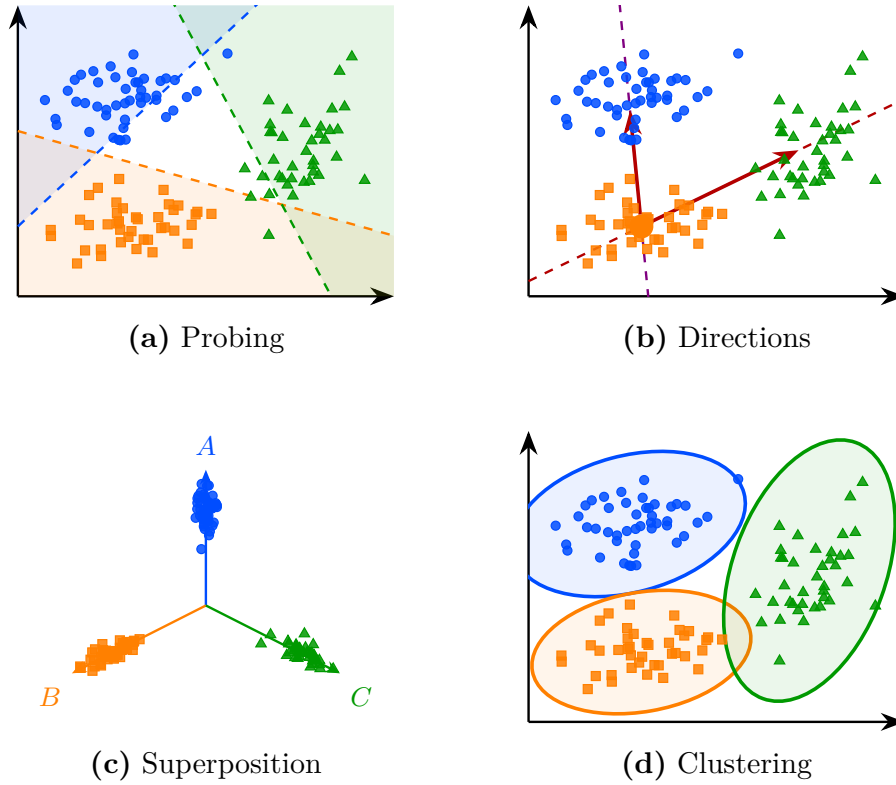


Figure 2.11: Overview of the four latent space analysis methods studied in this chapter, applied to the same point cloud.

958 studied in this chapter provide a practical foundation for analyzing the internal
 959 representations of NN.

960 Building on the *Concept Hypothesis* (Hyp. 1) and the insights developed
 961 throughout this chapter, this manuscript investigates a representation-based
 962 approach to explainability in RL. To establish the necessary foundations, the
 963 next chapter introduces the principles of RL. The subsequent chapter then
 964 presents the proposed method for analyzing these representations and studying
 965 their potential for explainability in RL (Chapter 5).

Chapter 3

Reinforcement Learning

This chapter introduces the fundamental concepts of Reinforcement Learning (RL), providing the necessary foundation for the ideas developed throughout the rest of this manuscript. Its structure is as follows:

Section 3.1: An overview of the milestones that led to the emergence of RL as a unified field.

Section 3.2: A clarification of what is meant by an agent and how it relates to the RL algorithm.

Section 3.3: A formalization of the problem that RL algorithms aim to solve.

Section 3.4: An explanation of how policies are progressively refined through interaction with the environment to approach the optimal policy.

Section 3.5: A summary of the key ideas and a discussion of the broader significance of RL.

3.1 Historical Developments

RL is today presented as a major field within artificial intelligence. However, it originates from several research traditions that progressively converged toward a common framework. In this section, we outline the main stages in the emergence of this field, highlighting the key ideas and milestones that shaped its evolution.

3.1.1 Behavioral foundations (1900–1950)

The origins of RL can be traced back to 20th-century animal psychology studies [Thorndike, 1911]. These studies aimed to understand how behaviors are formed and modified. The central idea emerging from this work is that learning is driven by the consequences of actions. Behaviors followed by positive outcomes are more likely to be repeated, whereas those associated with negative outcomes tend to gradually decrease in frequency.

This perspective was later developed within behaviorism. In this school of psychology, rewards and punishments are used to gradually shape behavior

through repeated interaction with the environment [Skinner, 1938]. The term “reinforcement” in the context of learning comes from this school of thought, where it refers precisely to the process of strengthening a behavior through its consequences.

This idea led to the suggestion that learning principles observed in living organisms could be transferred to artificial systems. Machines were therefore envisioned as potentially learning through reward-like mechanisms, guided by evaluative feedback signals resembling a “pleasure–pain system” [Turing, 1948]. Although these studies did not involve formal mathematical models, they introduced the central idea of RL, trial-and-error learning guided by environmental feedback.

3.1.2 Sequential decision-making (1950–1970)

A second foundation of RL comes from optimal control theory. This field studies how to determine sequences of decisions that optimize a cumulative performance criterion over time. In this context, the decision-making entity is called an agent. The agent interacts with the environment by selecting actions. The function that maps an observed state to the chosen action is called a policy. The objective is to find an optimal policy, in the sense that it maximizes the expected cumulative reward over time.

This framework led to the formalization of Markov Decision Processes (MDP), which provide a mathematical model for sequential decision-making [Howard, 1960]. MDP describe a agent interacting with an environment through a set of states, actions, rewards, and transition rules. The earliest method proposed to solve MDP is dynamic programming [Bellman, 1957]. It proceeds by breaking down a complex decision problem into smaller, more manageable subproblems, solving each one, and combining the results to obtain an overall solution. However, dynamic programming requires a full description of the environment, which is only feasible for small problems.

3.1.3 Emergence of reinforcement learning (1970–2000)

In order to address this issue, temporal-difference learning methods were introduced [Sutton, 1988]. This mechanism enables incremental learning directly from experience without requiring a complete model of the environment. It can be interpreted as a sample-based approximation of the Bellman equations.

Building on these ideas, the notion of value functions emerged as a way to estimate the long-term benefit of being in a given state or taking a given action. These functions assign a numerical score to states or state-action pairs, reflecting the expected cumulative reward an agent can obtain from that point onward. They thus provide a compact way to evaluate and compare different courses of action. This line of work culminated in Q-learning, which showed that optimal action-value functions can be learned directly from interaction with the environment, without requiring a model of its dynamics [Watkins, 1989, Watkins and Dayan, 1992].

The publication of the foundational reference work [Sutton and Barto, 1998] contributed to the formalization of RL as a coherent field. From that point onward, the field can be considered well established and widely recognized as a distinct area of research.

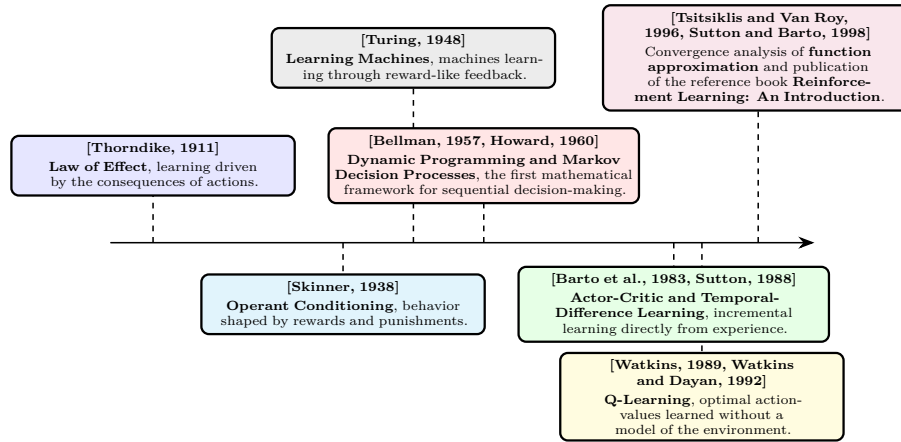


Figure 3.1: Evolution of reinforcement learning prior to its fusion with deep learning.

3.1.4 Consolidation of reinforcement learning (2000–2012)

A limitation of early algorithms such as Q-learning is their reliance on explicit state-action tables. Although these methods remove the need to know the full dynamics of the MDP, they still require storing a value for each possible state-action pair. As a result, the agent must effectively visit and maintain estimates for a potentially enormous number of states in the environment. This requirement becomes infeasible in large-scale MDP, where the state space is too large to be exhaustively explored or stored in memory.

To address this issue, research shifted toward more general representations of value functions and policies. Instead of maintaining explicit tables, function approximation methods were introduced. These allowed value functions and policies to be represented using parametric models such as linear approximators or perceptron-like architectures [Barto et al., 1983, Tsitsiklis and Van Roy, 1996]. This change enabled generalization across states, reducing the dependence on exhaustive exploration of the state space. These parametric representations nonetheless remained limited in expressiveness. The next step consisted in replacing such approximators with Neural Network (NN), which would soon reshape the field.

Figure 3.1 provides a timeline of the key milestones that shaped the development of reinforcement learning prior to its fusion with deep learning.

3.1.5 Deep reinforcement learning (2013–2026)

The introduction of Deep Learning (DL) into RL marked a major turning point in the field. This shift was initiated by Deep Q-Networks (DQN), which used a deep NN to approximate action-value functions at scale. This made it possible to train an agent to play Atari games at a human level directly from raw pixels [Mnih et al., 2013]. The use of deep NN provided RL with a powerful and flexible function approximator. This removed an important practical limitation by making it possible to efficiently represent the complex functions required by RL.

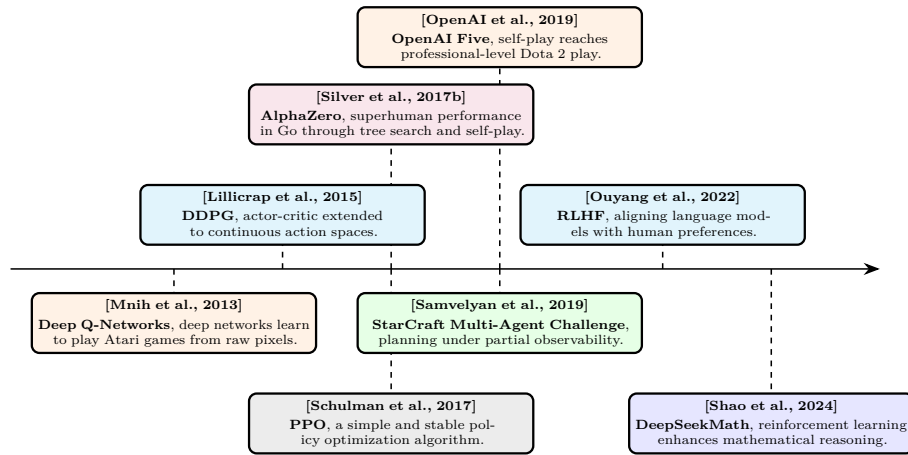


Figure 3.2: Evolution of deep reinforcement learning.

A series of subsequent algorithms further extended and improved this approach. Deep Deterministic Policy Gradient (DDPG) extended actor-critic methods to continuous action spaces [Lillicrap et al., 2015]. Proximal Policy Optimization (PPO) later became a widely adopted algorithm in RL due to its simplicity, stability, and ability to handle both continuous and discrete action spaces [Schulman et al., 2017].

These advances established a foundation for large-scale policy optimization. Notably, AlphaZero demonstrated that the combination of deep NN, RL, and tree search could reach superhuman performance in the game of Go [Silver et al., 2017b]. This success was later extended to even more complex real-time strategy environments such as StarCraft II, where agents must handle long-term planning, partial observability, and large action spaces [Samvelyan et al., 2019]. Similarly, in Dota 2, large-scale multi-agent RL systems such as OpenAI Five showed that self-play combined with deep policy optimization can achieve professional-level performance in highly dynamic and stochastic environments [OpenAI et al., 2019].

More recently, RL has expanded beyond its traditional application domains. With the emergence of increasingly agentic views of Large Language Model (LLM), it has become possible to train these models to perform specific tasks using RL-based techniques. In particular, RL from human feedback has become a method for aligning LLM with human preferences. It does so by formalizing the problem as an MDP and using RL to solve it [Ouyang et al., 2022]. Similarly, DeepSeekMath shows how RL can enhance the mathematical reasoning abilities of LLM [Shao et al., 2024]. It demonstrates that an artificial agent can learn to reason in natural language in order to accomplish a given task. The skills acquired during this learning process also transfer to other similar tasks, leading to improved overall performance. For instance, training a LLM with RL on mathematical problem-solving can also enhance its programming capabilities.

Figure 3.2 provides a timeline of the key milestones that shaped the emergence of deep reinforcement learning.

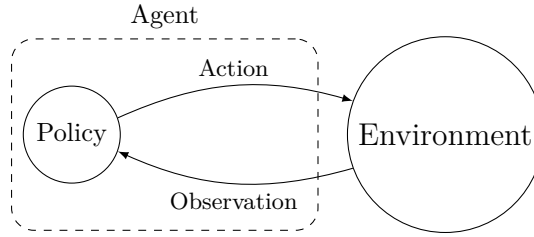


Figure 3.3: The agent paradigm.

3.2 Agent Paradigm

The RL framework is built around the agent paradigm, in which the decision-making entity is referred to as the agent. A standard definition is:

“An agent is anything that can be viewed as perceiving its environment through sensors and acting upon that environment through actuators.”
[Russell and Norvig, 2021]

In the RL setting, this definition translates into an interaction between the agent and its environment. The agent perceives its environment through observations. These observations are then processed by a decision-making mechanism referred to as the policy. The policy determines how the agent modifies its environment through the actions it selects. Figure 3.3 illustrates this interaction.

The objective of an RL agent is to maximize the cumulative reward it receives through its interactions with the environment. To achieve this objective, the agent’s policy must progressively improve through experience. The procedure responsible for this learning process is the RL algorithm, which progressively modifies the policy based on the experience collected through interaction.

In this manuscript, the RL algorithm is considered as a process external to the agent. Other presentations of RL describe the agent as both the learner and the decision-making entity [Sutton and Barto, 1998]. We chose to distinguish the agent from the RL algorithm because this provides a more general definition of an agent.

An agent can therefore exist without learning, for example when its behavior is entirely specified by a set of programmed rules. Such an agent still perceives, processes information, and acts upon its environment, but its behavior is not the result of RL.

This distinction also makes the definition consistent with modern implementations of RL [Espeholt et al., 2018, Liang et al., 2018], in which the acting policy and the learning algorithm are separate software components. Several copies of the policy, each interacting with its own instance of the environment, collect experience in parallel, while a single learning algorithm uses this experience to update the shared policy parameters.

1132 3.3 Problem Formulation

1133 Having outlined the main developments shaping RL and introduced the concept
 1134 of the agent, the RL problem can now be stated formally. In essence, RL aims to
 1135 build a agent that makes sequential decisions in order to maximize a cumulative
 1136 reward signal over the long term. The purpose of this section is to translate
 1137 this intuitive goal into a mathematical framework. To this end, the presentation
 1138 proceeds step by step:

- 1139 1. Modeling the RL problem as a Markov Decision Process.
- 1140 2. Defining the notion of a policy, which formalizes the agent’s behavior.
- 1141 3. Describing the interaction between the agent and the environment through
 1142 trajectories.
- 1143 4. Introducing value functions to evaluate policies.
- 1144 5. Defining the notion of an optimal policy.

1145 3.3.1 Markov Decision Processes

1146 To specify the task and the agent’s capacity to perceive and act, the problem is
 1147 modeled as a MDP.

1148 Before providing a more detailed definition, it is useful to clarify the scope
 1149 of the MDP problems considered in this manuscript. We therefore restrict the
 1150 analysis to the following configuration:

- 1151 • The single-agent learning setting is considered. If any other agents are
 1152 present in the environment, they are treated as part of the environment
 1153 dynamics.
- 1154 • Finite-horizon MDP are considered. Each interaction sequence between the
 1155 MDP and the policy has a defined beginning and a defined end. Between
 1156 these two points, the number of interaction steps may vary depending on
 1157 the situation, but it always remains finite.
- 1158 • The partially observable framework is adopted, where the agent does
 1159 not have direct access to the true state of the system, but instead relies
 1160 on limited observations that provide only partial information about the
 1161 environment.

1162 The resulting framework can be formally described as a single-agent finite-
 1163 horizon partially observable Markov decision process. For simplicity, we will
 1164 refer to this setting as an “MDP” throughout this manuscript.

1165 A MDP is defined by the tuple $(\mathcal{S}, \mathcal{O}, \mathcal{A}, \mathcal{R}, p)$:

- 1166 • $\mathcal{S}$: The set of states, representing all possible configurations of the en-
 1167 vironment. A state belonging to this set is denoted by $s \in \mathcal{S}$. Among
 1168 these, two special subsets are distinguished: initial states, from which an
 1169 interaction can begin, and terminal states, in which the interaction ends.
 1170 The set of initial states is denoted by $\mathcal{S}^{\text{init}}$, with an initial state written as
 1171 $s^{\text{init}} \in \mathcal{S}^{\text{init}}$. The set of terminal states is denoted by $\mathcal{S}^{\text{ter}}$, with a terminal
 1172 state written as $s^{\text{ter}} \in \mathcal{S}^{\text{ter}}$.
- 1173 • $\mathcal{O}$: The set of observations that the agent can perceive from the environ-
 1174 ment. A observation belonging to this set is denoted by $o \in \mathcal{O}$. Unlike
 1175 the state $s \in \mathcal{S}$, which is hidden from the agent, the observation $o \in \mathcal{O}$
 1176 provides partial information about the current state.
- 1177 • $\mathcal{A}$: The set of actions that the agent may execute. A action belonging to
 1178 this set is denoted by $a \in \mathcal{A}$. Actions influence the dynamics of the MDP,
 1179 thereby shaping its future course. They represent the agent's capacity to
 1180 act upon its environment.
- 1181 • $\mathcal{R}$: The set of rewards that the agent can receive from the environment.
 1182 A reward belonging to this set is denoted by $r \in \mathcal{R}$. $\mathcal{R}$ is a subset of $\mathbb{R}$.
 1183 Rewards can be either positive or negative, depending on whether the
 1184 corresponding outcome is considered beneficial or detrimental for the agent.
 1185 The agent's goal is to maximize the cumulative sum of these rewards over
 1186 time.
- 1187 • p : The transition function, which defines the dynamics of the environment.
 1188 Given a state s and an action a , this function defines a probability distribu-
 1189 tion over the joint outcome consisting of the next state s' , the observation
 1190 o , and the reward r . This function is denoted by p , and is defined as:
 1191 $p : \mathcal{S} \times \mathcal{A} \rightarrow \mathcal{P}(\mathcal{S} \times \mathcal{O} \times \mathcal{R})$. The notation $(s', o, r) \sim p(s, a)$ means that
 1192 the tuple (s', o, r) is sampled from the distribution induced by $p(\cdot \mid s, a)$.
 1193 The notation $p(s', o, r \mid s, a)$ denotes the probability of observing the tuple
 1194 (s', o, r) under the distribution $p(\cdot \mid s, a)$.

1195 The two quantities, observation o and state s , are not independent. They are
 1196 jointly generated via $(s', o, r) \sim p(s, a)$, which implicitly defines the relationship
 1197 between them. In our MDP formalization, this relationship is left implicit
 1198 because it is never manipulated directly during learning.

1199 However, to express mathematical equations, it is convenient to introduce
 1200 the conditional distribution over states given an observation. We denote this
 1201 distribution by $b : \mathcal{O} \rightarrow \mathcal{P}(\mathcal{S})$. The notation $b(s \mid o)$ denotes the probability of
 1202 state s given the observation o , while $s \sim b(\cdot \mid o)$ indicates that the state s is
 1203 sampled from this distribution.

1204 3.3.2 Policy

1205 The agent's decision-making capability is modeled by a policy function. The
 1206 policy defines the stochastic mapping between observations $o \in \mathcal{O}$ and actions
 1207 $a \in \mathcal{A}$. A policy function is denoted by π , and is defined as: $\pi : \mathcal{O} \rightarrow \mathcal{P}(\mathcal{A})$. The
 1208 notation $a \sim \pi(\cdot \mid o)$ means that the action a is sampled from the distribution

induced by $\pi(\cdot | o)$. The notation $\pi(a | o)$ denotes the probability of selecting action a under the distribution $\pi(\cdot | o)$.

It is important to note that this policy does not include any explicit mechanism for estimating the underlying state from the observations. Instead, the policy must learn from experience how to infer the information about the underlying state that is not directly available in the observations. This reflects the design of many modern RL implementations, where policies are often parameterized by architectures capable of summarizing past observations, such as recurrent or attention-based networks, without explicitly modeling the uncertainty over the underlying state.

3.3.3 Trajectory

Now that both the MDP $(\mathcal{S}, \mathcal{O}, \mathcal{A}, \mathcal{R}, p)$ and the policy π have been defined, the interaction between them can be characterized. A sequence of consecutive interactions between an environment and a policy is called a trajectory. A trajectory is generated through the repeated interaction between the policy and the environment, following the sequence below:

1. The environment, being in state s , provides an observation o to the policy π . Based on this observation, the policy outputs a probability distribution over actions $\pi(\cdot | o)$, from which an action is sampled: $a \sim \pi(\cdot | o)$.
2. The sampled action is executed in the environment. The environment then generates a transition: $(s', o', r) \sim p(\cdot | s, a)$. It moves to a new state s' , and returns a new observation o' and a reward r .

A trajectory t of length T can be written as:

$$t = (s_1, o_1, a_1, r_1, s_2, o_2, a_2, r_2, \dots, s_T, o_T, a_T, r_T).$$

It is important to clearly distinguish between a trajectory distribution and its realizations. The distribution over trajectories is denoted by τ , and a particular trajectory is denoted by t . The distribution τ is determined by the initial state s , the initial observation o , the environment dynamics p , and the policy π , and is denoted by $\tau(s, o, p, \pi)$. The notation $t \sim \tau(s, o, p, \pi)$ means that the trajectory t is sampled from the distribution induced by s , o , p , and π . The notation $\tau(t | s, o, p, \pi)$ denotes the probability of observing the trajectory t under the distribution $\tau(\cdot | s, o, p, \pi)$.

An important property of the trajectory distribution is its recursive structure. Since both the policy π and the transition function p are known, the distribution over trajectories starting from s_1 with observation o_1 can be expressed in terms of the distribution over trajectories starting from the next state s_2 with observation o_2 . Specifically, the probability of a trajectory t factorizes as:

$$\tau(t | s_1, o_1, p, \pi) = \pi(a_1 | o_1) p(s_2, o_2, r_1 | s_1, a_1) \tau(t_2 | s_2, o_2, p, \pi),$$

where $t_2 = (s_2, o_2, a_2, r_2, \dots, s_T, o_T, a_T, r_T)$ denotes the trajectory starting from the second step.

There exist two special types of trajectories. First, a trajectory that terminates in a terminal state is called a terminal trajectory, and is denoted by $t_t \sim \tau_t(s, o, p, \pi)$. Second, a terminal trajectory that starts from an initial state and an initial observation is called an episode, and is denoted by $t_e \sim \tau_t(s^{\text{init}}, o, p, \pi)$.

3.3.4 Value Functions

To evaluate the ability of a policy π to collect rewards, the notion of return is introduced. The return of a trajectory $g(t, \gamma)$, is defined as the discounted sum of rewards collected along the trajectory:

$$g(t, \gamma) = r_1 + \gamma r_2 + \gamma^2 r_3 + \cdots + \gamma^{T-1} r_T = \sum_{i=1}^T \gamma^{i-1} r_i,$$

where $\gamma \in [0, 1]$ is the discount factor. This parameter controls the time horizon of the agent's objective. When γ is close to 0, the agent focuses almost exclusively on immediate rewards, leading to short-sighted behavior. As γ increases toward 1, future rewards gain more weight, and the agent adopts a more far-sighted strategy. In theory, since the goal is to maximize long-term cumulative reward, the discount factor would ideally be set to $\gamma = 1$. In practice, γ is often chosen slightly below 1 (e.g., 0.99) to stabilize learning, and to reflect the fact that distant future rewards are inherently more uncertain.

The return also admits a recursive form, which is central to the dynamic programming and RL algorithms presented later:

$$g(t, \gamma) = r_1 + \gamma g(t_2, \gamma),$$

From the notion of return, functions that estimate the expected return of a policy from a given starting point can be defined. These functions are called value functions. The first one considered is the state-value function V . It tells us how desirable a particular state is under a given policy π , given an observation o , the environment dynamics p , and the discount factor γ . Strictly speaking, this function should be written as $V(s, o, p, \gamma, \pi)$ to reflect all its dependencies. However, since both the environment dynamics p and the discount factor γ are fixed throughout learning, they are omitted from the notation. To emphasize that the value is tied to a specific policy, π is placed as a subscript. This yields the compact notation $V_\pi : \mathcal{S} \times \mathcal{O} \rightarrow \mathbb{R}$.

If $V_\pi(s, o) > V_\pi(s', o')$, then, under policy π , the agent prefers being in state s with observation o over state s' with observation o' . Formally, the state-value function is defined as:

$$V_\pi(s, o) = \mathbb{E}[g(t, \gamma)], \quad \text{with } t \sim \tau_t(s, o, p, \pi).$$

It is worth noting that the state-value function V_π is recursive, a property that follows directly from the recursive form of the return:

$$\begin{aligned} V_\pi(s, o) &= \mathbb{E}[g(t, \gamma)], \quad \text{with } t \sim \tau_t(s, o, p, \pi) \\ &= \mathbb{E}[r_1 + \gamma g(t_2, \gamma)] \\ &= \mathbb{E}[r_1 + \gamma V_\pi(s_2, o_2)] \end{aligned}$$

If the expectation is expressed explicitly in terms of the distributions p and π , the Bellman equation for V_π is obtained:

$$\begin{aligned}
V_\pi(s, o) &= \mathbb{E}\left[r_1 + \gamma V_\pi(s_2, o_2)\right] \\
&= \sum_{a \in \mathcal{A}} \pi(a | o) \sum_{\substack{s' \in \mathcal{S} \\ o' \in \mathcal{O} \\ r \in \mathcal{R}}} p(s', o', r | s, a) \left[r + \gamma V_\pi(s', o')\right].
\end{aligned}$$

Now that the value of a state has been expressed, the next step is to consider the value of taking a specific action in a given state. Indeed, it is useful to estimate the expected future gain of an action a in state s , in order to compare different actions and determine which one is the most promising under a given policy π . This function is called the state-action value function $Q_\pi : \mathcal{S} \times \mathcal{A} \rightarrow \mathbb{R}$. If $Q_\pi(s, a) > Q_\pi(s, a')$, then, under policy π , choosing action a in state s yields a higher expected return than choosing action a' . It is defined as:

$$Q_\pi(s, a) = \mathbb{E}\left[r + \gamma g(t, \gamma)\right], \quad \text{with } t \sim \tau_t(s', o', p, \pi), \quad (s', o', r) \sim p(\cdot | s, a).$$

The function Q_π can be related to V_π by recognizing that, after taking action a in state s , the remaining return from the next state s' with observation o' is precisely $V_\pi(s', o')$. This yields:

$$\begin{aligned}
Q_\pi(s, a) &= \mathbb{E}\left[r + \gamma g(t, \gamma)\right], \quad \text{with } t \sim \tau_t(s', o', p, \pi), \quad (s', o', r) \sim p(\cdot | s, a) \\
&= \mathbb{E}\left[r + \gamma V_\pi(s', o')\right] \\
&= \sum_{\substack{s' \in \mathcal{S} \\ o' \in \mathcal{O} \\ r \in \mathcal{R}}} p(s', o', r | s, a) \left[r + \gamma V_\pi(s', o')\right]
\end{aligned}$$

3.3.5 Optimal Policy

The goal of RL is to find a policy that maximizes the expected cumulative reward from any observation until the end of the trajectory. This policy is called the optimal policy and is denoted by π^* . In a partially observable setting, the optimal action is the one that maximizes the expected state-action value over the distribution of possible states given the observation:

$$\pi^*(a | o) = \begin{cases} 1 & \text{if } a = \arg \max_{a' \in \mathcal{A}} \sum_{s \in \mathcal{S}} b(s | o) Q_{\pi^*}(s, a'), \\ 0 & \text{otherwise} \end{cases}$$

As shown, both V_π and Q_π can be expressed explicitly in terms of the environment dynamics p and the policy π . If p , b and π are fully known, one can theoretically compute the optimal policy by unfolding the graph of all possible trajectories starting from a given state s and observation o . This is possible precisely because V_π and Q_π are recursive functions. Solving an MDP in this way is known as dynamic programming.

In practice, most environments do have too many states to allow an explicit representation of the dynamics p . Storing and computing over the full transition function becomes intractable. To overcome this limitation, instead of directly

constructing the optimal policy, an iterative approach is adopted: starting from an initial policy, it is progressively improved so that it converges toward π^* . This iterative improvement through interaction with the environment is precisely what constitutes learning in RL, and it is the subject of the next section.

3.4 Learning

Now that the optimal policy π^* has been defined, the question is how to obtain it. The recursive Bellman equations for V_π and Q_π suggest dynamic programming approaches, but these scale poorly to large state spaces. RL offers an alternative by solving the MDP iteratively and generating a sequence of policies that progressively approach π^* through direct interaction with the environment. This section introduces two major families of RL algorithms, critic-only methods and actor-critic methods.

At a high level, every algorithm presented below repeats the same loop: the current policy interacts with the environment, the resulting trajectories are sent to the RL algorithm, and the algorithm uses them to produce a new policy.

3.4.1 Critic-Only Methods

The central idea behind all critic-only approaches is to iteratively construct an approximation of the state-action value function Q_π , denoted by $\hat{Q}_\pi$. Although $\hat{Q}_\pi$ is only an approximation of the true Q_π , the policy always acts greedily with respect to this approximation.

Since $\hat{Q}_\pi$ serves as the basis for the agent's decisions, it cannot rely on more information than the agent itself has access to. It must therefore be defined solely over observations, as a function $\hat{Q}_\pi : \mathcal{O} \times \mathcal{A} \rightarrow \mathbb{R}$. Its objective is to approximate the true Q_π , which depends on the underlying states s , using only the partial information available through o . In practice, this does not necessarily pose a problem, as observations often contain sufficient information to make accurate predictions about the underlying state. Moreover, the strong representational capacity of NN allows the critic to capture these relationships without explicitly representing the mapping between observations and states.

This leads to the following mathematical formulation:

$$\pi(a \mid o) = \begin{cases} 1 & \text{if } a = \arg \max_{a' \in \mathcal{A}} \hat{Q}_\pi(o, a'), \\ 0 & \text{otherwise.} \end{cases} \quad (3.1)$$

It is precisely this way of modeling the policy that gives this family of methods its name. The policy is not represented explicitly as a standalone function. Rather, it simply emerges from searching over the set of actions to maximize $\hat{Q}_\pi$.

To give a clear example of critic-only methods, they are presented in their simplest form: model-free, without temporal-difference learning, without replay buffer, without bootstrapping, and using only ε -greedy exploration. Naturally, many variants and extensions of this basic scheme exist, but the core idea remains the same.

When implementing a RL algorithm, one of the first questions is how to represent the value function. In critic-only methods, this means choosing a

model for $\hat{Q}_\pi$. Any machine learning model capable of regression task can be used for this purpose. The set of parameters of the model is denoted by θ . In deep RL, a NN is used to model this function, in which case θ represents the weights and biases of the network.

In our simple setting, exploration is handled via ε -greedy selection. With probability $1 - \varepsilon$, the policy selects the greedy action that maximizes $\hat{Q}_\pi$, and with probability ε it selects an action uniformly at random. Formally, this ε -greedy policy is obtained by modifying the greedy policy defined above as follows:

$$\pi(a \mid o) = \begin{cases} 1 - \varepsilon + \frac{\varepsilon}{|\mathcal{A}|} & \text{if } a = \arg \max_{a' \in \mathcal{A}} \hat{Q}_\pi(o, a'), \\ \frac{\varepsilon}{|\mathcal{A}|} & \text{otherwise.} \end{cases} \quad (3.2)$$

The trajectory distribution induced by the resulting policy now depends on both π and ε , and is denoted by $t \sim \tau(s, o, p, \pi, \varepsilon)$. The parameter ε controls the degree of exploration. If the policy always chooses the action it currently believes to be optimal, it may miss better alternatives that $\hat{Q}_\pi$ has not yet learned to value correctly. Exploring suboptimal actions from time to time is therefore essential to verify that $\hat{Q}_\pi$ accurately reflects the true values, while still focusing most of the time on the most promising actions. In more advanced implementations, ε can be adapted over the course of learning to control exploration dynamically, for instance by decreasing it gradually over time.

The learning process consists of two steps that repeat until convergence:

1. **Data collection.** The current policy π interacts with the MDP. For a given starting state s_1^{init} and observation o_1 , the interaction produces a episode t_e sampled from $\tau_t(s_1^{\text{init}}, o_1, p, \pi, \varepsilon)$. For each step i along the episode, the return from that step onward is $g(t_{i:}) = \sum_{k=i}^T \gamma^{k-i} r_k$, which serves as the target value for the state-action pair (s_i, a_i) . This procedure is detailed in Algorithm 2.
2. **Model update.** The tuples $(s_i, a_i, g(t_{i:}))$ obtained in the previous step serve as supervised training data: the state-action pair (s_i, a_i) is the input, and the return $g(t_{i:})$ is the associated target. These examples are used to train $\hat{Q}_\theta$. The improved approximation is denoted by $\hat{Q}'_\theta$, which now predicts returns more accurately under π . Since $\hat{Q}'_\theta$ has changed, the policy derived from it also changes. This new policy is denoted by π' .

Because the policy is now π' , the approximation $\hat{Q}'_\theta$ must be trained to predict returns under π' . This requires returning to step 1 to collect fresh interaction data generated by π' . The two steps thus alternate until the policy and the value approximation stabilize. Algorithm 3 summarizes this whole critic-only learning loop.

Even this simple algorithm already illustrates a difficulty inherent to all forms of RL: the instability of the learning process. Each update from $\hat{Q}_\theta$ to $\hat{Q}'_\theta$ causes the policy to shift from π to π' . For the learning to remain stable, the change from $\hat{Q}_\theta$ to $\hat{Q}'_\theta$ should be as small as possible. This way, $\hat{Q}'_\theta$, trained to predict returns under π , remains reasonably accurate under π' . However, the smaller the change, the slower the learning progresses. A trade-off must therefore be struck between stability and learning speed. Many improvements introduced later in the literature aim at stabilizing this learning loop [Hessel et al., 2017].

Algorithm 2: Data Collection**Input:**MDP $(\mathcal{S}, \mathcal{O}, \mathcal{A}, \mathcal{R}, p)$ Policy π Discount factor $\gamma \in [0, 1]$ **Output:**Dataset $\mathcal{D}$ *Sample an initial state.***1** $i \leftarrow 1$;**2** Sample $s_i^{\text{init}} \in \mathcal{S}^{\text{init}}$;**3** Observe o_i ;*Episode generation by interaction with the MDP.***4 repeat***Action selection.***5** Sample $a_i \sim \pi(\cdot \mid o_i)$;*Environment transition.***6** Sample $(s_{i+1}, o_{i+1}, r_i) \sim p(\cdot \mid s_i, a_i)$;**7** $i \leftarrow i + 1$;**8 until** $s_i \in \mathcal{S}^{\text{ter}}$;*Return computation for each step of the trajectory.***9** $T \leftarrow i - 1$;**10 for** $i = 1, \dots, T$ **do****11** $g_i \leftarrow \sum_{k=i}^T \gamma^{k-i} r_k$;*Dataset construction***12** $\mathcal{D} \leftarrow \{(s_i, o_i, a_i, g_i)\}_{i=1}^T$;**13 Return** $\mathcal{D}$

Convergence guarantees for such algorithms have been proved under a different set of assumptions than the setting presented here. For example, in the classical convergence results, the MDP is fully observable, the value function is stored in a table, and the learning rate is decreased at a suitable rate over time [Watkins and Dayan, 1992]. In modern implementations, the value function is approximated by a NN. Theoretical convergence proofs no longer hold in this setting. Nevertheless, deep methods have demonstrated remarkable empirical success, making them the tool of choice for RL tasks.

Critic-only approaches suffer from several limitations. The main limitation is scalability to large action spaces. Since the policy must evaluate all possible actions at each decision step to select the maximum, the computational cost grows with the size of the action set. To address this issue, another family of RL algorithms exists: actor-critic methods.

Algorithm 3: Deep Critic-Only Reinforcement Learning**Input:**

MDP $(\mathcal{S}, \mathcal{O}, \mathcal{A}, \mathcal{R}, p)$
 Architecture $\hat{Q}$
 Exploration rate $\varepsilon \in [0, 1]$
 Discount factor $\gamma \in [0, 1]$
 Learning rate $\eta > 0$
 Initial parameters θ (optional)

Output:

Trained policy π_θ

```

1 if  $\theta$  is not given then
2   | Initialize  $\theta$  randomly;
   Exploration policy as defined in Equation (3.2)
3  $\pi_\theta \leftarrow \varepsilon$ -greedy policy with respect to  $\hat{Q}_\theta$ 
   MSE loss function as defined in Equation (1.2)
4  $\mathcal{L} \leftarrow$  Mean Squared Error (MSE);
5 while stopping criterion not satisfied do
   Data collection see Algorithm 2
6    $\mathcal{D} \leftarrow \text{DataCollection}(\text{MDP}, \pi_\theta, \gamma)$ 
   observation-action pairs and their returns, from  $\mathcal{D}$ 
7    $\mathcal{D}' \leftarrow \{((o, a), g) \mid (s, o, a, g) \in \mathcal{D}\}$ 
   Model update with SGD see Algorithm 1
8    $\theta \leftarrow \text{SGD}(\mathcal{D}', \hat{Q}, \mathcal{L}, \eta, \theta)$ 
   Final policy without exploration as defined in Equation (3.1)
9  $\pi_\theta \leftarrow$  policy with respect to  $\hat{Q}_\theta$ ;
10 Return  $\pi_\theta$ 

```

3.4.2 Actor-Critic Methods

Two principal modifications distinguish actor-critic methods from critic-only approaches. First, the objective is to approximate the state-value function V , instead of the state-action value function Q . Second, the policy π is given its own dedicated set of parameters. Two separate components are thus obtained: a value function approximation $\hat{V}_{\theta_1}$ and a parameterized policy π_{θ_2} .

Analogously to critic-only methods, $\hat{V}_{\theta_1}$ is trained using supervised learning on pairs obtained from interaction with the MDP. For each step i along a trajectory, the input is the state-observation pair (s_i, o_i) and the target is the observed return $g(t_{i:})$. The value function is thus trained to correct its predictions based on data collected by the current policy interacting with the environment.

The key difference lies in how the policy π itself is optimized. Since the policy now has its own set of parameters θ_2 , it can be updated directly. The update uses the notion of advantage $A : \mathcal{S} \times \mathcal{A} \rightarrow \mathcal{R}$, defined as:

$$A(s, a) = g(t) - \hat{V}_{\theta_1}(s, o),$$

1420 where t is the trajectory that starts by taking action a in state s and then follows
 1421 π for all subsequent steps.

1422 When $A(s, a) > 0$, the action a performed better than expected, and the
 1423 policy parameters θ_2 are adjusted to increase the probability $\pi_{\theta_2}(a | o)$. When
 1424 $A(s, a) < 0$, the action a performed worse than expected, and the probability is
 1425 decreased.

1426 Both $\hat{V}_{\theta_1}$ and π_{θ_2} are improved simultaneously within the same loop: the
 1427 critic provides a learned baseline for the actor, and the actor generates the data
 1428 used to train the critic. This interplay reduces the instability inherent to RL.
 1429 Algorithm 4 summarizes this whole actor-critic learning loop.

1430 PPO (Proximal Policy Optimization), which is the current standard in RL,
 1431 belongs to this family of actor-critic methods. It will be used throughout this
 1432 manuscript for all RL experiments.

1433 3.5 Conclusion

1434 This chapter has provided an introduction to the fundamental principles of RL.
 1435 Only the essential concepts required to understand the RL framework have been
 1436 presented. Despite this limited scope, the two defining steps of RL can already
 1437 be observed in its simple learning procedure. The first step consists of estimating
 1438 the reward that can be obtained from the interaction between the agent and its
 1439 environment in a given situation. The second step uses this estimate to improve
 1440 the current policy. Learning therefore results from the repeated alternation of
 1441 these two basic operations.

1442 This simple iterative structure brings RL conceptually close to DL. In both
 1443 paradigms, simple operations can lead to complex behaviors when repeated at
 1444 scale. Their combination is particularly powerful. A wide range of decision-
 1445 making problems can be formulated as MDP, allowing RL to be applied to a
 1446 broad range of tasks. DL, in turn, provides flexible function approximators
 1447 capable of representing the value functions and policies required to solve these
 1448 tasks. Together, these two paradigms form the basis of deep RL, which has
 1449 achieved remarkable success across a wide range of challenging tasks.

1450 From a broader perspective, the evolution of RL has brought the field closer
 1451 than ever to its initial biological inspiration. In modern implementations, agents
 1452 progressively improve their behavior through trial and error directly based on
 1453 experience. Moreover, their decision-making capabilities are represented by
 1454 neural architectures that resemble their biological counterparts. It is therefore
 1455 not surprising that RL has become one of the core paradigms for pursuing
 1456 artificial general intelligence [Silver et al., 2021].

1457 As the capabilities of RL continue to improve, a new challenge becomes
 1458 increasingly important. The ability to learn complex behaviors with limited
 1459 human intervention can make the resulting agent’s strategy difficult to understand.
 1460 To address this challenge, a parallel research direction has emerged, known as
 1461 eXplainable Reinforcement Learning (XRL). This field is the subject of the next
 1462 chapter.

Algorithm 4: Deep Actor-Critic Reinforcement Learning

Input:

MDP $(\mathcal{S}, \mathcal{O}, \mathcal{A}, \mathcal{R}, p)$
 Architecture $\hat{V}$ (critic)
 Architecture π (actor)
 Discount factor $\gamma \in [0, 1]$
 Learning rate $\eta > 0$
 Initial parameters θ_1 of the critic (optional)
 Initial parameters θ_2 of the actor (optional)

Output:

Trained policy π_{θ_2}

```

1 if  $\theta_1$  is not given then
2   | Initialize  $\theta_1$  randomly;
3 if  $\theta_2$  is not given then
4   | Initialize  $\theta_2$  randomly;

   MSE loss function as defined in Equation (1.2)
5  $\mathcal{L} \leftarrow \text{MSE}$ ;

6 while stopping criterion not satisfied do
   Data collection see Algorithm 2
7    $\mathcal{D} \leftarrow \text{DataCollection}(\text{MDP}, \pi_{\theta_2}, \gamma)$ 

   Advantage estimation: how much better/worse each action performed compared
   to the critic's expectation
8   for  $(s_i, o_i, a_i, g_i) \in \mathcal{D}$  do
9     |  $A(s_i, a_i) \leftarrow g_i - \hat{V}_{\theta_1}(s_i, o_i)$ ;

   state and their returns, from  $\mathcal{D}$ 
10   $\mathcal{D}' \leftarrow \{(s, g) \mid (s, o, a, g) \in \mathcal{D}\}$ 

   Critic update with SGD see Algorithm 1
11   $\theta_1 \leftarrow \text{SGD}(\mathcal{D}', \hat{V}, \mathcal{L}, \eta, \theta_1)$ 

   Actor update: increase the probability of actions with positive advantage, decrease
   it for those with negative advantage
12   $\theta_2 \leftarrow \theta_2 + \eta \sum_{(s_i, o_i, a_i, g_i) \in \mathcal{D}} A(s_i, a_i) \nabla_{\theta_2} \log \pi_{\theta_2}(a_i \mid o_i)$ ;

13 Return  $\pi_{\theta_2}$ 
```

Chapter 4

Explainability in Reinforcement Learning

Today, the majority of Artificial Intelligence (AI) agents with which humans interact are deployed in controlled or low-risk environments. For example, agents capable of playing games such as Go operate in settings where potential errors remain limited in terms of consequences [Silver et al., 2017a]. Even more autonomous systems, such as agents based on Large Language Model (LLM), remain constrained by human supervision mechanisms and are often restricted to tasks such as code generation or information retrieval [Chen et al., 2021, Lewis et al., 2021]. Safety-critical applications integrating machine learning components are still rare. Introducing autonomous agents in such environments, requires the establishment of strong guarantees [Seshia et al., 2022]. These requirements include in particular:

- validating the system’s functioning before deployment,
- identifying the origin of a malfunction in case of failure,
- and certifying that an observed failure will not occur again.

The main challenge lies in the fact that the performing agents currently available rely on both Reinforcement Learning (RL) and Deep Learning (DL) [Mnih et al., 2013]. As a result, the mechanisms underlying their decisions cannot be understood directly from the learned model. This lack of explainability makes their use difficult in safety-critical contexts. To address this limitation, the field of eXplainable Reinforcement Learning (XRL) has emerged as a subfield of eXplainable Artificial Intelligence (XAI). The objective of XRL is to develop explainability methods that make it possible to obtain explanations of RL agents. Due to the relevance of the topic and to foster broader real-world adoption of RL, a growing number of research teams are working on developing new explainability methods. This enthusiasm is reflected in a rapid increase in the number of publications, as illustrated in Figure 4.1.¹

¹The complete search protocol, including the databases consulted and the full set of query terms, has been archived on Zenodo <https://zenodo.org/records/19387785>.

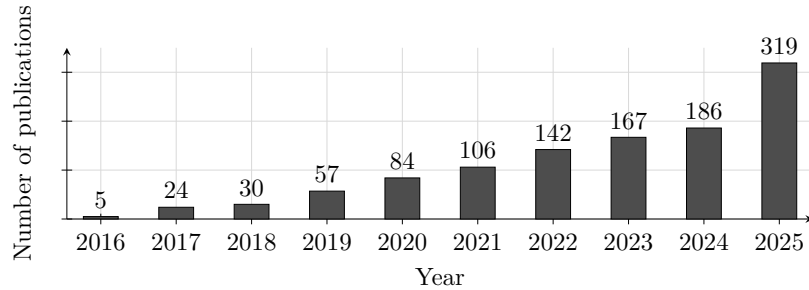


Figure 4.1: Yearly growth of Explainable Reinforcement Learning research with at least one citation on arXiv.

The growing number and diversity of these methods make it increasingly difficult to obtain a unified view of the field. This motivated our work on developing a unified taxonomy of explainability methods for RL, initially presented in [Alaarabiou et al., 2024]. The taxonomy provides a common structure for organizing existing approaches according to the type of information they use and the form of explanation they provide. Building on this work, this chapter provides a detailed presentation of the proposed taxonomy and uses it to organize a broader state-of-the-art review of XRL.

This chapter is organized as follows:

- Section 4.1 begins by clarifying what we mean by an explanation in the context of XAI.
- Section 4.1 then introduces a taxonomy for categorizing existing XRL methods.
- Section 4.3 provides a comprehensive state-of-the-art review structured around this taxonomy.
- Section 4.4 concludes the chapter with a summary and discussion.

4.1 Explanation in Artificial Intelligence

In this section, we define what an explanation is in the context of XAI and how it can be obtained. We first introduce the concept of explanation in general. We then describe how this concept applies to XAI. Finally, we further detail the properties that characterize an explainability method in the context of XAI.

4.1.1 Explanation

Defining what constitutes an explanation is challenging because the concept is inherently multidisciplinary. When we refer to explanation, we are inherently referring to an explanation provided to a human. As soon as humans are involved, the problem is no longer purely computational or mathematical. It involves several fields of research, such as philosophy, psychology, and sociology. So far, these disciplines have not converged on a unified definition of explanation that can be directly applied to XAI [Miller, 2019, Bellucci et al., 2021].

For this reason, we propose the following definition of explanation. An explanation involves two entities: the object (Definition 4.1.1) being explained and the receiver of the explanation. The receiver is a human who aims to understand how the object operates and will be referred to as the observer throughout this manuscript (Definition 4.1.2). The observer seeks to understand the object and, in doing so, builds a mental model of its functioning. An explanation is therefore what allows the observer to refine and correct their mental model of how the object operates (Definition 4.1.4). This refinement is obtained through a transformation process that produces an explanation from the object being explained. We refer to this process as an explainability method (Definition 4.1.5), detailed further in Section 4.1.3. For an observer to understand an explanation, the information it contains must be interpretable (Definition 4.1.3). Information is considered interpretable when it can be directly understood by a human without requiring any additional transformation process.

Definition 4.1.1: Object

The object of an explanation is the system the observer wants to understand.

Definition 4.1.2: Observer

The observer is the human who aims to understand an object. The observer possesses a mental model of the object being explained. This mental model is an internal representation of how the observer believes the system operates. This representation may be incomplete or inaccurate.

Definition 4.1.3: Interpretable

Information is interpretable when it can be directly understood by an observer without requiring any additional transformation process.

Definition 4.1.4: Explanation

An explanation is any interpretable information that enables an observer to refine or correct their mental model of the object of the explanation.

Definition 4.1.5: Explainability Method

An explainability method is a transformation process that produces an explanation from the object being explained.

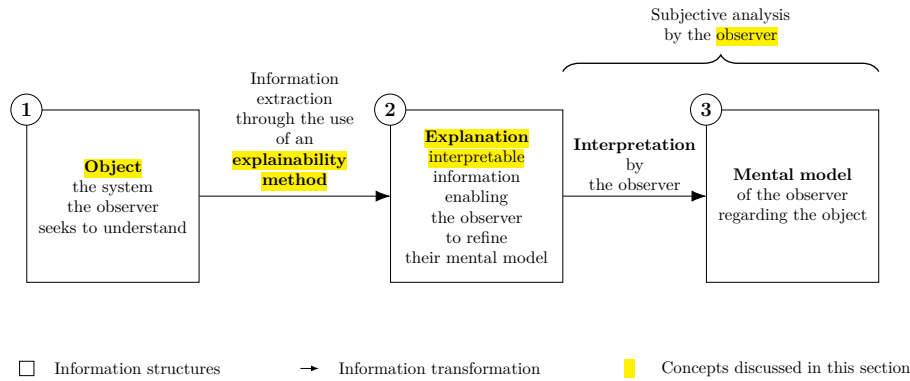


Figure 4.2: Explanation process.

Figure 4.2 summarizes this whole explanation process, from the object of the explanation to the mental model refined by the observer, as a chain of three information structures, depicted as boxes and connected by arrows denoting a transformation of information:

- (1) The object of the explanation, that is, the system the observer seeks to understand.
- (2) The explanation, the interpretable information produced from the object. This transformation is performed through the use of an explainability method.
- (3) The mental model held by the observer regarding the object.

A brace spanning boxes (2) and (3) indicates that this second half of the process, from the explanation to the resulting mental model, constitutes a subjective analysis carried out by the observer.

This definition of explanation establishes the approach to explainability adopted throughout this manuscript. Under this definition, explainability is regarded as a continuum that evolves as knowledge develops [Siegel and Craver, 2024, Zednik, 2019].

This quantitative perspective highlights that not all explanations provide the same level of value [Doshi-Velez and Kim, 2017]. The effectiveness of an explanation depends on its ability to improve the observer’s understanding of the system. For example, describing a marginal aspect of a system is different from correcting an observer’s misunderstanding of a key mechanism.

1562 4.1.2 Explainable Artificial Intelligence

1563 Now that the general framework of explanation has been established, we can
 1564 examine how this concept applies in the context of AI. In the context of XAI, the
 1565 object of explanation is an AI system. More precisely, XAI emerged as a response
 1566 to the success of DL models [Gunning and Aha, 2019, Barredo Arrieta et al.,
 1567 2020]. In these architectures, information is represented in a distributed manner
 1568 across the network through patterns of activation and connections between
 1569 computational units. Unlike symbolic approaches, individual components of
 1570 Neural Network (NN) cannot be directly associated with human concepts. From
 1571 this starting point, we identify two types of approaches:

- 1572 • **Substitutive explainability:** In this approach, the assumption is that
 1573 there is nothing to explain within a NN. There is no semantic meaning
 1574 associated with each computational component, and therefore nothing to
 1575 interpret. This corresponds to the “black box” view of NN. Under this
 1576 assumption, XAI consists of replacing as much of the DL system as possible
 1577 with alternative AI methods [Rudin, 2019].
- 1578 • **Intrinsic explainability:** In this approach, it is argued that NN contain
 1579 mechanisms that can be explained. The limitation is considered to lie not
 1580 in the networks themselves, but in the absence of adequate tools to analyze
 1581 their functioning [Olah et al., 2020].

1582 This manuscript considers methods from both approaches in order to provide
 1583 a comprehensive overview of the XAI field. Although both approaches have
 1584 produced convincing results, this manuscript primarily adopts the perspective
 1585 of the *Mechanistic Hypothesis* (Hyp. 3) [Siegel and Craver, 2024, Zednik, 2019].
 1586 This perspective is more closely aligned with the definition of explainability
 1587 adopted in this manuscript, which considers explainability as a continuum that
 1588 evolves with the development of knowledge.

Hypothesis 3: Mechanistic Hypothesis

Every AI system results from a sequence of causal computations. Because
 the rules governing these interactions are mechanical, the behavior of the
 system can be explained. If an AI system cannot be explained, this reflects
 a limitation of knowledge rather than an intrinsic property of the system.

1589

1590 4.1.3 Explainability Method in XAI

1591 Now that the general framework of XAI has been established, we need to further
 1592 detail the properties of an explanation in this context. In our framework, an
 1593 explanation in the field of XAI can be characterized by three components:

- 1594 • **Source:** refers to the origin of the information used by the explainability
 1595 method. The explainability method transforms the raw information pro-
 1596 duced by the AI system into an explanation that can be understood by an
 1597 observer.
- 1598 • **Form:** refers to the representation through which the explanation is
 1599 communicated. The form defines how the information is presented to
 1600 the observer and must allow human interpretation. An explanation can
 1601 take many different forms, including text, images, computer code, and
 1602 mathematical equations.
- 1603 • **Subject:** refers to the aspect of the AI system that is being explained. In
 1604 XAI, this distinction is commonly associated with the difference between
 1605 local and global explanations. A local explanation describes why an AI
 1606 system produced a specific output in a given situation. A global explanation
 1607 aims to describe the general functioning of the AI system.

1608 These three properties of an explanation are entirely determined by the
 1609 explainability method (Definition 4.1.5) used to produce it. In this context,
 1610 an explainability method is a transformation process that maps a source of
 1611 information into an explanation characterized by a specific form and subject.
 1612 Research in XAI focuses on developing new methods capable of performing this
 1613 transformation in order to overcome the lack of interpretability associated with
 1614 DL models.

1615 4.2 Taxonomy of XRL Methods

1616 Now that the concept of XAI has been defined, we turn to its application in RL,
 1617 known as XRL. As in XAI, explanations in XRL are produced by explainability
 1618 methods and can be described in terms of their subject, source, and form. This
 1619 section reviews the possible variations of these three aspects in the specific
 1620 context of RL.

1621 4.2.1 Subject

1622 The first question to address is the subject of explanation in the context of RL.
 1623 To answer this question, it is necessary to identify the different components
 1624 involved in a RL system and determine which of them constitutes the object of
 1625 explanation. A RL system can be described through two main components: the
 1626 environment and the policy.

1627 The environment defines the dynamics in which the agent evolves, including
 1628 the possible states, observations, actions, and transitions. Environments are
 1629 simulations designed by humans, and their dynamics are explicitly defined.
 1630 Therefore, it is assumed that the environment is known to the observer and does
 1631 not require explanation.

The main object of interest is therefore the agent’s policy. The policy represents the decision-making mechanism of the agent. It is a function that maps observations to actions and is learned through interaction with the environment in order to maximize the expected cumulative reward. The policy is the component that encodes the learning process and that XRL aims to explain.

Explaining a policy introduces additional challenges compared with classical XAI settings. In traditional XAI problems, the objective is to explain an isolated decision, such as determining which features of an image led to a classification result. In contrast, RL introduces a temporal dimension into the decision-making process. An action is not only determined by the current observation but also influences future states and rewards. Therefore, understanding the behavior of an agent may require considering a sequence of interactions between the agent and its environment.

This temporal dimension leads to a distinction between two objectives of explanation in XRL:

- **Decision explanations:** focus on identifying the elements of the current observation that influenced the action selected by the policy. This type of explanation is close to classical XAI approaches, as it analyzes the relationship between inputs and outputs while abstracting away the sequential nature of the decision process.
- **Strategy explanations:** aim to characterize the agent’s behavior over time. They focus on sequences of decisions and interactions with the environment in order to understand how the agent achieves its long-term objective. This type of explanation is specific to RL, as it addresses the temporal and goal-oriented nature of the learning process.

These two objectives should not be considered as strictly separated categories, but rather as two extremes of a continuum. Understanding the elements of an observation that influence a specific decision can help the observer generalize this information to infer broader aspects of the agent’s behavior. Conversely, understanding the agent’s overall strategy can provide insights into the reasoning behind individual decisions.

1663 4.2.2 Source

1664 We identify three sources of explanation in RL, all obtained during or after the
1665 learning process and containing information about the agent:

- 1666 • **Policies:** policies are functions that generate a distribution over the action
1667 space for a given observation. The objective of RL is to learn a policy that
1668 maximizes the expected cumulative future rewards. During the learning
1669 phase, the policy is iteratively improved through interactions with the
1670 environment.
- 1671 • **Trajectories:** trajectories represent sequences of successive interactions
1672 between the agent and the environment. At each step, the agent selects
1673 an action based on its observation, which modifies the environment and
1674 produces a reward as well as a new observation. This process continues
1675 until the end of the trajectory.
- 1676 • **Value functions:** value functions provide predictions about the future
1677 outcome of a trajectory from a given observation. The most commonly
1678 used value function is the critic function, which estimates the expected
1679 sum of future rewards. This prediction guides the learning process, as the
1680 agent updates its policy to maximize the estimated future return. However,
1681 other types of value functions can also be used as sources of explainability.

1682 4.2.3 Form

1683 We identify six forms of explanation in the RL setting:

- 1684 • **Policy Model:** an interpretable model of the agent’s policy function.
- 1685 • **Saliency Map:** a weighting of the input features representing their
1686 influence on the output of the policy function.
- 1687 • **Counterfactual:** a modified observation obtained by perturbing the
1688 original observation in order to produce a different action from the agent.
- 1689 • **Agent Trajectory:** a sequence of successive interactions between the
1690 agent and the environment.
- 1691 • **Trajectory Prediction:** a prediction about the future evolution of the
1692 trajectory produced by a value function.
- 1693 • **Interaction Model:** an interpretable model of the interactions between
1694 the agent and the environment.

1695 Among the three components characterizing an explanation, the form is the
1696 one that varies most significantly across explainability methods. While subjects
1697 and sources tend to be relatively stable in the XRL literature, the diversity in
1698 how explanations are presented is considerably greater. The list above is by no
1699 means definitive and may evolve as new approaches emerge. For this reason, the
1700 following section is organized according to the form of explanations and presents
1701 existing explainability methods based on this criterion.

Explanation Subject	Explanation Form	Explainability Method Family	Source of Explanation		
			Policy	Trajectories	Value functions
Decision	Policy Model	<i>Interpretable policy learning</i>	✓		
		<i>Policy approximation via interpretable model</i>	✓		
	Saliency Map	<i>Observation perturbation</i>	✓		
		<i>Observation gradient analysis</i>	✓		
		<i>Attention-based architecture</i>	✓		
	Counterfactual	<i>Latent space usage for counterfactual generation</i>	✓		
Strategy	Agent Trajectory	<i>Hierarchical decomposition</i>	✓	✓	
		<i>Extraction via critic analysis</i>		✓	✓
	Trajectory Prediction	<i>Critic enrichment</i>			✓
	Interaction Model	<i>Bayesian network learning</i>		✓	
		<i>Latent space usage for MDP generation</i>	✓		

Table 4.1: Taxonomy of explainability methods in reinforcement learning.

4.3 State-of-the-Art Review of XRL Methods

The purpose of this section is to detail the functioning of existing explainability methods (see Table 4.1). They are presented according to the form of explanation they produce, following the taxonomy introduced in Section 4.2.

A family of methods is defined as a set of methods that share the same source, form, and subject of explanation, as well as a similar procedure for generating explanations. Methods belonging to the same family may differ in their implementation details while relying on the same principle.

The remainder of this review is organized by form of explanation, following the six forms identified in Section 4.2.3. For each form, we first describe its principle and its limitations, then present the families of methods that produce it.

4.3.1 Policy Model

The policy function can be represented by a wide variety of function classes. In modern RL, NN is the most commonly used models for representing policies. Although these models achieve the highest performance, their connectionist nature makes their internal representations non-interpretable.

For this reason, a substitutive approach to XRL aims to replace these models with alternative models considered interpretable. Such models can be understood through direct analysis of their representations. The main classes of interpretable models used to represent policy functions include decision trees [Topin et al., 2021], algorithms [Trivedi et al., 2022], formal logic [Payani and Fekri, 2019], and linear models [Garreau and von Luxburg, 2020].

Explanations derived from an interpretable model primarily provide insights into the decision-making process, since what is modeled is the mapping between an observation and an action. If the model is sufficiently well structured, the

observer can form a mental representation of a sequence of decisions across the environment and thus obtain information about the agent’s strategy.

The main limitation of these methods is that interpretable models rely on symbolic representations. When the policy to be represented is too complex, achieving comparable performance requires a large number of rules or conditions. The resulting model can quickly become difficult to analyze and lose its interpretability.

For example, representing a policy capable of playing Go using a symbolic model would require considering a very large number of possible situations and specific cases. Even if such a representation were possible, the resulting model would likely become intractable for human analysis due to its complexity. This illustrates why NN-based approaches have become dominant in such domains, as they can learn complex decision strategies without requiring an explicit enumeration of rules.

To obtain an interpretable policy model, two families of explainability methods are available: *interpretable policy learning* and *policy approximation via interpretable model*.

Interpretable Policy Learning

This family of methods aims at learning a policy represented by an interpretable model [Topin et al., 2021, Trivedi et al., 2022]. It is the most direct approach to addressing the challenges posed by XRL. This type of training requires modifying the Markov Decision Processes (MDP) in order to adapt it to the search space and to the type of policy representation. Moreover, this search space often needs to be reduced to make learning feasible. Such a reduction requires the integration of expert knowledge into the learning process. With this family of methods, the *interpretable policy function* itself is used to provide explanations to the observer.

Policy Approximation via Interpretable Model

An interpretable model can be trained to approximate a policy learned through deep RL [Sieusahai and Guzdial, 2021]. In this setting, the problem is formulated as a supervised learning task, where the initial deep RL policy is used to label each observation with the action it selects, such as the observation from the Atari game “Pong” shown in Figure 4.3a. Using this dataset, the interpretable model is trained to reproduce the behavior of the initial policy, yielding for instance the decision tree illustrated in Figure 4.3b, obtained by approximating a deep RL policy trained on that environment. The resulting interpretable model is then used as an explanation for the observer. If this approximation is sufficiently faithful, it can also be deployed in the environment in place of the initial policy, thereby providing a high degree of control over the agent.

4.3.2 Saliency Map

The influence of each part of the input on the action selected by the policy can be visualized using heatmaps (Figure 4.4). In the RL setting, the input corresponds to the observation received by the agent at a given time step. Heatmaps therefore highlight the regions of the observation that have the greatest influence on the

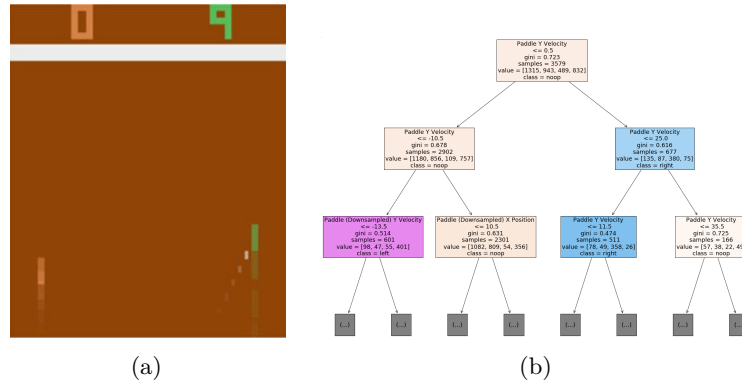


Figure 4.3: Policy approximation via an interpretable model [Sieusahai and Guzdial, 2021].

action selection process. By providing this information, they allow the observer to identify which elements of the environment are considered important by the policy when making a decision. In Figure 4.4, both panels correspond to the same observation of the Atari game “**Breakout**”, with panel (a) obtained by observation perturbation and panel (b) by observation gradient analysis.

A limitation of heatmap-based methods is that the resulting explanations are not always directly meaningful to the observer. As illustrated in Figure 4.4b, interpreting which regions are important often requires human reasoning and involves a degree of subjectivity. This limitation becomes particularly relevant in RL, where the objective is not only to explain a single decision but also to understand the factors influencing a sequence of decisions over time.

Three families of explainability methods can be used to obtain this type of explanation: *observation perturbation*, *observation gradient analysis*, and *attention-based architectures*.

Observation Perturbation

Perturbation analysis consists in applying various modifications to the observation in order to study their impact on the output of the policy function [Greydanus et al., 2018]. The greater the variation in the output caused by a perturbation applied to a given component of the input vector, the higher the weight assigned to that component. The magnitude of the perturbation is measured by computing the distance between the original output and the output obtained from the perturbed input. Using this set of weights, a *heatmap* is constructed, enabling the observer to visualize which parts of the observation vector are most sensitive in the action-selection process (see Figure 4.4a).

Observation Gradient Analysis

Observation gradient analysis refers to a set of methods that require a differentiable policy function in order to extract information from it [Simonyan et al., 2014, Weitekamp et al., 2019, Huber et al., 2019]. To this end, the gradient of the policy function is computed with respect to all components of the input vector. For each input component, the resulting gradient provides a weighting

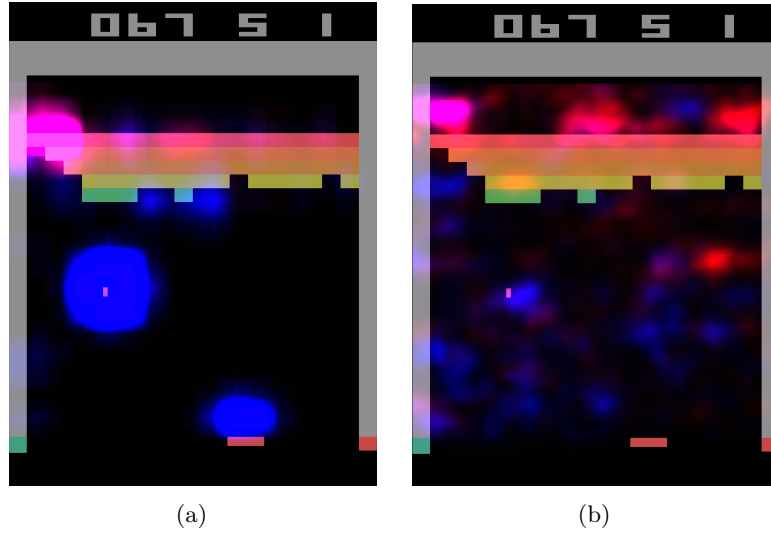


Figure 4.4: Heatmaps for the same observation of the Atari game “**Breakout**” [Greydanus et al., 2018].

that represents its importance in the output of the policy function. Based on these weightings, a *heatmap* is constructed to allow the observer to visually identify the parts of the input vector that are the most influential in the policy function’s output (see Figure 4.4b).

Attention-Based Architecture

Attention blocks constitute a NN architecture that makes it possible to weight the relative importance of one part of a vector with respect to the other parts of the same vector [Vaswani et al., 2023]. These weights are computed as a function of the vector itself and are learned by the NN during training. For a given observation, each component of the observation vector is assigned a scalar value. It can therefore be assumed that the components associated with the highest weights are the most influential in determining the output [Mott et al., 2019, Itaya et al., 2021]. This attention-based *heatmap* is then provided to the observer as the explanation, as illustrated for the Atari game “**Breakout**” in Figure 4.5a, where the highlighted regions are easily interpretable. However, as shown for “**Ms. Pac-Man**” in Figure 4.5b, the attention weights are not always concentrated on a small, easily interpretable set of regions, which can make this type of explanation difficult to exploit.

4.3.3 Counterfactual

A counterfactual explanation is a new observation, close to the original one, that leads the agent to select a different action. Comparing the two observations reveals which elements of the environment must change for the agent to behave differently, and thus provides insight into its decision-making process. Unlike the other forms of explanation, this form is currently associated with a single

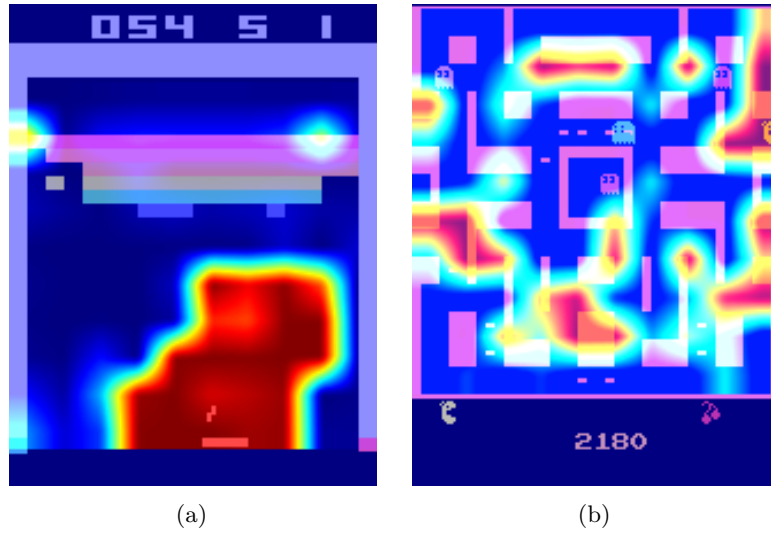


Figure 4.5: Attention-based heatmaps for the Atari games “**Breakout**” and “**Ms. Pac-Man**” [Mott et al., 2019].

1826 family of methods, based on the *use of the Latent Space (LS) for counterfactual*
 1827 *generation*.

1828 *Latent Space Usage for Counterfactual Generation*

1829 This family of methods uses generative models to build counterfactuals from the
 1830 Latent Representation (LR) of the observation [Bond-Taylor et al., 2021, Olson
 1831 et al., 2021]. The observation is encoded into the LS, its representation is modified
 1832 until the agent selects the desired action, and this modified representation is
 1833 decoded back into a new observation. Working in the LS rather than at the pixel
 1834 level keeps the counterfactual close to the original state and consistent with the
 1835 environment. The fidelity of the generated observations to the real game can
 1836 then be assessed through user studies [Olson et al., 2021].

1837 Figure 4.6 illustrates this approach on the Atari game *Space Invaders*
 1838 [Olson et al., 2021]. In the original observation (Figure 4.6a) the agent selects
 1839 “LEFT”, whereas in the counterfactual observation (Figure 4.6b) it selects
 1840 “RIGHT_FIRE”. Here, removing an enemy is enough to change the action,
 1841 which suggests that the position of that enemy influences the agent’s decision.
 1842 Such observations remain only indicative. Identifying a reliable pattern requires
 1843 repeating the analysis across many situations.

1844 4.3.4 Agent Trajectory

1845 Until now, the presented methods have focused on explaining individual decisions
 1846 made by the agent. The methods considered from this point on aim to provide
 1847 insight into the agent’s strategy by analyzing its behavior over time.

1848 The most direct approach consists in visualizing agent trajectories, most
 1849 commonly through complete episodes. By analyzing these trajectories, the
 1850 observer can infer the dynamics underlying the agent’s behavior and how its

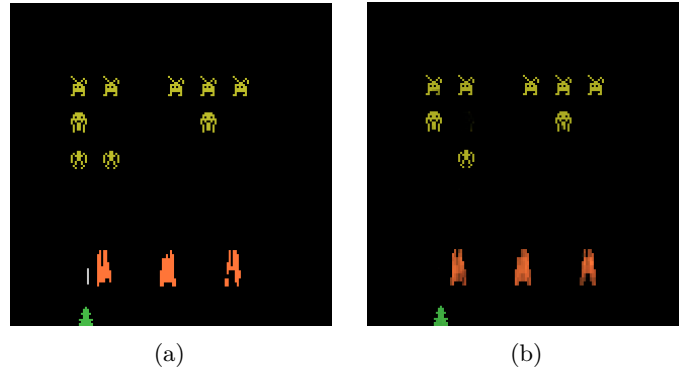


Figure 4.6: Counterfactual explanation of the agent’s behavior for the Atari game *Space Invaders* [Olson et al., 2021]. Panel a shows the original observation and panel b the counterfactual observation produced by a generative model.

decisions evolve through time. This type of visualization is not specific to XRL. In practice, visualizing agent episodes is a standard step in the development and evaluation of RL. It provides a simple way to verify that the agent behaves as expected and to detect potential implementation errors.

This simple approach can be further enhanced using families of methods such as *hierarchical decomposition* and *extraction via critic analysis*.

Extraction via Critic Analysis

In this family of methods, the predictions of the critic are used to determine which trajectories to present to the observer [Sequeira and Gervasio, 2020]. A set of states is selected from multiple trajectories. For a given state, all possible actions are examined. The state is assigned a value equal to the difference between the critic’s prediction for the lowest-valued action and the prediction for the highest-valued action. Trajectories containing the states with the highest values according to this procedure are then chosen to be presented to the observer. These trajectories are considered critical moments of the agent and are therefore relevant for analysis. For this family of methods, the *selected trajectories* represent the interpretable information provided to the observer.

Hierarchical Decomposition

Hierarchical agents are composed of a set of sub-policies specialized for specific tasks within subsets of the environment’s states. This family of methods distributes the optimization of the reward function across multiple models, significantly simplifying the learning process. Moreover, this decomposition of the policy function can also provide explainability in the agent’s strategy [Beyret et al., 2019]. If a sub-policy is selected, it indicates that the agent will execute a specialized behavior. This behavioral specialization provides insight into the strategy the agent employs to maximize the reward function. The agent’s *trajectory*, together with the *sub-policy* selected for each action, is used as interpretable information.

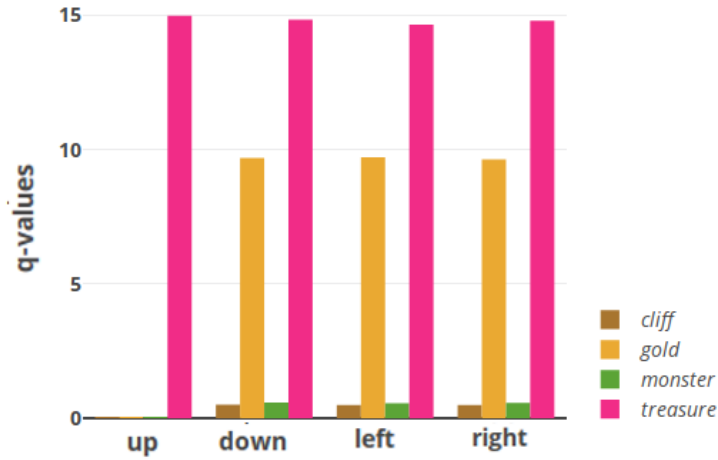


Figure 4.7: Prediction of sub-rewards as a function of the action selected by the agent for a given state [Juozapaitis et al., 2019].

4.3.5 Trajectory Prediction

Making predictions about the future of a trajectory allows one to form a representation of what the system expects, based on its training experience. These predictions are produced using value functions. Such functions provide insights into the future evolution of the agent’s trajectory. By their nature, this form of explanation informs the observer about the strategy adopted by the agent. Value functions provide an estimate of the expected cumulative reward from a given state. Consequently, a value function maps an observation to a scalar value in $\mathbb{R}$.

This aggregated prediction provides limited information about the underlying factors that contribute to this value. It does not directly reveal which aspects of the task are being optimized or how the predicted return is composed. Unlike the other forms of explanation, this form is currently associated with a single family of methods, aimed at improving the interpretability of value-based explanations: *critic enrichment*.

Critic Enrichment

In order to obtain interpretable information from the predictions of the critic, it is possible to decompose the critic into multiple sub-functions [Juozapaitis et al., 2019]. The reward function is thus decomposed into multiple sub-reward functions, each representing a sub-objective of the task to be accomplished. Similarly, for each of these sub-reward functions, corresponding sub-critic functions are defined, along with a function that combines their outputs. During training, the agent selects its actions so as to maximize this combination function of the sub-critics. Using this approach, for each observation and action, a prediction is obtained regarding the sub-objectives the agent is attempting to maximize, so that the action chosen by the agent reveals which sub-reward it aims to maximize (see Figure 4.7). This *prediction* is then used as the explanation for the observer.

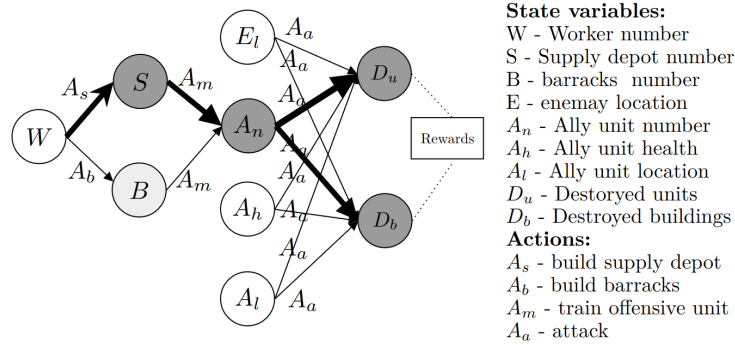


Figure 4.8: Agent-environment interaction model for the video game *StarCraft II* [Madumal et al., 2020].

4.3.6 Interaction Model

Modeling the interactions between an agent and its environment makes it possible to understand how the agent’s decisions are made and what impact they have on the environment. By incorporating these interactions into an explanatory model, one can better grasp the complex relationships between the agent and its environment, thereby identifying the determining factors in the decision-making process. By explicitly handling the agent-environment interaction, this type of explanation aims to explain the agent’s strategy. The families of methods used to obtain explanations in the form of agent-environment interactions are: *Bayesian network learning* and *LS usage for MDP generation*.

Bayesian Network Learning

The evolution of an agent within an environment can be observed as a set of random variables connected through a Bayesian network [Madumal et al., 2020]. Random variables are defined to represent both external aspects (the environment) and internal aspects of the agent (the actions). By analyzing trajectories, it becomes possible to infer causal relationships among these random variables. At the end of this process, a weighted causal graph is obtained, which serves as an explanation for the observer, as illustrated for the video game *StarCraft II* in Figure 4.8, where nodes represent random variables and edges represent causal relationships between them. This *Bayesian network*, representing the interactions between the agent and the environment, is then used as the explanation.

Latent Space Usage for MDP Generation

Each layer of a NN can be viewed as a projection from one space to another [Zahavy et al., 2017]. As information propagates through successive layers of a NN, it becomes increasingly abstract. It can therefore be assumed that two vectors that are close in this projected space are also close in terms of the agent’s perception. Based on this observation, it is possible to redefine an MDP by leveraging the abstract representations produced by the final layers of a NN.

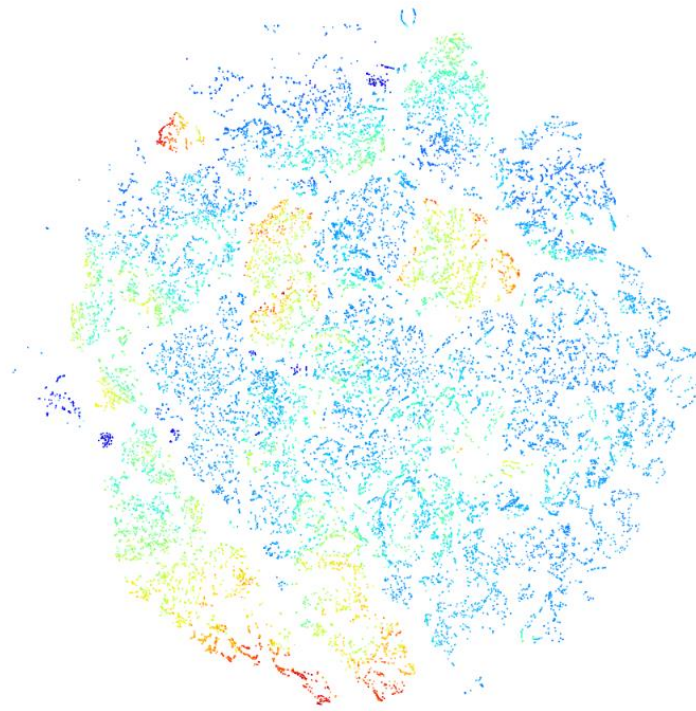


Figure 4.9: t -SNE projection of a neural network's latent space [Zahavy et al., 2017].

Once the new MDP has been constructed, a model of the interaction between the agent and its environment is obtained. This model, represented as a *graph*, is then used as the explanation.

Figure 4.9 illustrates this idea using a two-dimensional t -SNE projection of the LS of the NN used to represent the policy. Each point is colored according to the value function's prediction for the corresponding state, with warmer colors indicating higher predicted values. An agreement can be observed between the structure of this projected space and the critic's predictions: states that are assigned similar values by the critic also tend to be located close to one another in the projection. This agreement makes it possible to manually identify clusters of states, which likely correspond to states that are perceived as similar by the agent.

4.4 Conclusion

The taxonomy and the state-of-the-art review presented in this chapter make it possible to draw three main observations.

4.4.1 Absence of Explanation of the Learning Process

Regarding the subject of explanation, Table 4.1 shows that none of the reviewed methods addresses the learning process itself. Such an explanation would concern how the policy is progressively updated throughout training, and why it evolves over successive iterations to converge toward a particular solution. Several reasons explain this absence.

First, among the different machine learning paradigms, RL is arguably the one that would benefit the most from explanations of the learning process. Unlike supervised or unsupervised learning, it relies on an explicit exploration–exploitation trade-off, making the learning trajectory itself an informative subject of explanation. However, most XRL methods are inherited from the XAI literature, where explanations focus exclusively on the behavior of already-trained models. This limitation has naturally carried over to XRL.

Second, from a practical perspective, explanations of the learning process have limited interest. In most real-world applications, practitioners deploy an already-trained agent, while training itself takes place offline. Users are generally interested in understanding the behavior of the final policy rather than the optimization process that produced it.

Finally, explaining the learning process raises significant methodological challenges. Training in RL can be viewed as the iterative improvement of a policy through repeated interactions with the environment. In this sense, explaining learning would amount to explaining the evolution of an entire sequence of policies rather than a single one. Current XRL methods still struggle to provide comprehensive explanations of an individual policy, so extending them to a full training trajectory appears considerably more difficult.

4.4.2 Lack of Unified Metrics for Explainability

The diversity of explanation forms identified in this review also reveals another limitation shared by nearly all families of XRL methods, which is the absence of reliable metrics for evaluating explainability. In many studies, explanations are not evaluated with human participants. Instead, authors focus on technical properties of the proposed method, such as fidelity, sparsity, or computational efficiency. When human evaluations are conducted, they usually rely on subjective satisfaction questionnaires. Although useful, self-reported satisfaction remains prone to numerous biases and does not necessarily indicate that the explanation has improved the observer’s mental model or understanding of the agent.

This limitation is not specific to XRL, it reflects a broader challenge affecting XAI as a whole. In our view, this limitation comes from how explainability has developed in artificial intelligence. Most of the proposed approaches have been developed by the computer science community. Explainability has consequently often been studied from an algorithmic or mathematical perspective, with less attention given to how humans actually benefit from explanations.

Our definition of explanation explicitly places the observer at the center of the explanatory process. From this perspective, the value of an explanation should not only depend on its technical properties but also on its ability to improve the observer’s understanding. Addressing this challenge will require stronger interdisciplinary collaboration between computer science and disciplines such as cognitive science, psychology, philosophy, and sociology. These fields provide

the theoretical and experimental tools needed to study human understanding and to develop evaluation protocols that measure the actual effectiveness of explanations.

4.4.3 Toward Intrinsic Explainability in XRL

Our review also suggests that current XRL research remains largely dominated by substitutive explainability approaches. Replacing a deep model with a simpler one therefore risks losing precisely the complexity that makes deep RL effective. For this reason, we argue that a promising direction for XRL is to develop explanations directly from the NN that forms the agent’s policy. Such approaches aim to analyze the representations learned by the network itself. From this perspective, the NN should not necessarily be regarded as an black box, but as a complex representation whose apparent opacity may instead reflect the limitations of current analysis tools.

Among the methods reviewed, *latent space usage for MDP generation* (Section 4.3.6) appears closest to this intrinsic perspective. However, this approach still relies on a largely manual and qualitative analysis of the LS, which limits its reproducibility and its ability to produce systematic explanations. The next chapter builds upon this idea and proposes a mechanistic XRL approach based on LS analysis.

Chapter 5

Clustering Agent Representations

As discussed in the previous chapter, current explainability methods for Reinforcement Learning (RL) still lack intrinsic approaches capable of explaining the internal mechanisms of deep neural policies. To the best of our knowledge, only [Zahavy et al., 2017] have proposed exploiting the Latent Space (LS) learned by neural policies in order to identify the concepts used by the agent. Its main idea is to analyze the Latent Representation (LR) produced by the Neural Network (NN) and organize them into clusters that correspond to high-level concepts.

While the core idea of this paper is closely aligned with our vision of explainability in RL, this approach suffers from two important limitations. First, the clusters are manually constructed. Such a procedure heavily depends on the user’s expertise and neither guarantees reproducibility or scales. Second, no evaluation protocol is proposed to assess the quality of the obtained clusters. Their relevance is essentially determined through visual inspection.

The Clustering Agent Representation (CAR) method proposed in this chapter addresses these two limitations. First, it automates concept discovery by relying on unsupervised clustering algorithms to identify the Concept Structure (CS) of the LS. Second, it introduces an objective evaluation metric based on the stability of the extracted representations across independently trained policies.

This chapter is organized as follows:

- Section 5.1 lays out the theoretical assumptions underlying the proposed CAR method, which we term the *Representation Convergence Hypothesis* (Hyp. 4).
- Section 5.2 then details the CAR method itself, covering both the extraction of LR using surrogate policies and the automatic detection of CS through clustering.
- Section 5.3 introduces the metrics we propose for evaluation.
- Section 5.4 situates CAR within the broader landscape of mechanistic interpretability.

5.1 Representation Convergence Hypothesis

The CAR method relies on a assumption, referred to as the *Representation Convergence Hypothesis* (Hyp. 4). This hypothesis follows the two assumptions established previously throughout this manuscript.

The *Mechanistic Hypothesis* (Hyp. 3) rejects the notion that deep NN constitute irreducible black boxes. Adopting a mechanistic perspective, we consider that every process results from a sequence of causal computations. Although these computations may be complex, they remain explainable through the analysis of the internal mechanisms.

The *Concept Hypothesis* (Hyp. 1) states that NN achieve generalization by learning concepts. During optimization, the network progressively organizes its LS into representations that capture the information required to solve the task.

Following these two hypotheses, it becomes possible to explain a NN by studying the organization of its LS. This family of approaches is referred to as mechanistic interpretability. Since the CAR method is grounded in these two hypotheses, it falls within this field of research. More specifically, CAR focuses on a specific aspect of mechanistic interpretability known as universality. Universality refers to the consistency with which a concept is recovered across different models trained on the same task [Templeton et al., 2024].

To provide a explanation for this phenomenon, we developed the *Representation Convergence Hypothesis* (Hyp. 4). This hypothesis states that, when NN are trained to solve the same task, a set of concepts should systematically emerge because these concepts provide a effective way of representing the problem. As a result, independently trained models should develop similar concepts in their learned representations.

Hypothesis 4: Representation Convergence Hypothesis

For a given task, independently trained NN tend to develop similar sets of concepts because these concepts provide an efficient representation for solving the task.

The CAR method is designed to investigate the *Representation Convergence Hypothesis* (Hyp. 4) by assessing whether it can consistently recover similar concepts across independently trained models. The hypothesis therefore serves both as the foundation of the method and as the phenomenon it aims to investigate. If CAR consistently recovers the same set of concepts across different training runs, this set is more likely to reflect invariant properties of the task than model-specific artifacts. Conversely, the absence of representation convergence would provide evidence against the hypothesis and would also question the validity of the CAR approach.

It is important to clarify that this hypothesis does not imply convergence at the parameter level. Independently trained models are expected to converge toward different sets of weights. The proposed hypothesis concerns a abstract level of representation. We assume that the conceptual organization induced by their LR converges toward an equivalent CS, even though the numerical implementation of the networks differ.

The remainder of this chapter develops the methodology used to investigate this hypothesis.

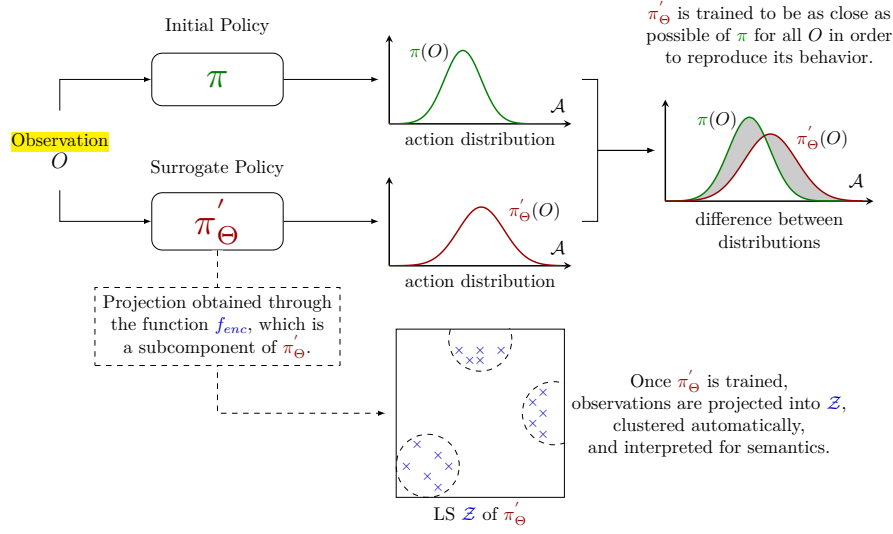


Figure 5.1: Illustration of the Clustering Agent Representation method.

5.2 Latent Clustering of Surrogate Policies

Following the work of [Zahavy et al., 2017], we analyze the LS through clustering techniques, placing our approach within the broader field of Deep Clustering. This design choice is motivated by our rejection of the *Superposition Hypothesis* (Hyp. 2) and, more generally, of any view that assumes a linearly organized CS, as discussed in Section 2.5.

The CAR methodology is composed of two main stages. The first stage trains multiple independent NN, called surrogate policies, which implement the same decision policy. The second stage clusters their LR in order to identify the underlying conceptual organization. Once these two stages have been completed, the resulting clustering can be analyzed from two complementary perspectives. First, the consistency of the resulting clusters across independent training runs can be assessed to determine whether the identified clusters are stable. Second, the semantic content of the clusters can be examined to determine whether they correspond to meaningful concepts. Figure 5.1 provides a visual overview of the two stages, from the imitation of the initial policy to the clustering of the surrogate representations.

This section is organized into three parts. The first explains why multiple surrogate policies are used to study the stability of the representations. The second describes how these policies are trained to match the initial policy as closely as possible. The third presents how their LR are clustered to identify the CS.

5.2.1 Multiple surrogate policies

The CAR method was developed to test the *Representation Convergence Hypothesis* (Hyp. 4). This requires training several NN independently and comparing their LR. Consistent similarities in the resulting CS would provide empirical support for this hypothesis.

In supervised learning, this protocol is straightforward since several models can be independently trained on the same target function. RL introduces an additional difficulty. Because of stochastic exploration, independently trained agents are not guaranteed to converge toward the same policy. Even if they achieve comparable performance, they may solve the task using different strategies. This makes it difficult to determine whether differences in the LS result from different representations of the same strategy or from differences in the learned strategies themselves.

To overcome this limitation, we consider an already trained initial policy. This policy is then used as a target for imitation learning [Ross et al., 2011]. In this process, several surrogate NN are independently trained in a supervised manner to reproduce its behavior. This makes it possible to investigate whether equivalent policies also exhibit equivalent conceptual organizations.

It is important to emphasize that the use of surrogate policies does not make CAR a surrogate explainability method, as defined in Subsection 4.1.2. The surrogate policies remain deep NN. Their role is not to simplify the decision process but to provide multiple independent realizations of the same policy function, enabling the study of representation convergence.

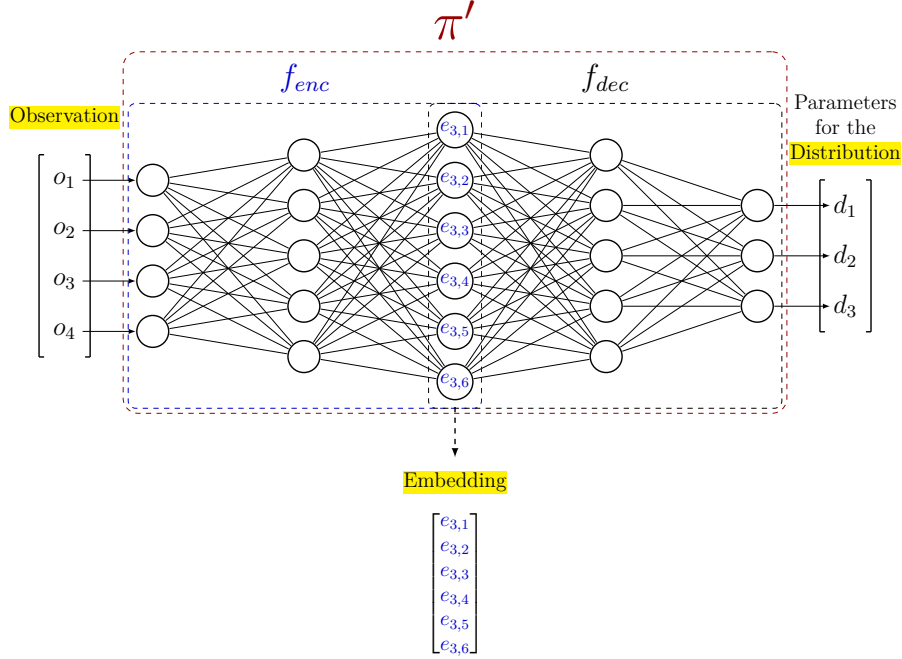
5.2.2 Training surrogate policies

Throughout the remainder of this manuscript, the initial policy we aim to explain is denoted as π . This initial policy is obtained without imposing any constraints on its origin. It may result from classical or deep reinforcement learning [Sutton and Barto, 2018], alpha-beta algorithm [Knuth and Moore, 1975], or even from demonstrations provided by humans [Ross and Bagnell, 2010]. The crucial requirement is the availability of a dataset containing a sufficient number of projections from the observation space $\mathcal{O}$ to the action space $\mathcal{A}$ induced by π to train a NN.

The replicated policies are referred to as surrogate policies. To assess the stability of the learned representations, n surrogate policies are trained independently, denoted by $\pi'_1, \dots, \pi'_n$. All surrogate policies share the same architecture. It consists of a NN with two components, an encoder f_{enc} and a decoder f_{dec} , highlighted by the blue and black dashed rectangles, respectively, in Figure 5.2. The encoder maps the observation O to the embedding, and the decoder reconstructs the parameters of the action distribution from this embedding. For a given surrogate policy π'_i , parameterized by Θ_i , its encoder and decoder are written f_{enc_i} and f_{dec_i} .

To train each surrogate policy π'_i to accurately replicate the action distribution, we define a loss function $\mathcal{L}$ optimized via gradient descent as introduced in Section 1.4. This loss, defined in Equation (5.1), is computed as the Mean Squared Error (MSE) between the parameters of the action distributions, obtained by applying the function p , which maps a distribution to its parameterization. In practice, this function p does not need to be explicitly defined, as neural policies directly output the parameters of the action distribution.

$$\min_{\Theta_i} \sum_{O \in \mathcal{O}} \text{MSE}\left(p(\pi(O)), p(\pi'_{\Theta_i}(O))\right) \quad (5.1)$$

Figure 5.2: Example of a surrogate policy architecture π' .

5.2.3 Cluster extraction

Once the surrogate policies are trained, their encoders are used to generate a point cloud from a set of observations. This point cloud provides a sample of the LS to be analyzed. The proposed CAR method adopts the deep clustering approach introduced in Section 2.5. Under this approach, conceptually similar observations are expected to produce nearby LR. A CS can therefore be viewed as a region of the LS containing related LR. To identify these regions, a clustering algorithm is applied to the point cloud.

The clustering is performed independently for each surrogate policy. For a surrogate policy π'_i , applying its encoder f_{enc_i} to a subset of observations $\mathbb{O} \subseteq \mathcal{O}$ yields the set of embeddings $\mathcal{E}_i$. A clustering function f_{clust} then partitions the point cloud $\mathcal{E}_i$ into groups of nearby LR, assigning each observation a cluster label and yielding the partition $\mathcal{C}_i$. The same observation set $\mathbb{O}$ is used for every surrogate policy, so that the partitions $\mathcal{C}_1, \dots, \mathcal{C}_n$ are defined over the same observations and can be compared. This comparison is the basis of the evaluation introduced in the following section. Algorithm 5 states the complete CAR procedure.

Two choices govern this analysis without being part of the clustering algorithm itself. The first is the choice of layer used to extract the representations. Each hidden layer defines its own LS, reflecting the information represented at that stage of the network. CAR can therefore be applied to the LR extracted from any layer. This allows the conceptual organization of the policy to be examined at different levels of the network.

The second is the number of clusters. CAR does not assume that a task

Algorithm 5: Clustering Agent Representation (CAR)**Input:**Initial policy π Observation dataset $\mathbb{O}$ Number of surrogate policies n Surrogate policy architecture π' (encoder f_{enc} , decoder f_{dec})Learning rate η Clustering function f_{clust} **Output:**Trained surrogate policies $\pi'_{\Theta_1}, \dots, \pi'_{\Theta_n}$ Clusterings $\mathcal{C}_1, \dots, \mathcal{C}_n$ *Behavioral cloning dataset built from the target policy π* **1** $\mathcal{D} \leftarrow \{(O, p(\pi(O))) \mid O \in \mathbb{O}\};$ *MSE loss between action-distribution parameters, as defined in Equation (??)***2** $\mathcal{L} \leftarrow \text{MSE};$ *Independently train n surrogate policies by behavioral cloning***3** **for** $i = 1, \dots, n$ **do****4** Initialize Θ_i randomly;*Behavioral cloning of π via SGD, see Algorithm 1***5** $\Theta_i \leftarrow \text{SGD}(\mathcal{D}, \pi', \mathcal{L}, \eta, \Theta_i);$ *Cluster the latent representations of each surrogate policy***6** **for** $i = 1, \dots, n$ **do***Embeddings produced by the encoder of π'_{Θ_i}* **7** $\mathcal{E}_i \leftarrow \{f_{enc_i}(O) \mid O \in \mathbb{O}\};$ *Partition of the embedded observations into clusters***8** $\mathcal{C}_i \leftarrow f_{clust}(\mathcal{E}_i);$ **9** **Return** $(\pi'_{\Theta_1}, \dots, \pi'_{\Theta_n}), (\mathcal{C}_1, \dots, \mathcal{C}_n)$

2184 admits a single correct number of concepts. This choice instead determines the
 2185 granularity at which the conceptual organization is examined. A small number
 2186 reveals broad, strategic concepts, while a larger number reveals finer, tactical
 2187 ones. The influence of both choices is studied in Chapter 6.

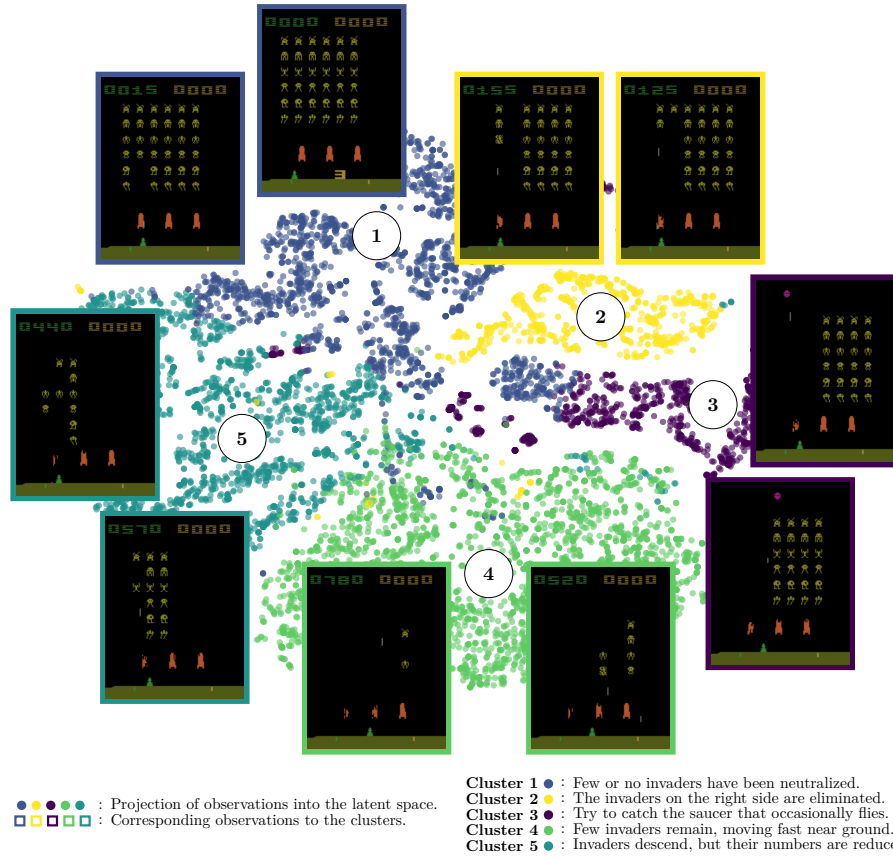


Figure 5.3: Two-dimensional projection of the LS embeddings of a surrogate policy trained on **Space Invaders**, together with the CAR clusters and their corresponding observations.

Figure 5.3 shows the outcome of this procedure for a surrogate policy trained on **Space Invaders**, using five clusters. The clusters are computed in the original high-dimensional LS. The point cloud is a two-dimensional projection used only for visualization. Each cluster gathers observations that correspond to a distinct, interpretable stage of a game:

- **Cluster 1:** The start of a game, with few or no invaders neutralized.
- **Cluster 2:** The right-hand group of invaders eliminated.
- **Cluster 3:** The pursuit of the bonus saucer.
- **Cluster 4:** The end of a game, with a few fast invaders close to the ground.
- **Cluster 5:** The middle of a game, with the invaders descending but already thinned out.

The position of an observation in the LS thus reflects the stage of the environment it belongs to, suggesting a correspondence between the geometry of the LS and the semantic properties of environment states.

This example illustrates what CAR is expected to produce, but it relies on the visual inspection of a single surrogate policy, which is precisely the type of evidence whose limitations motivated this work. Three aspects must therefore be evaluated before such an analysis can be trusted. First, the surrogate policy must faithfully reproduce the initial policy so that its explanations remain relevant to the target policy. Second, the recovered conceptual organization must be consistent across independently trained surrogate policies. Finally, the clusters must capture genuine abstractions rather than simply reproducing structure already present in the observations or action distributions.

The following section introduces one evaluation metric for each of these aspects and details how it is computed.

5.3 Evaluation Metrics

To evaluate the effectiveness of CAR, four evaluation metrics are introduced:

- ***Behavior Reconstruction*** : it measures how well a surrogate policy faithfully reproduces the behavior of the initial policy by comparing their cumulative rewards.
- ***Clustering Universality*** : it evaluates the stability of clusterings by measuring the similarity of clusters obtained from independently trained surrogate policies.
- ***Abstraction Score*** : it quantifies the extent to which LS clusters capture abstractions rather than directly reflecting the structure of observations, of the action distribution, or the network architecture prior to training.
- ***Adjusted Clustering Universality***: it combines the two previous metrics to measure the agreement between surrogate policies beyond what can be explained by the baselines used in *Abstraction Score*.

The remainder of this section details the motivation for each metric and describes how they are calculated. In the first subsection, we develop the *Behavior Reconstruction*. We then introduce the Adjusted Rand Index (ARI) measure, which serves as the foundation for the three other metrics introduced in the subsequent subsections, namely *Clustering Universality*, *Abstraction Score*, and *Adjusted Clustering Universality*.

5.3.1 Behavior Reconstruction

We introduce the *Behavior Reconstruction* metric to assess whether π' faithfully reproduces the behavior of π . This metric compares the returns obtained by the initial policy π and the surrogate policies π' over a set of evaluation episodes. We use the definition of return introduced in Section 3.3.4, where $g(t, \gamma)$ denotes the discounted sum of rewards collected along a trajectory t . Here, the return is computed over complete episodes with $\gamma = 1$, so g corresponds to the total reward collected during the episode. Each policy is evaluated over m episodes. We denote by g_{π}^i the return obtained by the initial policy in episode i , and by $g_{\pi'_j}^i$ the return obtained by surrogate policy π'_j in episode i . The *Behavior*

2244 *Reconstruction* is then defined as the mean return of the surrogate policies
 2245 normalized by the mean return of the initial policy:

$$\text{Behavior Reconstruction} = \frac{1}{n \sum_{i=1}^m g_{\pi}^i} \sum_{j=1}^n \sum_{k=1}^m g_{\pi'_j}^k \quad (5.2)$$

2246 In imitation learning, it is unlikely to expect the surrogate policy π' to
 2247 outperform the initial policy π it aims to approximate through supervised
 2248 learning [Ross and Bagnell, 2010]. Consequently this metric ranges in $[0, 1]$.
 2249 Values approaching 1 indicate minimal loss in reproducing π , while values close
 2250 to 0 indicate poor reproduction.

2251 We deliberately adopt a reward-based definition rather than alternatives based
 2252 on actions or action distributions (e.g., training loss, accuracy, KL divergence
 2253 [Shlens, 2014], or optimal transport [Peyré and Cuturi, 2020]). Since surrogate
 2254 policies are trained in a supervised manner, they cannot develop strategies
 2255 beyond those demonstrated by the initial policy. This means that if the rewards
 2256 achieved are comparable, the surrogate policy is effectively copying the initial
 2257 policy’s behavior.

2258 5.3.2 Adjusted Rand Index

2259 Within CAR, concepts are assumed to correspond to clusters in the LS. Com-
 2260 bined with the *Representation Convergence Hypothesis* (Hyp. 4), this suggests
 2261 that independently trained surrogate policies should develop similar conceptual
 2262 organizations. Their LR should therefore produce similar clusterings when
 2263 analyzed using the same clustering procedure.

2264 When a particular observation is processed by the network, it is expected to
 2265 activate the same concept regardless of the specific neural implementation of the
 2266 policy. If independently trained policies converge toward the same conceptual
 2267 organization, a given observation should activate the same concept across all
 2268 models. As a consequence, observations grouped together within a cluster for
 2269 one policy should also be grouped together for another independently trained
 2270 policy.

2271 In the clustering literature, the most used measures for comparing two par-
 2272 titions is the ARI [Hubert and Arabie, 1985]. This measure quantifies the
 2273 agreement between cluster assignments independently of the spatial organiza-
 2274 tion of the clusters. Both metrics introduced later in this chapter, *Clustering*
 2275 *Universality* and *Abstraction Score*, rely on the ARI. Since the ARI is a complex
 2276 measure, we review its definition and discuss its main properties.

2277 The ARI is derived from the Rand Index (RI) [Rand, 1971], which measures
 2278 the agreement between two partitions by considering every possible pair of points
 2279 in the point cloud. For each pair, four situations are possible:

- 2280 • **Agreement:** The points belong to the same cluster in both partitions.
- 2281 • **Agreement:** The points belong to different clusters in both partitions.
- 2282 • **Disagreement:** The points belong to the same cluster in the first partition
 2283 but to different clusters in the second.
- 2284 • **Disagreement:** The points belong to different clusters in the first partition
 2285 but to the same cluster in the second.

The RI measures the proportion of pairs for which the two partitions agree:

$$\text{RI} = \frac{\text{number of agreeing pairs}}{\text{total number of pairs}}.$$

2286 Although the RI is intuitive, it suffers from an important limitation. In most
 2287 clustering problems, the majority of pairs of points belong to different clusters.
 2288 Consequently, even two completely independent partitions will agree on a large
 2289 number of pairs, simply because both assign them to different clusters. This
 2290 effect becomes more pronounced as the number of points or clusters increases.

2291 The ARI addresses this issue by accounting for the agreement that would
 2292 be expected purely by chance. It compares the observed agreement RI to
 2293 the expected agreement under random partitions $\mathbb{E}[\text{RI}]$. The score is then
 2294 normalized by the maximum agreement achievable beyond this random baseline,
 2295 $\max(\text{RI}) - \mathbb{E}[\text{RI}]$. As a result, an ARI of 0 corresponds to the level of agreement
 2296 expected from random clusterings, while an ARI of 1 indicates perfect agreement
 2297 between the two partitions:

$$\text{ARI} = \frac{\text{RI} - \mathbb{E}[\text{RI}]}{\max(\text{RI}) - \mathbb{E}[\text{RI}]}$$

2298 In practice, the ARI is computed directly from the cluster assignments of the
 2299 two partitions. Let clustering A be composed of partitions $\{A_i\}$ and clustering B
 2300 of partitions $\{B_j\}$. Let $a_i = |A_i|$ denote the number of samples in partition i of
 2301 clustering A , and $b_j = |B_j|$ the number of samples in partition j of clustering B .
 2302 Furthermore, let $n_{ij} = |A_i \cap B_j|$ be the number of samples shared by partition i
 2303 of clustering A and partition j of clustering B , and let n be the total number of
 2304 samples. With these definitions, the ARI can be computed as:

$$\text{ARI} = \frac{\sum_{i,j} \binom{n_{ij}}{2} - \left[\sum_i \binom{a_i}{2} \sum_j \binom{b_j}{2} \right] / \binom{n}{2}}{\frac{1}{2} \left[\sum_i \binom{a_i}{2} + \sum_j \binom{b_j}{2} \right] - \left[\sum_i \binom{a_i}{2} \sum_j \binom{b_j}{2} \right] / \binom{n}{2}}$$

2305 Because it is bounded by 1, a value such as 0.5 can be misleadingly interpreted
 2306 as “halfway to a perfect match.” This interpretation is incorrect because the
 2307 ARI is normalized against the agreement expected from two random partitions.
 2308 As a result, values close to 1 require the two clusterings to be nearly identical,
 2309 while a value of 0.5 already indicates substantial agreement [Amezquita et al.,
 2310 2020].

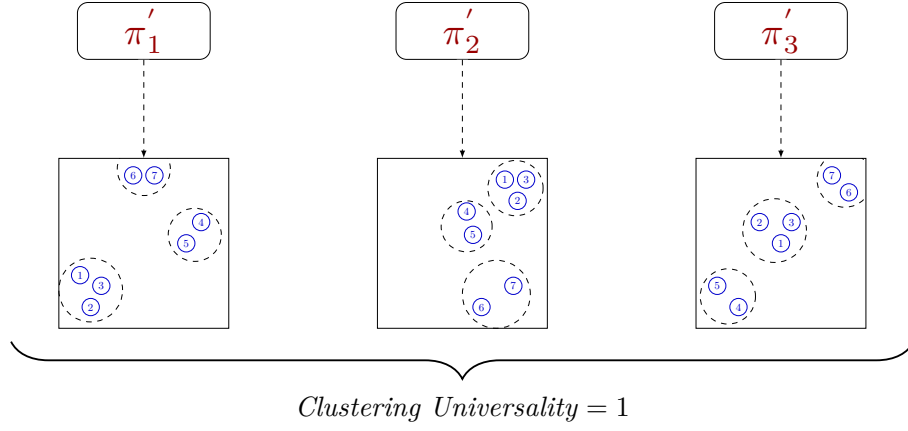


Figure 5.4: Illustration of *Clustering Universality* in the ideal case where projections of the same observations are grouped within the same clusters across all surrogate policies.

5.3.3 Clustering Universality

Having introduced the ARI, we can now define the *Clustering Universality* metric. While the ARI quantifies the similarity between two partitions, the objective of CAR is to assess the consistency of conceptual organizations across an entire set of independently trained surrogate policies.

To this end, the same set of observations is projected into the LS of each surrogate policy and clustered. Under the *Representation Convergence Hypothesis* (Hyp. 4), these independently obtained partitions are expected to be similar. *Clustering Universality* quantifies this by computing the mean pairwise ARI:

$$\text{Clustering Universality} = \frac{1}{\binom{n}{2}} \sum_{i=2}^n \sum_{j=1}^{i-1} \text{ARI}(\mathcal{C}_i, \mathcal{C}_j). \quad (5.3)$$

This metric takes values in $[-1, 1]$. A value close to 1 indicates that surrogate policies produce highly similar partitions, suggesting that the clusters reflect intrinsic properties of the policy. Conversely, a value near 0 implies that the similarity between partitions is no better than that of two randomly generated clusterings, meaning that the clustering fails to capture any meaningful structure in the policy. A negative value, on the other hand, signals that the agreement between partitions is worse than what would be expected by chance, indicating a systematic disagreement between the clusterings.

Figure 5.4 illustrates *Clustering Universality* in the ideal case. It shows three LS obtained by projecting the same seven observations, labeled 1 to 7, through three independently trained surrogate policies. Although the embeddings occupy different positions, the clusters group exactly the same elements in all three cases. The partitioning is therefore identical, resulting in an ARI of 1.

5.3.4 Abstraction Score

If *Clustering Universality* is high, it becomes important to determine what underlies this stability. Two possibilities arise:

- (i) the LS provides no abstraction, as defined in Definition 2.1.1, because it either reproduces the observation space or action distribution, or reflects a structure already present in the network architecture before training;
- (ii) the LS captures similarities in decision-making, resulting in behaviorally meaningful clusters.

To distinguish among these cases, we introduce the *Abstraction Score*. This metric contextualizes latent clusterings by examining how their patterns align with three reference baselines: the raw observations, the action distribution of the initial policy, and an untrained network of the same architecture.

We consider $\mathcal{C}_{obs}$, the clustering of the observation space $\mathbb{O}$, and $\mathcal{C}_{act}$, the clustering of the action distribution of the initial policy. Concretely, $\mathcal{C}_{act}$ is computed on the parameters of this distribution (the logits, before the softmax) produced by the initial policy for the same observations, which are precisely the quantities the surrogate policies are trained to reproduce. We cluster these continuous parameters rather than the sampled discrete action, since a discrete action takes only a few distinct values, which would limit the number of clusters that can be distinguished. Both clusterings use the same algorithm, distance metric, and hyperparameters as those used for $\mathcal{C}_i$ in the LS. This ensures that the three clusterings are obtained under the same conditions.

The third baseline addresses the case where the LS merely reflects an inductive bias of the network architecture rather than something learned. To this end, we introduce m untrained networks sharing the same architecture as the surrogate policies, but with randomly initialized, never-updated weights. Each of these m untrained networks is used to project the same observations into its own latent space, which is then clustered in the same way, yielding partitions $\mathcal{C}_{untr,1}, \dots, \mathcal{C}_{untr,m}$. This baseline isolates what the architecture already imposes on the representation before any learning takes place, so that a high similarity between $\mathcal{C}_i$ and this baseline indicates that the corresponding structure cannot be attributed to training.

For each latent clustering $\mathcal{C}_i$, we define its similarity to these three baselines using the ARI:

$$\begin{aligned} A_i^{obs} &= \text{ARI}(\mathcal{C}_i, \mathcal{C}_{obs}) \\ A_i^{act} &= \text{ARI}(\mathcal{C}_i, \mathcal{C}_{act}) \\ A_i^{untr} &= \frac{1}{m} \sum_{k=1}^m \text{ARI}(\mathcal{C}_i, \mathcal{C}_{untr,k}) \end{aligned}$$

The *Abstraction Score* is subsequently calculated as one minus the average of the maximum absolute similarity values across these three reference baselines:

$$\text{Abstraction Score} = 1 - \frac{1}{n} \sum_{i=1}^n \max(|A_i^{obs}|, |A_i^{act}|, |A_i^{untr}|) \quad (5.4)$$

The *Abstraction Score* ranges from $[0, 1]$. A score around 0 means that the latent clusters are strongly aligned with the observation space, the action distribution, or the untrained network, reflecting no abstraction, case (i). A score close to 1 indicates that the latent clusters are independent of the observation space, the action distribution, and the network’s initialization, capturing abstractions that meaningfully reflect the agent’s behavior, case (ii).

5.3.5 Adjusted Clustering Universality

As discussed above, *Clustering Universality* alone does not indicate whether the recovered structure is meaningful. We therefore introduce the *Adjusted Clustering Universality*, which measures how much of the agreement between the different surrogate policies remains after accounting for the three trivial baselines.

$$\frac{\text{Adjusted Clustering Universality}}{\text{Clustering Universality}} = \frac{\text{Clustering Universality}}{\text{Clustering Universality}} - \frac{1}{n} \sum_{i=1}^n \max(|A_i^{obs}|, |A_i^{act}|, |A_i^{untr}|) \quad (5.5)$$

This metric ranges in $[-1, 1]$. A value close to 1 indicates that surrogate policies converge toward a partition that is both highly reproducible across seeds and clearly not explained by the observation space, the action distribution, or the network’s initialization, providing the strongest evidence for *Representation Convergence Hypothesis* (Hyp. 4). A value close to 0, or negative, indicates that whatever agreement exists between independently trained surrogate policies is largely trivial: the surrogate policies converge, but toward a partition that could just as easily emerge from raw pixel similarity, from the action distribution of the initial policy, or from an untrained network sharing the same architecture.

5.4 Conclusion

The CAR method was developed to investigate how concepts are organized within the representations of RL policies. It is based on three hypotheses introduced throughout this manuscript, the *Mechanistic Hypothesis* (Hyp. 3), the *Concept Hypothesis* (Hyp. 1), and the *Representation Convergence Hypothesis* (Hyp. 4).

CAR translates these assumptions into an empirical analysis. Multiple surrogate policies are trained independently to reproduce the same policy. Their LR are then clustered and compared to assess whether similar conceptual structures emerge across policies. The *Clustering Universality* and *Abstraction Score* provide quantitative measures for evaluating the consistency and abstraction of the resulting clusters.

This makes CAR a general procedure for analyzing agent representations. The analysis and evaluation metrics are fully automated and require no manual intervention or environment-specific adjustments. The method can therefore be applied to a wide range of RL settings and provides a basis for systematically studying the internal representations of neural policies.

The next chapter applies CAR to several environments to evaluate its ability to identify meaningful conceptual structures.

Chapter 6

Experiments

This chapter presents the experimental study of the Clustering Agent Representation (CAR) method introduced in the previous chapter. It focuses on evaluating the method’s ability to identify conceptual structures. The remainder of this chapter is organized as follows:

- Section 6.1 describes the software implementation of the CAR pipeline.
- Section 6.2 details the experimental configuration, covering the environments, the Reinforcement Learning (RL) setup, and the surrogate policy training.
- Section 6.3 presents the training of the initial and surrogate policies and evaluates how faithfully the surrogate policies reproduce the behavior of the initial policy.
- Section 6.4 provides a quantitative analysis of the Latent Representations (LRs) using the *Clustering Universality* (CU), *Abstraction Score* (AS), and *Adjusted Clustering Universality* (ACU) metrics.
- Section 6.5 provides a qualitative analysis of the resulting clusters by manually examining their semantic content.
- Section 6.6 summarizes the findings and discusses their implications.

6.1 Software Implementation

This section details the experimental evaluation of the proposed method. Deep Learning (DL) and RL are strongly empirical fields, where experimental evaluation plays an important role in assessing how methods perform in practice. Particular attention is therefore given to the implementation of CAR. It relies on established libraries that provide efficient implementations of its main components. The code is made publicly available as open-source to support reproducible replication of the reported results.¹

The CAR implementation is organized as a pipeline of five stages. Each stage performs a specific operation and produces the artifacts required by the

¹https://github.com/Maxalaar/Clustering_Agent_Latent_Space

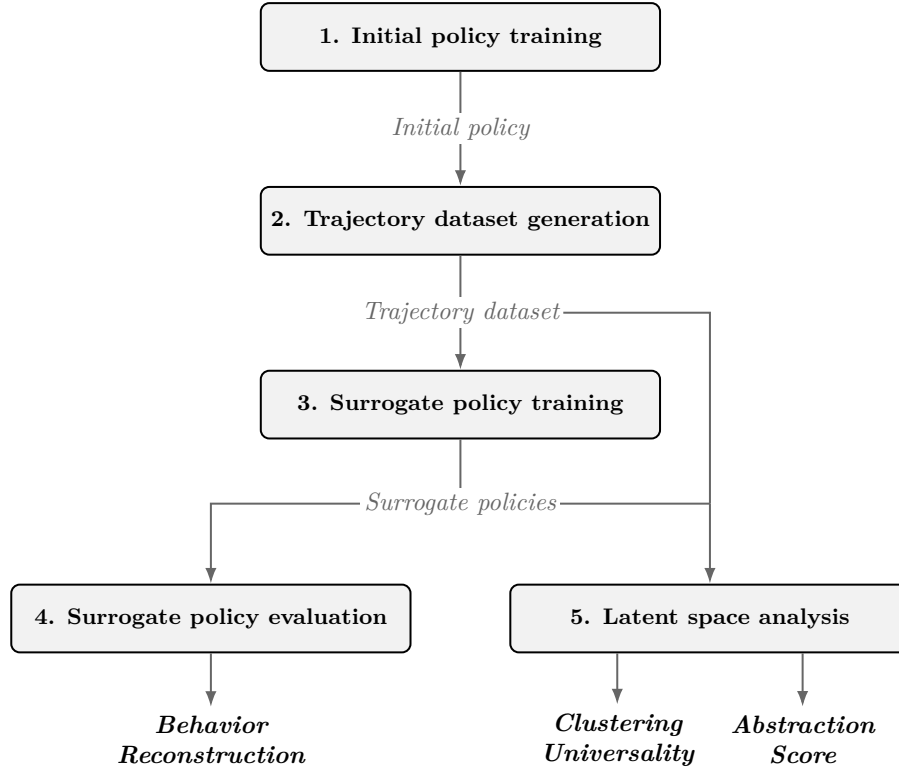


Figure 6.1: The CAR pipeline.

subsequent stages. The stages are implemented as independent executables allowing each stage to be rerun independently. The CAR pipeline is illustrated in Figure 6.1, and Table 6.1 summarizes the libraries used throughout the pipeline. The remainder of this section describes the purpose and implementation of each stage.

6.1.1 Initial policy training

The first stage trains the initial policy using RLlib [Moritz et al., 2018, Liang et al., 2018]. RLlib provides scalable implementations of RL algorithms and supports distributed training. The use of RLlib also allows CAR to be applied to environments implemented using the Gymnasium interface [Towers et al., 2024]. By default, the initial policy is trained using Proximal Policy Optimization (PPO) [Schulman et al., 2017]. Other RLlib algorithms, such as Asynchronous Proximal Policy Optimization (APPO) [Espeholt et al., 2018], Deep Q-Networks (DQN) [Mnih et al., 2013], and DreamerV3 [Hafner et al., 2023], can also be selected through the same configuration.

The policy and critic networks are implemented using PyTorch [Paszke et al., 2019]. PyTorch is the standard DL framework and is widely used for developing and training Neural Network (NN). It provides GPU acceleration and supports a wide range of NN architectures. This makes it well suited for implementing the different policy and critic architectures used in the experiments.

Pipeline Stage	Library	Description
Initial policy training	RLlib [Moritz et al., 2018, Liang et al., 2018]	Distributed RL library used to train the initial policy.
	PyTorch [Paszke et al., 2019]	Deep learning framework implementing the policy and critic networks.
	Gymnasium [Towers et al., 2024]	Common interface used to plug in any RL environment.
Trajectory dataset generation	RLlib [Moritz et al., 2018, Liang et al., 2018]	Distributed environment runners used to collect trajectories in parallel.
	Apache Arrow [Apache Arrow, 2016]	Columnar file format (Parquet) used to store the trajectory datasets.
Surrogate policy training	PyTorch Lightning [Falcon, 2019]	Structured training loop used for the behavior cloning of the surrogate policies.
Surrogate policy evaluation	RLlib [Moritz et al., 2018, Liang et al., 2018]	Wraps each surrogate policy as an RL policy to evaluate it in the environment.
Latent space analysis	cuML [RAPIDS, 2023]	GPU-accelerated clustering algorithms used to analyze the latent representations.

Table 6.1: Main software libraries used at each stage of the CAR pipeline.

6.1.2 Trajectory dataset generation

The second stage builds the dataset used to train the surrogate policies. RLlib is used to collect trajectories in parallel using multiple environment runners. For each step, the dataset stores the observation together with the parameters of the action distribution produced by the initial policy.

Trajectories are stored as Parquet files [Apache Arrow, 2016]. Its columnar structure allows efficient data access and supports concurrent processing. This is useful here because the datasets contain around one million observation–action pairs and cannot be loaded entirely into memory.

6.1.3 Surrogate policy training

The third stage trains the surrogate policies using Behavioral Cloning (BC) on the trajectory dataset. This provides a supervised learning approach for replicating the initial policy. The training procedure is implemented using PyTorch Lightning [Falcon, 2019]. PyTorch Lightning provides a simplified interface for training NN with PyTorch. It handles common training operations such as data loading, optimization, checkpointing, and GPU execution.

6.1.4 Surrogate policy evaluation

The fourth stage evaluates how faithfully the surrogate policies reproduce the behavior of the initial policy. Each surrogate is wrapped as an RLlib policy and evaluated in the original environment. Its mean cumulative reward over multiple episodes is compared with that of the initial policy. Evaluation episodes are collected using parallel environment runners.

This stage computes the *Behavior Reconstruction*. It therefore verifies that the surrogate policies reproduce the behavior of the initial policy sufficiently well before their LRs are analyzed. If a surrogate does not meet the required level of behavior reconstruction, its training can be repeated.

6.1.5 Latent space analysis

The fifth stage performs the main analysis of CAR. The trajectory dataset and trained surrogate policies are used to generate the LRs. These representations form the point clouds that are analyzed using clustering algorithms.

The clustering step relies on the cuML library [RAPIDS, 2023], which provides GPU-accelerated clustering algorithms. This reduces the computational cost and allows point clouds containing tens of thousands of representations to be analyzed.

The resulting partitions are used to compute the *Clustering Universality* and the *Abstraction Score*. The stage also produces the material required for the qualitative analysis, including the clustered point clouds and the cluster assignment of each observation.

6.2 Experimental Configuration

This section describes the configuration of the experiments reported in this chapter. It covers the environments, the neural architectures used, and the main parameters of the pipeline.

6.2.1 Environments

All experiments presented in this manuscript use environments from the Atari benchmark [Bellemare et al., 2013], accessed through the Arcade Learning Environment. The use of Atari environments offers several advantages:

- **Standard benchmark:** Atari is widely used in the RL community, making the experimental results easier to compare with existing work.
- **Limited environment-specific bias:** The environments were not specifically designed for RL, reducing the risk of introducing biases related to their design.
- **Diversity:** The benchmark provides a diverse set of well-known environments with different types of tasks and levels of complexity, allowing CAR to be evaluated under different conditions.

The configuration of the Atari environments follows common practice in RL research [Machado et al., 2017]. In each environment, the agent receives visual observations in the form of raw pixels. Each image is resized to 64×64 pixels and converted to grayscale to reduce memory requirements. To provide information about the dynamics of the environment, four consecutive observations are stacked along the channel dimension. The resulting observation has a shape of $64 \times 64 \times 4$. This allows the agent to infer the motion of objects from successive observations.

The actions correspond to the different buttons available on the Atari controller. The resulting action space is therefore discrete. Since Atari games are fully deterministic, sticky actions are introduced to add stochasticity to the trajectories. With a probability of 0.05, the environment repeats the action selected at the previous step instead of applying the new action. This prevents the agent from simply memorizing a fixed sequence of actions during training.

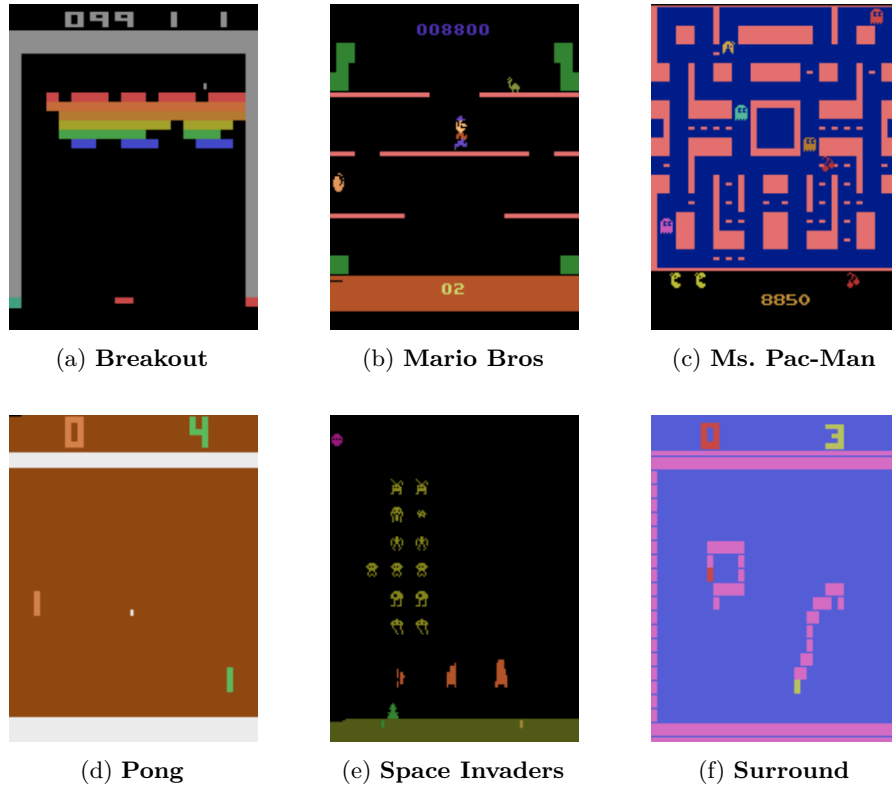


Figure 6.2: The six Atari environments used to evaluate CAR.

To reduce the duration of each episode, frame skipping is used. The agent selects a new action every five frames and repeats it for the following frames. This reduces the number of decisions required during an episode while maintaining sufficient temporal resolution for effective control.

The experiments use the six environments described below:

- **Breakout:** The agent moves a paddle at the bottom of the screen (Figure 6.2a) to bounce a ball into a wall of bricks. Each broken brick scores one point, and the ball speeds up as the wall is cleared.
- **Mario Bros.:** A single-screen platform game (Figure 6.2b) in which the player clears each phase by hitting enemies from below to flip them, then kicking them off the platforms before they recover. A central POW block can flip all enemies at once but can only be used a few times per phase.
- **Ms. Pac-Man:** The player moves through a maze (Figure 6.2c), eating pellets while avoiding four ghosts. Power pellets temporarily make the ghosts edible.
- **Pong:** A two-player game (Figure 6.2d) in which the agent controls one paddle and must send the ball past the opponent.
- **Space Invaders:** The agent moves a cannon along the bottom of the screen (Figure 6.2e), firing at descending rows of aliens while avoiding their

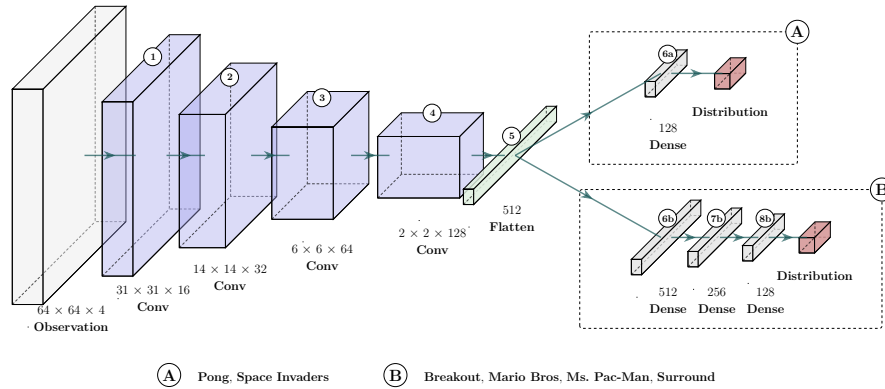


Figure 6.3: Architecture of the surrogate policies.

shots and using destructible shields. A bonus saucer occasionally crosses the top of the screen.

- **Surround:** A two-player game on a grid (Figure 6.2f) in which each player leaves a trail behind a moving cursor. The goal is to force the opponent to crash into a trail.

6.2.2 Neural architecture

The surrogate policies use a Convolutional Neural Network (CNN) to process the visual observations. The convolutional part consists of four layers with 16, 32, 64, and 128 channels, using 4×4 kernels with a stride of 2. Each convolutional layer is followed by a LeakyReLU activation. The resulting representation is then flattened and passed through a MultiLayer Perceptron (MLP) that produces the parameters of the action distribution.

The architecture of the MLP varies across environments. For **Pong** and **Space Invaders**, it contains a single hidden layer with 128 units. For the other four environments, it contains three hidden layers with 512, 256, and 128 units. The size of the output layer matches the number of actions available in each environment: 4 for **Breakout**, 18 for **Mario Bros**, 9 for **Ms. Pac-Man**, 6 for **Pong**, 6 for **Space Invaders**, and 5 for **Surround**.

Figure 6.3 illustrates both configurations, together with the shape of the representation produced by each convolutional layer. Throughout the remainder of this chapter, the analyzed representations are numbered from 1 to 5. These representations correspond to the four convolutional layers and the flattened representation, which are common to all six environments.

The representations produced by the MLP are numbered separately. For **Pong** and **Space Invaders**, which use a single hidden layer, the representation is numbered 6a. For the other four environments, which use three hidden layers, the representations are numbered 6b, 7b, and 8b.

6.2.3 Other CAR parameters

Several other parameters must be defined for the different stages of the CAR pipeline. These parameters are less critical to the overall methodology than those discussed above, but they still need to be specified to reproduce the experiments.

Environment configuration	
Frame size	64×64
Frames stacked per observation	4
Frame skip	5
Sticky-action probability	0.05
Initial policy training	
Algorithm	PPO
Optimizer	Adam
Learning rate	1.5×10^{-4}
GAE λ	0.95
Policy clipping ε	0.1
Value-function clipping	10
Entropy coefficient	0.01
KL coefficient	0.5
Epochs per update	10
Gradient clipping (global norm)	100
Surrogate policy training	
Surrogate policies per environment	3
Observation–action pairs per dataset	$\geq 10^6$
Batch size	2048
Optimizer	Adam
Learning rate	1×10^{-4}
Latent space analysis	
Embedding per surrogate policy	20 000
Clustering algorithm	k -means
Distance	Euclidean distance
Number of clusters K	2 to 20

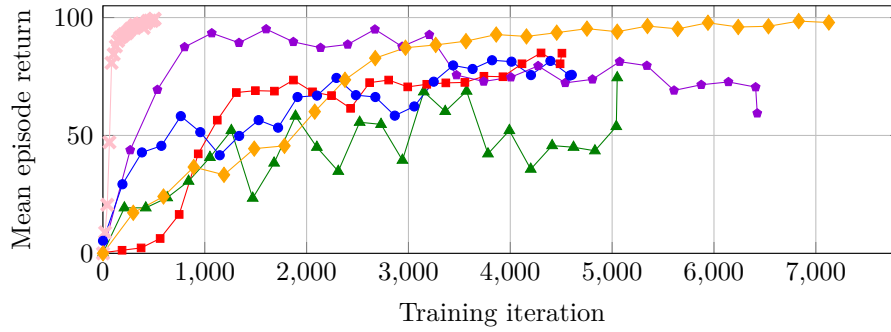
Table 6.2: Main experimental parameters.

Since each stage introduces its own configuration choices, there are too many parameters to discuss them all in detail. The main parameters used across the experiments are summarized in Table 6.2.

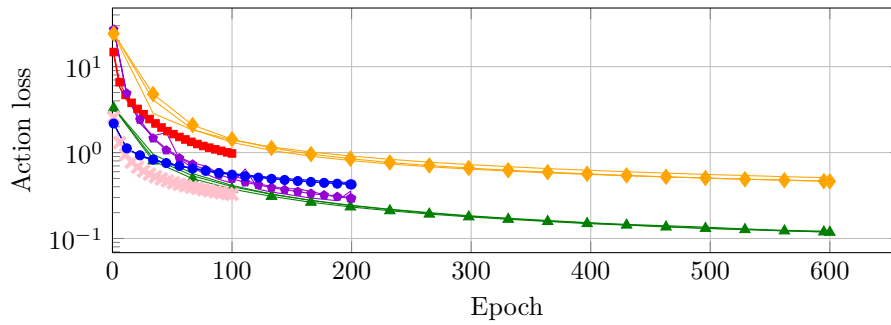
These settings are specific to the experiments presented in this chapter and do not restrict the CAR methodology itself. The different stages of the pipeline are modular and can be configured or replaced depending on the application. The evaluation metrics provide an objective basis for comparing these alternatives and assessing their effect on the consistency and meaningfulness of the resulting clusters.

6.3 Policy Performance and Surrogate Fidelity

This section reports the results of training the initial and surrogate policies for CAR, applied to the six environments. The first part verifies that the initial policies reach a competent level of play and evaluates how faithfully the surrogate policies reproduce their behavior using the *Behavior Reconstruction*.



(a) Training of the initial policies with PPO.



(b) Training action loss of the surrogate policies.

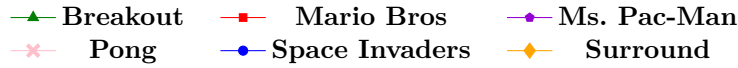


Figure 6.4: Training of the initial and surrogate policies.

6.3.1 Initial policy training

Figure 6.4a shows how the initial policies improve during PPO training. For each environment, the mean episode return is normalized so that 0% and 100% correspond to the lowest and highest mean return observed during that run. In every environment the return increases and then reaches a plateau, at which point training is stopped. The number of iterations required to reach this plateau varies from a few hundred, for **Pong**, to several thousand, for **Surround** and **Ms. Pac-Man**. For the subsequent analysis, the checkpoint with the best performance is retained rather than the final checkpoint.

To confirm that the trained policies play competently, their scores are compared with the reference scores reported by [Machado et al., 2017]. Each initial policy reaches a score of the same order as these references, so it constitutes a reasonable target for imitation.

6.3.2 Surrogate policy training

Figure 6.4b shows, for each environment, the training action loss of the three surrogate policies. In all cases, the loss decreases smoothly, and the three independently trained policies converge to the same value.

We further evaluate the ability of the surrogate policies to replicate the

Environment	<i>Behavior Reconstruction</i>
Breakout	0.83 $\pm$ 0.03
Mario Bros	0.99 $\pm$ 0.03
Ms. Pac-Man	1.00 $\pm$ 0.03
Pong	0.95 $\pm$ 0.01
Space Invaders	0.97 $\pm$ 0.07
Surround	1.00 $\pm$ 0.01

Table 6.3: Evaluation of *Behavior Reconstruction* across the six Atari environments.

behavior of the initial policies using the *Behavior Reconstruction* (Table 6.3). The surrogate policies reach a *Behavior Reconstruction* of 0.96 on average over 100 evaluation episodes, ranging from 0.83 for **Breakout** to 1.00 for **Ms. Pac-Man** and **Surround**. They therefore reproduce the decisions of the initial policies closely enough for their LR to be analyzed.

6.4 Clustering Quality of the Latent Space

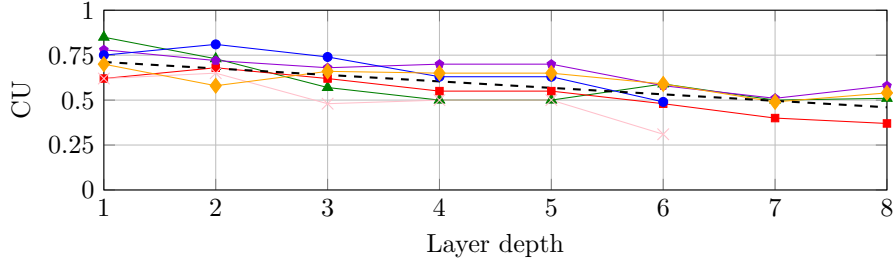
This section analyzes the Latent Space (LS) of the surrogate policies using CU, AS, and ACU. To do so, a random subset of 20 000 observations is selected for each environment. This subset is then embedded into the LS of each surrogate policy at each analyzed layer. This produces a point cloud for each combination of environment, surrogate policy, and layer. Each point cloud is partitioned using *k*-means clustering [MacQueen, 1967] for each number of clusters from 2 to 20.

The three metrics are then computed from these point clouds and their corresponding clusterings. The full results across all environments and analyzed layers are reported in Appendix 1. Two complementary analyses are then conducted on these results. First, the evolution of the three metrics across the different layers is examined to study the effect of representation depth. Second, for each environment, the layer achieving the highest average ACU is selected and analyzed in more detail across the different numbers of clusters.

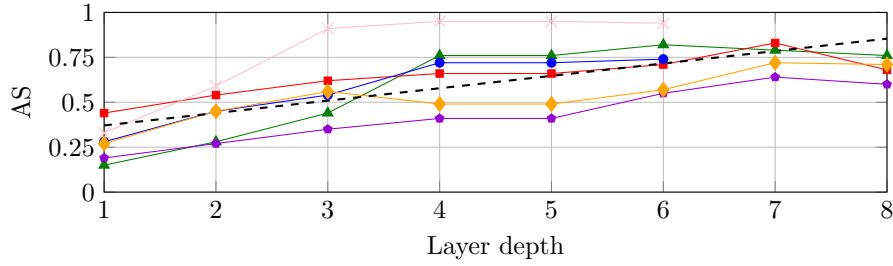
6.4.1 Analysis across layers

The first analysis examines how the three metrics evolve with the depth of the analyzed layer. For each layer, CU, AS, and ACU are averaged over the tested cluster numbers. Figure 6.5 reports these averages as a function of layer depth.

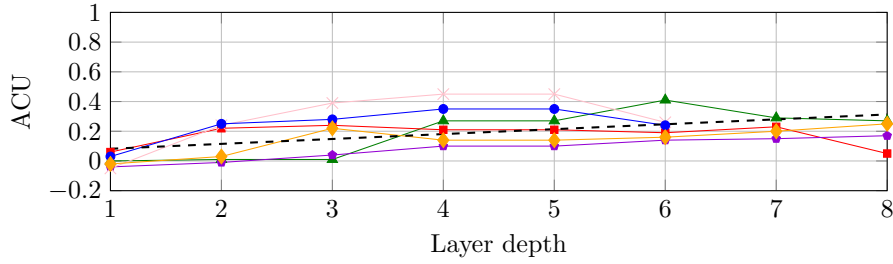
Across environments, CU tends to be highest in the earliest layers and decreases with depth. AS follows the opposite trend and generally increases as the representation becomes less tied to the raw observation, the action distribution, and the network’s initialization. This increase is however not monotonic in the deepest layers. For **Mario Bros** and **Ms. Pac-Man**, AS decreases at the last layer, from 0.83 to 0.68 and from 0.64 to 0.60 respectively, as the latent clustering becomes closer to the clustering of the action distribution, this layer being directly followed by the output layer. As ACU combines these two effects, its evolution depends on the environment.



(a) CU with respect to the layer depth.



(b) AS with respect to the layer depth.



(c) ACU with respect to the layer depth.

▲ Breakout ■ Mario Bros ◆ Ms. Pac-Man
 ✕ Pong ● Space Invaders ◆ Surround

Figure 6.5: CU, AS, and ACU averaged over cluster numbers as a function of layer depth.

The corresponding linear fits show that CU decreases from 0.72 at the first layer to 0.50 at the last, with a slope of approximately -0.04 per layer, while AS increases from 0.28 to 0.69, with a slope of approximately $+0.06$ per layer. Over the same range, ACU increases more moderately from 0.00 to 0.19, with a slope of approximately $+0.03$ per layer. This suggests that, despite the environment-specific variations, deeper representations tend on average to provide a better trade-off between CU and AS.

6.4.2 Analysis of the selected layers

The second analysis examines the effect of the number of clusters on the three metrics. For each environment, the layer with the highest average ACU across the tested cluster numbers is selected. Table 6.4 reports the selected layer,

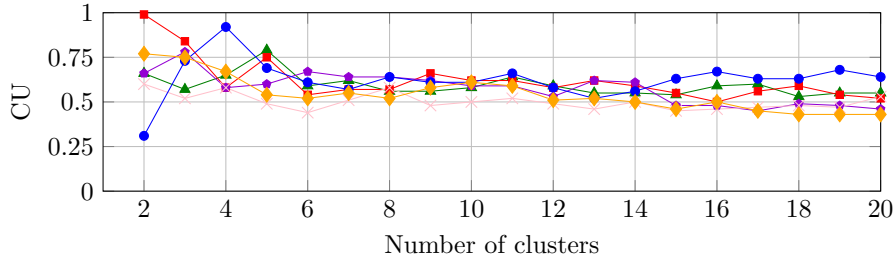
2650 together with the corresponding average CU, AS, and ACU.

2651 Figure 6.6 reports the evolution of CU, AS, and ACU as the number of clusters
2652 varies. **Pong** achieves the highest average ACU among the six environments,
2653 with a value of 0.45 ± 0.06 . This results from a moderate CU combined with
2654 an AS that saturates from the third layer onward. In contrast, **Ms. Pac-Man**
2655 achieves the lowest AS, with 0.60 ± 0.05 . The resulting ACU, 0.17 ± 0.06 , is the
2656 lowest across the six environments.

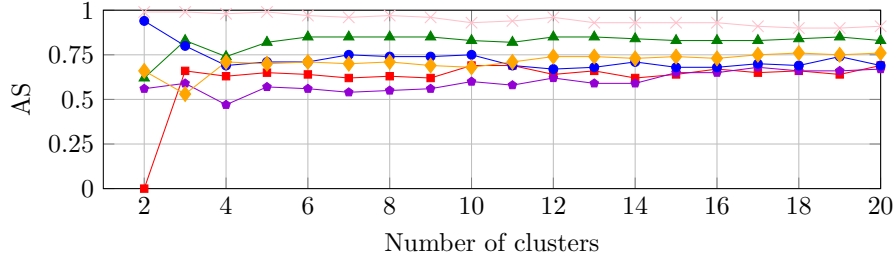
2657 The quantitative analysis provides a measure of the consistency and abstrac-
2658 tion of the identified clusters, but does not directly reveal their semantic content.
2659 The following section therefore provides a qualitative analysis of the clusters to
2660 examine the concepts they represent.

Environment	Layer	CU	AS	ACU
Breakout	6b	0.59 ± 0.06	0.82 ± 0.05	0.41 ± 0.06
Mario Bros	3	0.62 ± 0.12	0.62 ± 0.15	0.24 ± 0.10
Ms. Pac-Man	8b	0.58 ± 0.08	0.60 ± 0.05	0.17 ± 0.06
Pong	4	0.50 ± 0.04	0.95 ± 0.03	0.45 ± 0.06
Space Invaders	4	0.63 ± 0.11	0.73 ± 0.06	0.35 ± 0.09
Surround	8b	0.54 ± 0.10	0.71 ± 0.05	0.25 ± 0.06

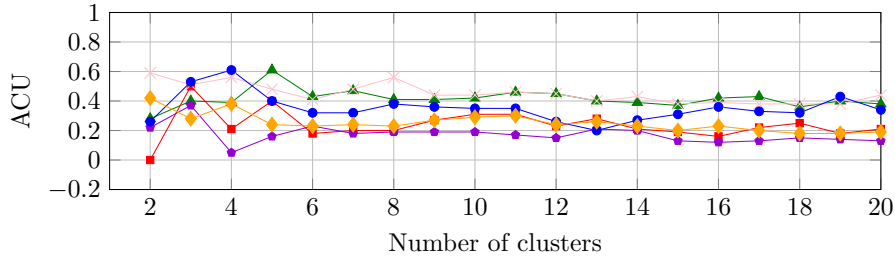
Table 6.4: Layer that maximizes the average ACU across $K \in [2, 20]$ for each environment, together with the corresponding average CU, AS, and ACU over K .



(a) CU with respect to the number of clusters.



(b) AS with respect to the number of clusters.



(c) ACU with respect to the number of clusters.



Figure 6.6: CU, AS, and ACU with respect to the number of clusters, for each of the six environments, for the layer that maximizes the average ACU in that environment.

2661 6.5 Semantic Interpretation of the Clusters

2662 The metrics reported in Section 6.4 evaluate the LS clusterings quantitatively
2663 but do not indicate what the resulting clusters actually represent. This section
2664 complements them with a qualitative, hand-labeled inspection of the clusters
2665 obtained for each environment.

2666 For each environment, two clusterings are selected for the qualitative analysis.
2667 They are obtained using two different numbers of clusters and may correspond
2668 to different layers of the network. The choice of the layer and the number of
2669 clusters is based on the following criteria:

- 2670 • **High ACU:** The selected clustering should achieve a sufficiently high
2671 ACU, indicating a good balance between consistency and abstraction.
- 2672 • **Manual analysis:** The number of clusters must remain small enough to
2673 allow the resulting clusters to be examined manually.
- 2674 • **Limited use of two-cluster partitions:** Clusterings with two clusters
2675 often achieve a high ACU, but provide limited information about the
2676 organization of the LS. Therefore, at most one of the two selected clusterings
2677 uses two clusters, while the other uses a larger number of clusters to provide
2678 a more detailed view of the LS.

6.5.1 Breakout

Two clusterings are examined for **Breakout** (Table 6.5), obtained at layer 4 with $K = 2$ and at layer 6b with $K = 5$.

At layer 4 (Figure 6.7), the partition captures the position of the ball relative to the brick wall. The ball remains below the wall in cluster A. The ball has passed behind the wall in cluster B, allowing it to continue clearing bricks without further interaction with the paddle.

At layer 6b (Figure 6.8), the partition again isolates situations in which the ball has passed behind the wall in cluster E. It also identifies the digging of a tunnel through the left side of the wall in cluster C. Finally, it distinguishes different paddle positions at the beginning of the episode in clusters D, F, and G.

The clustering at layer 4 with $K = 2$ is almost perfectly reproducible across surrogate policies, with a CU of 0.99 and an ACU of 0.87. The clustering at layer 6b with $K = 5$ is less consistent, with a CU of 0.79 and an ACU of 0.61, but reveals additional information about the organization of the LS.

Layer 4	$K = 2$	CU 0.99	AS 0.88	ACU 0.87
A	The ball is below the brick wall.			
B	The ball passes behind the brick wall.			
Layer 6b	$K = 5$	CU 0.79	AS 0.82	ACU 0.61
C	Digging a tunnel through the left side of the wall.			
D	Start of the episode, paddle on the right.			
E	The ball is behind the brick wall.			
F	Start of the episode, paddle in the middle.			
G	Start of the episode, paddle on the left.			

Table 6.5: CU, AS, and ACU of the two clusterings analyzed in the **Breakout** environment.

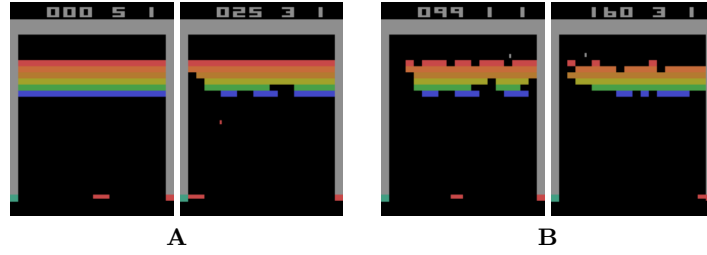


Figure 6.7: Clusters A–B identified by CAR in the **Breakout** environment (layer 4, $K = 2$).

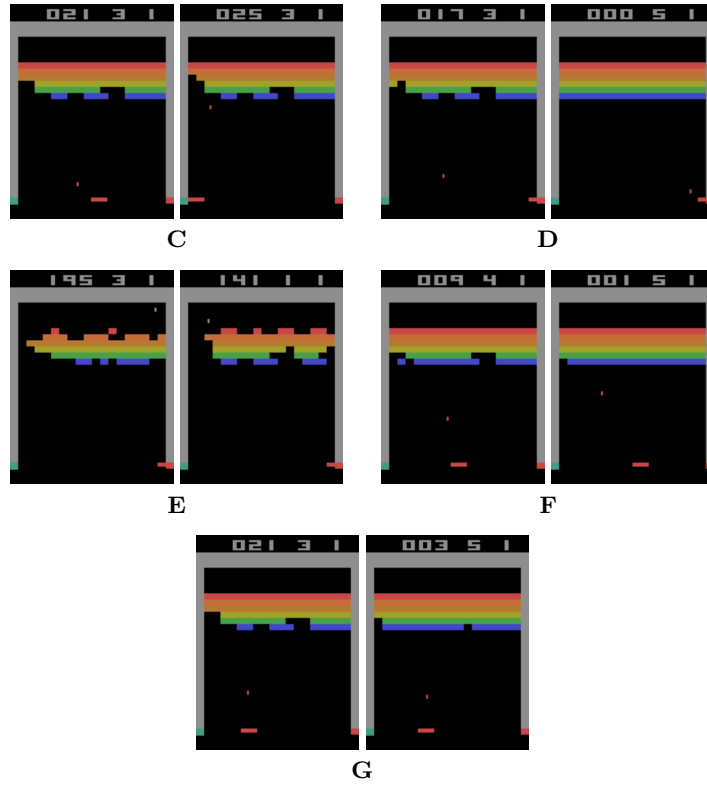


Figure 6.8: Clusters C–G identified by CAR in the **Breakout** environment (layer 6b, $K = 5$).

6.5.2 Mario Bros

Two clusterings are examined for **Mario Bros** (Table 6.6), both obtained at layer 3, with $K = 3$ and $K = 5$.

With $K = 3$ (Figure 6.9), the partition is organized around the location of the action. Mario fights enemies on the left side of the screen in cluster A. He fights enemies on the right side of the screen in cluster C. The POW block is present in cluster B, isolating a fixed and recognizable element of the level.

With $K = 5$ (Figure 6.10), the partition provides a more detailed description of the situations occurring in the level. Mario is on the top-left platform in cluster D. Mario attacks enemies at the top left in cluster F. Mario hits enemies from below in cluster H. Mario simply moves through the level in cluster G. The POW block remains isolated in cluster E.

Increasing K from 3 to 5 lowers CU from 0.84 to 0.75. The AS remains nearly unchanged, with values of 0.66 and 0.65. This results in a decrease in ACU from 0.50 to 0.40.

Layer 3	$K = 3$	CU 0.84	AS 0.66	ACU 0.50
A	Mario fights enemies on the left side of the screen.			
B	The POW block is present.			
C	Mario fights enemies on the right side of the screen.			
Layer 3	$K = 5$	CU 0.75	AS 0.65	ACU 0.40
D	Mario is on the top-left platform.			
E	The POW block is present.			
F	Mario attacks the enemies at the top right.			
G	Mario moves through the level.			
H	Mario hits enemies from below on the left side.			

Table 6.6: CU, AS, and ACU of the two clusterings analyzed in the **Mario Bros** environment.

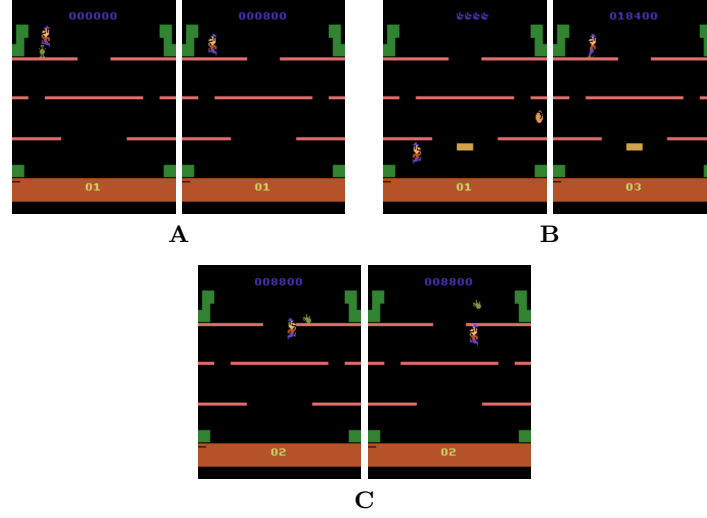


Figure 6.9: Clusters A–C identified by CAR in the **Mario Bros** environment (layer 3, $K = 3$).

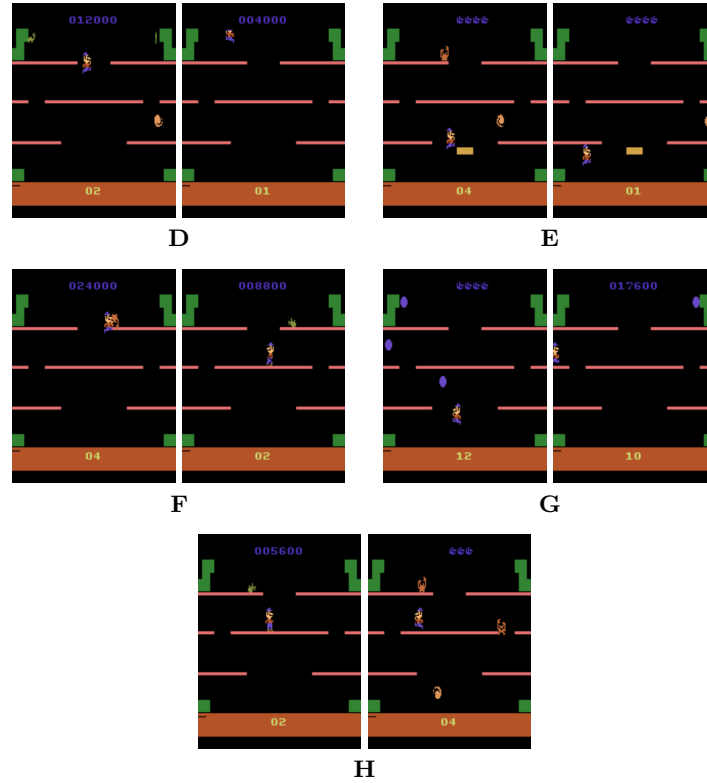


Figure 6.10: Clusters D–H identified by CAR in the **Mario Bros** environment (layer 3, $K = 5$).

2710 6.5.3 Ms. Pac-Man

2711 Two clusterings are examined for **Ms. Pac-Man** (Table 6.7), obtained at layer
2712 6b with $K = 3$ and at layer 4 with $K = 5$.

2713 With $K = 3$ (Figure 6.11), the partition captures the progress and state of
2714 the game rather than a specific spatial feature of the observation. The start of
2715 the episode is captured in cluster B. The middle of the episode is captured in
2716 cluster A. The end of the episode is captured in cluster C.

2717 With $K = 5$ (Figure 6.12), the partition provides a more detailed description
2718 of the episode. The initial animation, during which the character cannot move,
2719 is isolated in cluster H. The beginning of actual play is captured in cluster F. A
2720 situation specific to the game mechanics, in which Pac-Man is in booster mode
2721 and waits in front of the ghost spawn to eat the ghosts, is isolated in cluster D.
2722 The end of the episode, when at most one power pellet remains, is separated
2723 into two similar clusters, E and G.

2724 The two clusterings have the same CU of 0.88. The clustering at layer 4 has
2725 a lower AS of 0.47 compared with 0.52 at layer 6b. This results in a lower ACU
2726 of 0.35 compared with 0.40.

Layer 6b	$K = 3$	CU 0.88	AS 0.52	ACU 0.40
A	Middle of the episode.			
B	Start of the episode.			
C	End of the episode.			
Layer 4	$K = 5$	CU 0.88	AS 0.47	ACU 0.35
D	Pac-Man is in booster mode and waits in front of the ghost spawn to eat the ghosts.			
E	End of the episode, at most one power pellet remains (similar to cluster G).			
F	Start of the episode.			
G	End of the episode, at most one power pellet remains (similar to cluster E).			
H	Start-of-episode animation, the character cannot move.			

Table 6.7: CU, AS, and ACU of the two clusterings analyzed in the **Ms. Pac-Man** environment.

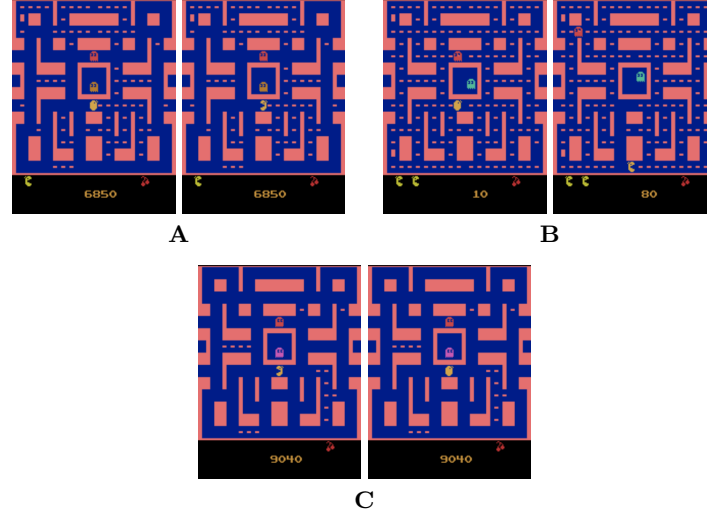


Figure 6.11: Clusters A–C identified by CAR in the **Ms. Pac-Man** environment (layer 6b, $K = 3$).

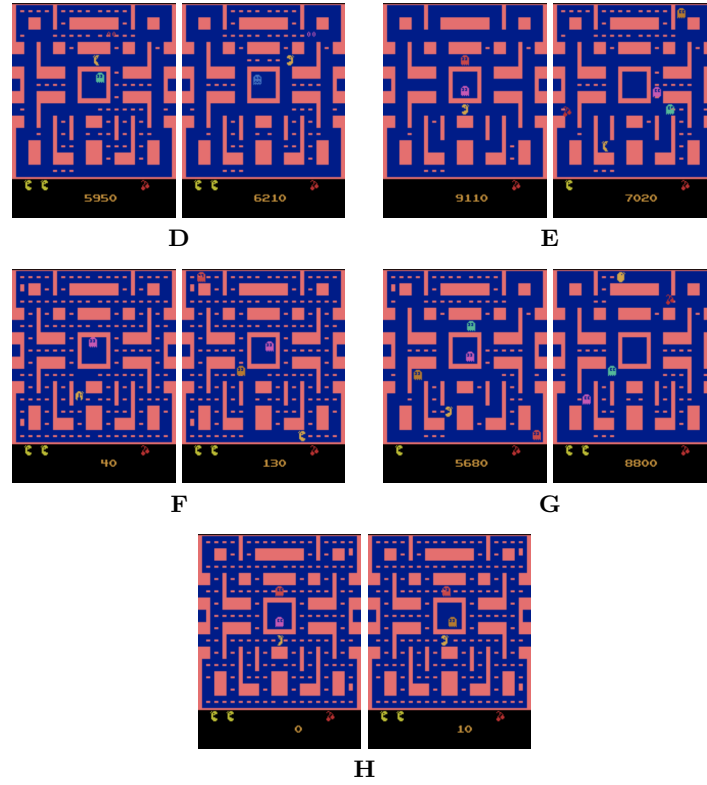


Figure 6.12: Clusters D–H identified by CAR in the **Ms. Pac-Man** environment (layer 4, $K = 5$).

2727 6.5.4 Pong

2728 Two clusterings are examined for **Pong** (Table 6.8), obtained at layer 2 with
 2729 $K = 2$ and at layer 3 with $K = 3$.

2730 At layer 2 (Figure 6.13), the partition separates the observations according to
 2731 the number of digits in the player’s score. A one-digit score corresponds to cluster
 2732 A. A two-digit score corresponds to cluster B. This reflects the progress of the
 2733 game, but only through its visual display at the top of the screen. The partition
 2734 therefore captures a feature of the observation that provides no information
 2735 about how the agent plays.

2736 At layer 3 (Figure 6.14), the partition instead follows the vertical position of
 2737 the ball. The ball is at the top of the field in cluster C. It is in the middle of
 2738 the field in cluster D. It is at the bottom of the field in cluster E. This feature
 2739 directly determines where the agent needs to place its paddle.

2740 The clustering at layer 2 is perfectly reproducible and perfectly abstract, with
 2741 CU, AS, and ACU all equal to 1.00. The clustering at layer 3 is less consistent,
 2742 with a CU of 0.67, and reaches a lower ACU of 0.65.

Layer 2	$K = 2$	CU 1.00	AS 1.00	ACU 1.00
A	The player’s score has two digits.			
B	The player’s score has one digit.			
Layer 3	$K = 3$	CU 0.67	AS 0.98	ACU 0.65
C	The ball is at the top.			
D	The ball is in the middle.			
E	The ball is at the bottom.			

Table 6.8: CU, AS, and ACU of the two clusterings analyzed in the **Pong** environment.

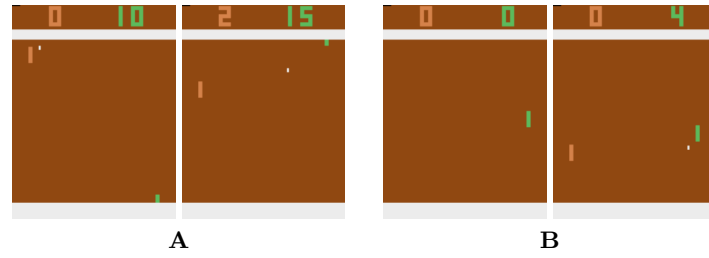


Figure 6.13: Clusters A–B identified by CAR in the **Pong** environment (layer 2, $K = 2$).

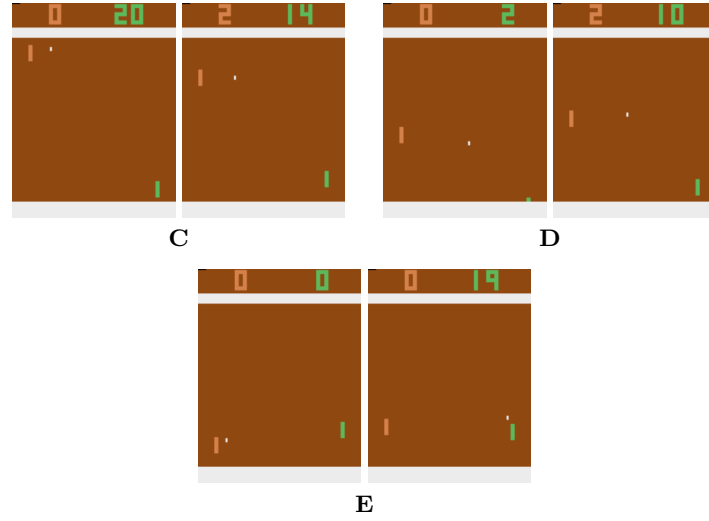


Figure 6.14: Clusters C–E identified by CAR in the **Pong** environment (layer 3, $K = 3$).

2743 6.5.5 Space Invaders

2744 Two clusterings are examined for **Space Invaders** (Table 6.9), obtained at layer
2745 4 with $K = 4$ and at layer 2 with $K = 5$.

2746 At layer 4 (Figure 6.15), the partition tracks the progression of the episode.
2747 The start of the episode is captured in cluster D. The middle of the episode is
2748 captured in cluster B. The end of the episode is captured in cluster C. A specific
2749 behavior in which the agent has cleared the leftmost column and is positioned to
2750 catch the flying saucer that occasionally crosses the top of the screen is captured
2751 in cluster A.

2752 At layer 2 (Figure 6.16), the partition again tracks the progression of the
2753 episode. The start of the episode is captured in cluster G. The middle of the
2754 episode is captured in cluster F. The end of the episode is captured in cluster I.
2755 Attacking the first two columns of the formation is captured in cluster E. Trying
2756 to catch the flying saucer is captured in cluster H.

2757 The clustering at layer 4 has a CU of 0.92 and a AS of 0.69, resulting in a
2758 ACU of 0.61. The clustering at layer 2 has the same CU of 0.92, but a lower AS
2759 of 0.49, resulting in a lower ACU of 0.41.

Layer 4	$K = 4$	CU 0.92	AS 0.69	ACU 0.61
A	The left column has been cleared; the flying saucer occasionally appears.			
B	Middle of the episode.			
C	End of the episode.			
D	Start of the episode.			
Layer 2	$K = 5$	CU 0.92	AS 0.49	ACU 0.41
E	Attacks the first two columns of the formation.			
F	Middle of the episode.			
G	Start of the episode.			
H	Tries to catch the flying saucer.			
I	End of the episode.			

Table 6.9: CU, AS, and ACU of the two clusterings analyzed in the **Space Invaders** environment.

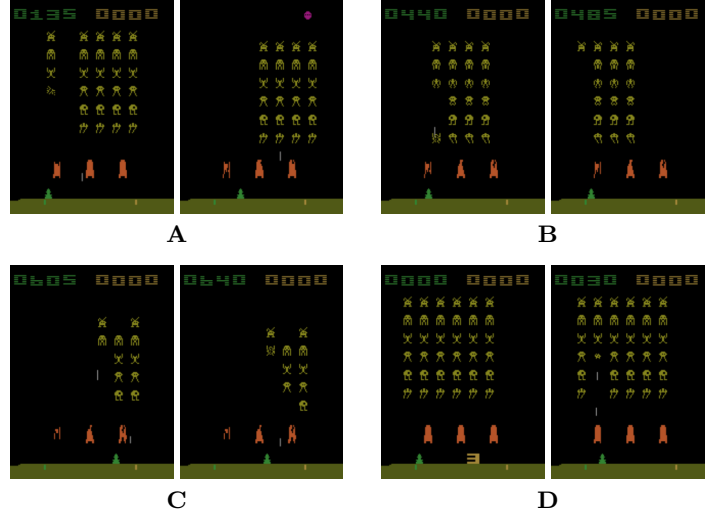


Figure 6.15: Clusters A–D identified by CAR in the **Space Invaders** environment (layer 4, $K = 4$).

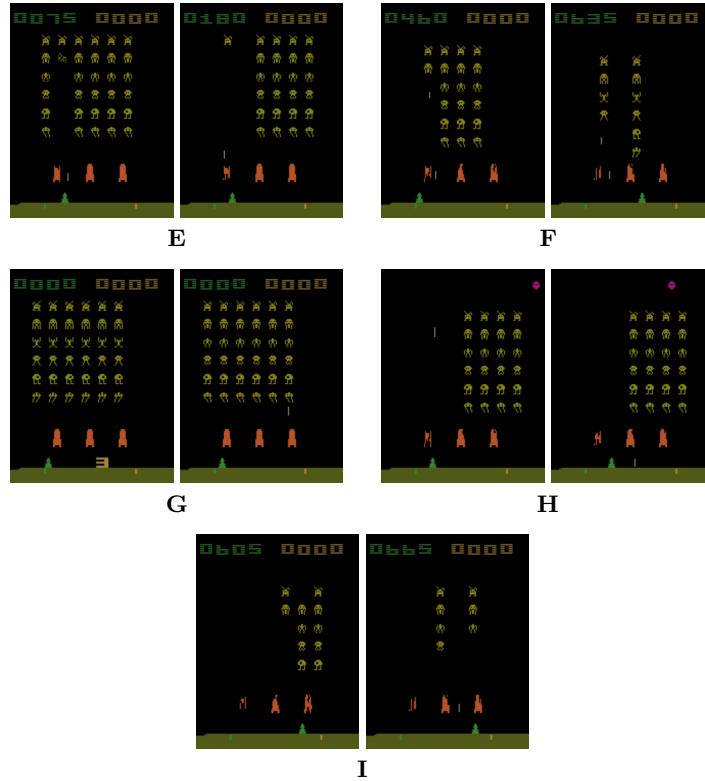


Figure 6.16: Clusters E–I identified by CAR in the **Space Invaders** environment (layer 2, $K = 5$).

6.5.6 Surround

Two clusterings are examined for **Surround** (Table 6.10), obtained at layer 8b with $K = 4$ and at layer 7b with $K = 6$.

At layer 8b (Figure 6.17), the partition captures the start of the episode in cluster B. The outcome of the episode, when the opponent loses, is captured in cluster D. The player's trail is confined to the bottom-left area in cluster A. The player reaches the top of the level from the left and then moves along the top in cluster C.

At layer 7b (Figure 6.18), the partition again captures the start of the episode in cluster J. The outcome of the episode, when the opponent loses, is captured in cluster F. The player moves in the upper-left corner in cluster E. The player moves in the lower-left corner in cluster G. The end of the game is played in the lower-left corner in cluster H. The player crosses the level from the top left to the bottom right in cluster I.

The clustering at layer 8b has a CU of 0.67, a AS of 0.71, and a ACU of 0.38. The clustering at layer 7b reaches the same values, with a CU of 0.67, a AS of 0.71, and a ACU of 0.38.

Layer 8b	$K = 4$	CU 0.67	AS 0.71	ACU 0.38
A	The agent's trail remains confined to the bottom-left area.			
B	Start of the episode.			
C	The agent moves along the top of the level, arriving from the left.			
D	The opponent loses by running into a trail.			
Layer 7b	$K = 6$	CU 0.67	AS 0.71	ACU 0.38
E	The player moves in the upper-left corner.			
F	The opponent loses the confrontation.			
G	The player moves in the lower-left corner.			
H	The end of the game is played in the lower-left corner.			
I	The player starts at the top left and ends at the bottom right.			
J	Start of the episode.			

Table 6.10: CU, AS, and ACU of the two clusterings analyzed in the **Surround** environment.

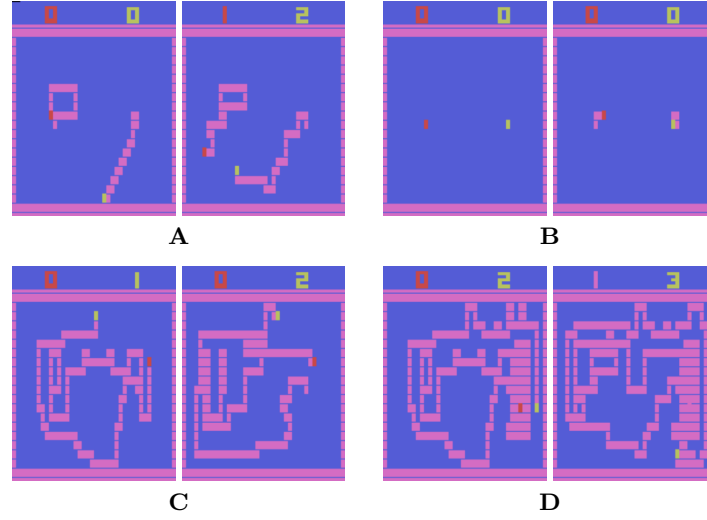


Figure 6.17: Clusters A–D identified by CAR in the **Surround** environment (layer 8b, $K = 4$).

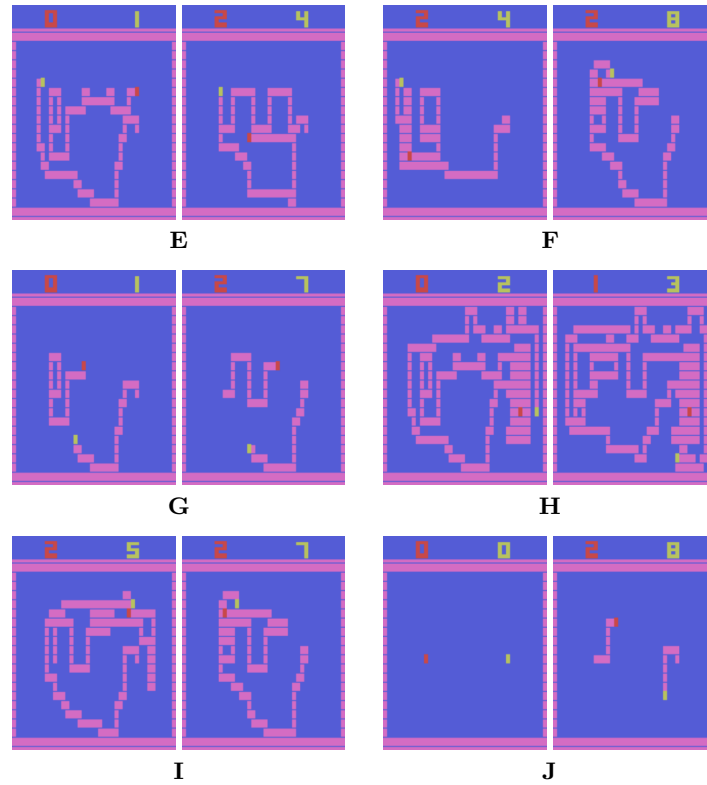


Figure 6.18: Clusters E–J identified by CAR in the **Surround** environment (layer 7b, $K = 6$).

2777 6.6 Conclusion

2778 The quantitative results show that CAR consistently recovers similar clustering
 2779 structures across independently trained surrogate policies. Across the six envi-
 2780 ronments, the CU frequently exceeds 0.5, indicating that independently trained
 2781 policies can develop similarly structured LRs [Amezquita et al., 2020]. The
 2782 CU also tends to decrease as the network gets deeper, with the highest values
 2783 generally observed in the earliest layers. Considering consistency alone would
 2784 therefore favor the earliest layers of the network.

2785 To complement this analysis, the level of abstraction of these representations
 2786 must also be considered. For this purpose, AS is analyzed and shows that the
 2787 earliest layers tend to exhibit lower levels of abstraction. Their representations
 2788 remain strongly related to the observations and the initial structure of the
 2789 network. In the deepest layers, the abstraction can decrease again in some
 2790 environments, as the latent clustering becomes closer to the clustering of the
 2791 action distribution.

2792 This highlights the importance of considering both consistency and abstrac-
 2793 tion when identifying meaningful LRs. The ACU addresses this limitation by
 2794 combining these two properties into a single measure, providing a practical crite-
 2795 rion for identifying layers that achieve a suitable balance between representational
 2796 consistency and abstraction.

2797 These quantitative analyses are complemented by the qualitative analysis,
 2798 which shows that the clusters identified by CAR correspond to interpretable
 2799 properties of the environment and the agent’s behavior. Across the different
 2800 environments, the clusters capture recurring aspects such as episode progression,
 2801 specific game situations, and behavioral patterns. These observations provide
 2802 qualitative evidence that the structures recovered by CAR can reflect semantically
 2803 aspects of the underlying task.

2804 The qualitative analysis also reveals a trade-off associated with the number of
 2805 clusters. Increasing the number of clusters allows more specific situations to be
 2806 distinguished, but tends to reduce the stability of the resulting partitions across
 2807 surrogate policies. Finer clusters have less clear semantic boundaries, reducing
 2808 agreement across independently trained policies. The choice of the number of
 2809 clusters therefore involves a trade-off between recovering stable, shared concepts
 2810 and capturing finer distinctions between situations.

2811 Overall, these results provide empirical support for the assumptions un-
 2812 derlying CAR. Independently trained surrogate policies exhibit a degree of
 2813 convergence in the organization of their LR that cannot be explained solely
 2814 by the considered baselines. The qualitative analysis also shows that these
 2815 clusters can correspond to meaningful properties of the environment and the
 2816 agent’s behavior. Taken together, these findings provide empirical support for
 2817 the *Representation Convergence Hypothesis* (Hyp. 4).

Appendix 1

Additional Results

This appendix reports, for each of the six Atari environments studied in Chapter 6, the full evolution of CU, AS, and ACU with respect to the number of clusters $K \in [2, 20]$, for every analyzed layer, together with the three baseline similarities entering the AS, namely the similarity to the observation clustering (A^{obs}), to the untrained-network latent clustering (A^{untr}), and to the clustering of the action distribution, i.e., of the policy logits before the softmax (A^{act}), underlying the analysis of Section 6.4. All these quantities are computed with the adjusted Rand index, consistent with the rest of this manuscript.

For each environment, one table is reported per metric and per baseline, with the analyzed layers as rows and $K \in [2, 20]$ as columns. In each table, a cell's background shade is proportional to its value (the closer to 1, the darker); a value in **bold** is the highest value of its column, i.e., the best layer for that number of clusters; a value that is underlined is the highest value of its row, i.e., the best number of clusters for that layer; and a value both **bold and underlined** is simultaneously the best of its row and of its column. The last column and the last row report, respectively, the mean $\pm$ standard deviation across K for each layer, and the mean $\pm$ standard deviation across layers for each K , with the best mean of each in bold; the bottom-right cell reports the mean $\pm$ standard deviation over the whole table. The same convention is applied to the baseline tables, where a high value (dark cell) therefore indicates a strong alignment of the latent clustering with the baseline, i.e., a lower contribution to the AS.

2841

1.1 Breakout

Heavy table skipped in draft mode

Table 1.1: CU — **Breakout**.

Heavy table skipped in draft mode

Table 1.2: AS — **Breakout**.

Heavy table skipped in draft mode

Table 1.3: ACU — **Breakout**.

Heavy table skipped in draft mode

Table 1.4: Observation baseline (A^{obs}) — **Breakout**.

Heavy table skipped in draft mode

Table 1.5: Untrained-network (latent) baseline (A^{untr}) — **Breakout**.

Heavy table skipped in draft mode

Table 1.6: Action baseline (A^{act}) — **Breakout**.

2842

1.2 Mario Bros

Heavy table skipped in draft mode

Table 1.7: CU — **Mario Bros.**

Heavy table skipped in draft mode

Table 1.8: AS — **Mario Bros.**

Heavy table skipped in draft mode

Table 1.9: ACU — **Mario Bros.**

Heavy table skipped in draft mode

Table 1.10: Observation baseline (A^{obs}) — **Mario Bros.**

Heavy table skipped in draft mode

Table 1.11: Untrained-network (latent) baseline (A^{untr}) — **Mario Bros.**

Heavy table skipped in draft mode

Table 1.12: Action baseline (A^{act}) — **Mario Bros.**

2843

1.3 Ms. Pac-Man

Heavy table skipped in draft mode

Table 1.13: CU — Ms. Pac-Man.

Heavy table skipped in draft mode

Table 1.14: AS — Ms. Pac-Man.

Heavy table skipped in draft mode

Table 1.15: ACU — **Ms. Pac-Man**.

Heavy table skipped in draft mode

Table 1.16: Observation baseline (A^{obs}) — **Ms. Pac-Man**.

Heavy table skipped in draft mode

Table 1.17: Untrained-network (latent) baseline (A^{untr}) — **Ms. Pac-Man**.

Heavy table skipped in draft mode

Table 1.18: Action baseline (A^{act}) — **Ms. Pac-Man**.

1.4 Pong

<i>Heavy table skipped in draft mode</i>
--

Table 1.19: CU — **Pong**.

<i>Heavy table skipped in draft mode</i>
--

Table 1.20: AS — **Pong**.

Heavy table skipped in draft mode

Table 1.21: ACU — **Pong**.

Heavy table skipped in draft mode

Table 1.22: Observation baseline (A^{obs}) — **Pong**.

Heavy table skipped in draft mode

Table 1.23: Untrained-network (latent) baseline (A^{untr}) — **Pong**.

Heavy table skipped in draft mode

Table 1.24: Action baseline (A^{act}) — **Pong**.

1.5 Space Invaders

2845

<i>Heavy table skipped in draft mode</i>
--

Table 1.25: CU — **Space Invaders**.

<i>Heavy table skipped in draft mode</i>
--

Table 1.26: AS — **Space Invaders**.

Heavy table skipped in draft mode

Table 1.27: ACU — **Space Invaders**.

Heavy table skipped in draft mode

Table 1.28: Observation baseline (A^{obs}) — **Space Invaders**.

Heavy table skipped in draft mode

Table 1.29: Untrained-network (latent) baseline (A^{untr}) — **Space Invaders**.

Heavy table skipped in draft mode

Table 1.30: Action baseline (A^{act}) — **Space Invaders**.

2846

1.6 Surround

Heavy table skipped in draft mode

Table 1.31: CU — **Surround**.

Heavy table skipped in draft mode

Table 1.32: AS — **Surround**.

Heavy table skipped in draft mode

Table 1.33: ACU — **Surround**.

Heavy table skipped in draft mode

Table 1.34: Observation baseline (A^{obs}) — **Surround**.

Heavy table skipped in draft mode

Table 1.35: Untrained-network (latent) baseline (A^{untr}) — **Surround**.

Heavy table skipped in draft mode

Table 1.36: Action baseline (A^{act}) — **Surround**.

Acronyms

2848	ACU <i>Adjusted Clustering Universality</i> 82, 90–95, 97, 99, 101, 103, 105, 107, 108,
2849	110, 113, 116, 119, 122, 125
2850	AI Artificial Intelligence 50, 54, 55
2851	APPO Asynchronous Proximal Policy Optimization 83
2852	ARI Adjusted Rand Index 76–80
2853	AS <i>Abstraction Score</i> 82, 90–93, 95, 97, 99, 101, 103, 105, 107–109, 112, 115,
2854	118, 121, 124
2855	BC Behavioral Cloning 84
2856	CAR Clustering Agent Representation 69–73, 75–77, 79, 81–88, 96, 98, 100,
2857	102, 104, 106, 107
2858	CNN Convolutional Neural Network 2, 15, 87
2859	CS Concept Structure 19, 23, 24, 29–32, 69–71, 73
2860	CU <i>Clustering Universality</i> 82, 90–93, 95, 97, 99, 101, 103, 105, 107–109, 112,
2861	115, 118, 121, 124
2862	DDPG Deep Deterministic Policy Gradient 37
2863	DL Deep Learning 1–7, 9, 14–16, 36, 48, 50, 54, 55, 82, 83
2864	DQN Deep Q-Networks 36, 83
2865	GPU Graphics Processing Unit 83–85
2866	LLM Large Language Model 3, 16, 20–22, 27, 28, 37, 50
2867	LR Latent Representation 16–18, 20–24, 26–32, 62, 69–71, 73, 77, 81, 82, 84, 85,
2868	90, 107
2869	LS Latent Space 15–20, 22–31, 62, 65, 66, 68–73, 75–77, 79, 80, 90, 94, 95
2870	LSTM Long Short-Term Memory 2, 15
2871	MDP Markov Decision Processes 35–37, 39–41, 43–49, 59, 65, 66
2872	MLP MultiLayer Perceptron 5–7, 15, 87
2873	MSE Mean Squared Error 4, 30, 47, 49, 72, 74
2874	NN Neural Network 1–13, 15–17, 19, 20, 24, 26, 30, 32, 33, 36, 37, 44–46, 54,
2875	58, 59, 61, 65, 66, 68–72, 83, 84
2876	PPO Proximal Policy Optimization 37, 83, 88, 89
2877	RI Rand Index 78
2878	RL Reinforcement Learning 16, 33–39, 41–46, 48, 50, 51, 55–60, 63, 67–69, 72,
2879	81–85

- 2880 **SAE** Sparse Autoencoder 26–29, 31
- 2881 **SGD** Stochastic Gradient Descent 10, 12, 15, 20, 21, 27, 30
- 2882 **XAI** eXplainable Artificial Intelligence 50, 51, 54–56, 67
- 2883 **XRL** eXplainable Reinforcement Learning 48, 50, 51, 55–59, 63, 67, 68

Bibliography

- [Alaarabiou et al., 2024] Alaarabiou, M., Delestre, N., and Vercouter, L. (2024). Explicabilité en Apprentissage par Renforcement : vers une Taxinomie Unifiée. In 18èmes Journées d’Intelligence Artificielle Fondamentale, La rochelle, France.
- [Amezquita et al., 2020] Amezcquita, R. A., Lun, A. T., Becht, E., Carey, V. J., Carpp, L. N., Geistlinger, L., Marini, F., Rue-Albrecht, K., Risso, D., Sone-son, C., et al. (2020). Orchestrating single-cell analysis with Bioconductor, volume 17. Nature Publishing Group US New York. **Book**.
- [Apache Arrow, 2016] Apache Arrow (2016). Apache Arrow and Parquet. <https://arrow.apache.org>.
- [Barredo Arrieta et al., 2020] Barredo Arrieta, A., Díaz-Rodríguez, N., Del Ser, J., Bennetot, A., Tabik, S., Barbado, A., García, S., Gil-López, S., Molina, D., Benjamins, R., Chatila, R., and Herrera, F. (2020). Explainable artificial intelligence (xai): Concepts, taxonomies, opportunities and challenges toward responsible ai. Information Fusion, 58:82–115. **Article**.
- [Barto et al., 1983] Barto, A. G., Sutton, R. S., and Anderson, C. W. (1983). Neuronlike adaptive elements that can solve difficult learning control problems. IEEE Transactions on Systems, Man, and Cybernetics, 13(5):835–846.
- [Bellemare et al., 2013] Bellemare, M. G., Naddaf, Y., Veness, J., and Bowling, M. (2013). The arcade learning environment: An evaluation platform for general agents. Journal of Artificial Intelligence Research, 47:253–279. **Article**.
- [Bellman, 1957] Bellman, R. E. (1957). Dynamic programming.
- [Bellucci et al., 2021] Bellucci, M., Delestre, N., Malandain, N., and Zanni-Merk, C. (2021). Towards a terminology for a fully contextualized xai. Procedia Computer Science, 192:241–250. **Article**.
- [Bengio et al., 2014] Bengio, Y., Courville, A., and Vincent, P. (2014). Representation learning: A review and new perspectives.
- [Beyret et al., 2019] Beyret, B., Shafti, A., and Faisal, A. A. (2019). Dot-to-dot: Explainable hierarchical reinforcement learning for robotic manipulation. In 2019 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS). IEEE. **Article**.

- [Bond-Taylor et al., 2021] Bond-Taylor, S., Leach, A., Long, Y., and Willcocks, C. G. (2021). Deep generative modelling: A comparative review of vaes, gans, normalizing flows, energy-based and autoregressive models. **Article**.
- [Bricken et al., 2023] Bricken, T., Templeton, A., Batson, J., Chen, B., Jermyn, A., Conerly, T., et al. (2023). Towards monosemanticity: Decomposing language models with dictionary learning. Transformer Circuits Thread. **Web**.
- [Brown et al., 2020] Brown, T., Mann, B., Ryder, N., Subbiah, M., Kaplan, J. D., Dhariwal, P., Neelakantan, A., Shyam, P., Sastry, G., Askell, A., et al. (2020). Language models are few-shot learners. Advances in neural information processing systems, 33:1877–1901.
- [Burda et al., 2018] Burda, Y., Edwards, H., Pathak, D., Storkey, A., Darrell, T., and Efros, A. A. (2018). Large-scale study of curiosity-driven learning.
- [Chen et al., 2021] Chen, M., Tworek, J., Jun, H., Yuan, Q., de Oliveira Pinto, H. P., Kaplan, J., et al. (2021). Evaluating large language models trained on code. **Article**.
- [Coates et al., 2011] Coates, A., Ng, A. Y., and Lee, H. (2011). An analysis of single-layer networks in unsupervised feature learning. In Proceedings of the Fourteenth International Conference on Artificial Intelligence and Statistics (AISTATS), pages 215–223.
- [Covington et al., 2016] Covington, P., Adams, J., and Sargin, E. (2016). Deep neural networks for youtube recommendations. In Proceedings of the 10th ACM conference on recommender systems, pages 191–198.
- [Doshi-Velez and Kim, 2017] Doshi-Velez, F. and Kim, B. (2017). Towards a rigorous science of interpretable machine learning. **Article**.
- [Elhage et al., 2022] Elhage, N., Hume, T., Olsson, C., Schiefer, N., Henighan, T., Kravec, S., et al. (2022). Toy models of superposition. Transformer Circuits Thread. **Web**.
- [Espeholt et al., 2018] Espeholt, L., Soyer, H., Munos, R., Simonyan, K., Mnih, V., Ward, T., Doron, Y., Firoiu, V., Harley, T., Dunning, I., Legg, S., and Kavukcuoglu, K. (2018). IMPALA: Scalable distributed deep-RL with importance weighted actor-learner architectures. In Proceedings of the 35th International Conference on Machine Learning (ICML).
- [Falcon, 2019] Falcon, W. (2019). PyTorch Lightning. <https://github.com/Lightning-AI/pytorch-lightning>.
- [Garreau and von Luxburg, 2020] Garreau, D. and von Luxburg, U. (2020). Explaining the explainer: A first theoretical analysis of lime. In Chiappa, S. and Calandra, R., editors, Proceedings of the Twenty Third International Conference on Artificial Intelligence and Statistics, volume 108 of Proceedings of Machine Learning Research, pages 1287–1296. PMLR. **Article**.
- [Goodfellow et al., 2016] Goodfellow, I., Bengio, Y., Courville, A., and Bengio, Y. (2016). Deep learning, volume 1. MIT press Cambridge.

- [Greydanus et al., 2018] Greydanus, S., Koul, A., Dodge, J., and Fern, A. (2018). Visualizing and understanding atari agents. **Article. Code.**
- [Gunning and Aha, 2019] Gunning, D. and Aha, D. (2019). Darpa’s explainable ai (xai) program. *AI Magazine*, 40(2):44–58. **Article.**
- [Guo et al.,] Guo, X., Gao, L., Liu, X., and Yin, J. Improved deep embedded clustering with local structure preservation. In *Proceedings of the Twenty-Sixth International Joint Conference on Artificial Intelligence*, pages 1753–1759. International Joint Conferences on Artificial Intelligence Organization.
- [Haas et al., 2024] Haas, R., Huberman-Spiegelglas, I., Mulayoff, R., Graßhof, S., Brandt, S. S., and Michaeli, T. (2024). Discovering interpretable directions in the semantic latent space of diffusion models. In *2024 IEEE 18th International Conference on Automatic Face and Gesture Recognition (FG)*, pages 1–9. IEEE.
- [Hafner et al., 2023] Hafner, D., Pasukonis, J., Ba, J., and Lillicrap, T. (2023). Mastering diverse domains through world models.
- [He et al., 2017] He, X., Liao, L., Zhang, H., Nie, L., Hu, X., and Chua, T.-S. (2017). Neural collaborative filtering.
- [Hessel et al., 2017] Hessel, M., Modayil, J., van Hasselt, H., Schaul, T., Ostrovski, G., Dabney, W., Horgan, D., Piot, B., Azar, M., and Silver, D. (2017). Rainbow: Combining improvements in deep reinforcement learning.
- [Hilbert and López, 2011] Hilbert, M. and López, P. (2011). The world’s technological capacity to store, communicate, and compute information. *science*, 332(6025):60–65.
- [Hochreiter and Schmidhuber, 1997] Hochreiter, S. and Schmidhuber, J. (1997). Long short-term memory. *Neural computation*, 9(8):1735–1780.
- [Hornik et al., 1989] Hornik, K., Stinchcombe, M., and White, H. (1989). Multilayer feedforward networks are universal approximators. *Neural networks*, 2(5):359–366.
- [Howard, 1960] Howard, R. A. (1960). Dynamic programming and markov processes. MIT Press.
- [Huber et al., 2019] Huber, T., Schiller, D., and André, E. (2019). Enhancing explainability of deep reinforcement learning through selective layer-wise relevance propagation. In *KI 2019: Advances in Artificial Intelligence: 42nd German Conference on AI, Kassel, Germany, September 23–26, 2019, Proceedings 42*, pages 188–202. Springer. **Article.**
- [Hubert and Arabie, 1985] Hubert, L. and Arabie, P. (1985). Comparing partitions. *Journal of classification*, 2(1):193–218. **Article.**
- [Itaya et al., 2021] Itaya, H., Hirakawa, T., Yamashita, T., Fujiyoshi, H., and Sugiura, K. (2021). Visual explanation using attention mechanism in actor-critic-based deep reinforcement learning. **Article.**

- [Jermyn et al., 2024] Jermyn, A., Templeton, A., Batson, J., and Bricken, T. (2024). Tanh penalty in dictionary learning. **Web**.
- [Jumper et al., 2021] Jumper, J., Evans, R., Pritzel, A., Green, T., Figurnov, M., Ronneberger, O., Tunyasuvunakool, K., Bates, R., Žídek, A., Potapenko, A., et al. (2021). Highly accurate protein structure prediction with alphafold. *nature*, 596(7873):583–589.
- [Juozapaitis et al., 2019] Juozapaitis, Z., Koul, A., Fern, A., Erwig, M., and Doshi-Velez, F. (2019). Explainable reinforcement learning via reward decomposition. In *IJCAI/ECAI Workshop on explainable artificial intelligence*. **Article**.
- [Karvonen, 2024] Karvonen, A. (2024). Emergent world models and latent variable estimation in chess-playing language models.
- [Kawaguchi et al., 2022] Kawaguchi, K., Bengio, Y., and Kaelbling, L. (2022). *Generalization in Deep Learning*, page 112–148. Cambridge University Press.
- [Knuth and Moore, 1975] Knuth, D. E. and Moore, R. W. (1975). An analysis of alpha-beta pruning. *Artificial intelligence*, 6(4):293–326. **Article**.
- [Krizhevsky et al., 2012] Krizhevsky, A., Sutskever, I., and Hinton, G. E. (2012). Imagenet classification with deep convolutional neural networks. *Advances in neural information processing systems*, 25.
- [LeCun et al., 1989] LeCun, Y., Boser, B., Denker, J. S., Henderson, D., Howard, R. E., Hubbard, W., and Jackel, L. D. (1989). Backpropagation applied to handwritten zip code recognition. *Neural computation*, 1(4):541–551.
- [Lewis et al., 2021] Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., Küttler, H., Lewis, M., tau Yih, W., Rocktäschel, T., Riedel, S., and Kiela, D. (2021). Retrieval-augmented generation for knowledge-intensive nlp tasks.
- [Li et al., 2024] Li, K., Hopkins, A. K., Bau, D., Viégas, F., Pfister, H., and Wattenberg, M. (2024). Emergent world representations: Exploring a sequence model trained on a synthetic task.
- [Liang et al., 2018] Liang, E., Liaw, R., Nishihara, R., Moritz, P., Fox, R., Goldberg, K., Gonzalez, J. E., Jordan, M. I., and Stoica, I. (2018). RLlib: Abstractions for distributed reinforcement learning. In *Proceedings of the 35th International Conference on Machine Learning (ICML)*.
- [Lillicrap et al., 2015] Lillicrap, T. P., Hunt, J. J., Pritzel, A., Heess, N., Erez, T., Tassa, Y., Silver, D., and Wierstra, D. (2015). Continuous control with deep reinforcement learning.
- [Lu et al., 2019] Lu, G., Ouyang, W., Xu, D., Zhang, X., Cai, C., and Gao, Z. (2019). Dvc: An end-to-end deep video compression framework.
- [Machado et al., 2017] Machado, M. C., Bellemare, M. G., Talvitie, E., Veness, J., Hausknecht, M., and Bowling, M. (2017). Revisiting the arcade learning environment: Evaluation protocols and open problems for general agents. **Article. Web**.

- [MacQueen, 1967] MacQueen, J. (1967). Multivariate observations. In Proceedings of the 5th Berkeley Symposium on Mathematical Statistics and Probability, volume 1, pages 281–297. **Article**.
- [Madumal et al., 2020] Madumal, P., Miller, T., Sonenberg, L., and Vetere, F. (2020). Explainable reinforcement learning through a causal lens. In Proceedings of the AAAI conference on artificial intelligence, volume 34, pages 2493–2500. **Article**.
- [Marconato et al., 2023] Marconato, E., Passerini, A., and Teso, S. (2023). Interpretability is in the mind of the beholder: A causal framework for human-interpretable representation learning.
- [McCulloch and Pitts, 1943] McCulloch, W. S. and Pitts, W. (1943). A logical calculus of the ideas immanent in nervous activity. The bulletin of mathematical biophysics, 5(4):115–133.
- [Miklautz et al.,] Miklautz, L., Klein, T., Sidak, K., Leiber, C., Lang, T., Shkabrii, A., Tschitschek, S., and Plant, C. Breaking the reclustering barrier in centroid-based deep clustering.
- [Miller, 2019] Miller, T. (2019). Explanation in artificial intelligence: Insights from the social sciences. Artificial Intelligence, 267:1–38. **Article**.
- [Minsky and Papert, 1969] Minsky, M. and Papert, S. (1969). An introduction to computational geometry. Cambridge tiass., HIT, 479(480):104.
- [Misak, 2013] Misak, C. (2013). The American Pragmatists. Oxford University Press.
- [Mitchell, 1997] Mitchell, T. M. (1997). Machine Learning. McGraw-Hill, New York.
- [Mnih et al., 2013] Mnih, V., Kavukcuoglu, K., Silver, D., Graves, A., Antonoglou, I., Wierstra, D., and Riedmiller, M. (2013). Playing atari with deep reinforcement learning.
- [Moritz et al., 2018] Moritz, P., Nishihara, R., Wang, S., Tumanov, A., Liaw, R., Liang, E., Elibol, M., Yang, Z., Paul, W., Jordan, M. I., and Stoica, I. (2018). Ray: A distributed framework for emerging AI applications. In 13th USENIX Symposium on Operating Systems Design and Implementation (OSDI), pages 561–577.
- [Mott et al., 2019] Mott, A., Zoran, D., Chrzanowski, M., Wierstra, D., and Jimenez Rezende, D. (2019). Towards interpretable reinforcement learning using attention augmented agents. Advances in neural information processing systems, 32. **Article**.
- [Newell, 1980] Newell, A. (1980). Physical symbol systems. Cognitive Science, 4(2):135–183.
- [Olah et al., 2020] Olah, C., Cammarata, N., Schubert, L., Goh, G., Petrov, M., and Carter, S. (2020). Zoom in: An introduction to circuits. Distill. **Article**.

- [Olson et al., 2021] Olson, M. L., Khanna, R., Neal, L., Li, F., and Wong, W.-K. (2021). Counterfactual state explanations for reinforcement learning agents via generative deep learning. *Artificial Intelligence*, 295:103455. **Article. Code.**
- [OpenAI et al., 2019] OpenAI, :, Berner, C., Brockman, G., Chan, B., Cheung, V., Dębiak, P., Dennison, C., Farhi, D., Fischer, Q., Hashme, S., Hesse, C., Józefowicz, R., Gray, S., Olsson, C., Pachocki, J., Petrov, M., d. O. Pinto, H. P., Raiman, J., Salimans, T., Schlatter, J., Schneider, J., Sidor, S., Sutskever, I., Tang, J., Wolski, F., and Zhang, S. (2019). Dota 2 with large scale deep reinforcement learning.
- [Ouyang et al., 2022] Ouyang, L., Wu, J., Jiang, X., Almeida, D., Wainwright, C. L., Mishkin, P., Zhang, C., Agarwal, S., Slama, K., Ray, A., Schulman, J., Hilton, J., Kelton, F., Miller, L., Simens, M., Askill, A., Welinder, P., Christiano, P., Leike, J., and Lowe, R. (2022). Training language models to follow instructions with human feedback.
- [Paszke et al., 2019] Paszke, A., Gross, S., Massa, F., Lerer, A., Bradbury, J., Chanan, G., Killeen, T., Lin, Z., Gimelshein, N., Antiga, L., Desmaison, A., Köpf, A., Yang, E., DeVito, Z., Raison, M., Tejani, A., Chilamkurthy, S., Steiner, B., Fang, L., Bai, J., and Chintala, S. (2019). PyTorch: An imperative style, high-performance deep learning library. In *Advances in Neural Information Processing Systems (NeurIPS)*.
- [Payani and Fekri, 2019] Payani, A. and Fekri, F. (2019). Inductive logic programming via differentiable deep neural logic networks. **Article. Code.**
- [Peirce, 1878] Peirce, C. S. (1878). How to make our ideas clear. *Popular Science Monthly*, 12:286–302.
- [Peyré and Cuturi, 2020] Peyré, G. and Cuturi, M. (2020). Computational optimal transport. **Article.**
- [Radford et al., 2021] Radford, A., Kim, J. W., Hallacy, C., Ramesh, A., Goh, G., Agarwal, S., Sastry, G., Askill, A., Mishkin, P., Clark, J., Krueger, G., and Sutskever, I. (2021). Learning transferable visual models from natural language supervision.
- [Raina et al., 2009] Raina, R., Madhavan, A., Ng, A. Y., et al. (2009). Large-scale deep unsupervised learning using graphics processors. In *Icml*, volume 9, pages 873–880.
- [Rand, 1971] Rand, W. M. (1971). Objective criteria for the evaluation of clustering methods. *Journal of the American Statistical association*, 66(336):846–850.
- [RAPIDS, 2023] RAPIDS (2023). RAPIDS cuML: GPU-accelerated machine learning. <https://github.com/rapidsai/cuml>.
- [Ren et al.,] Ren, Y., Pu, J., Yang, Z., Xu, J., Li, G., Pu, X., Yu, P. S., and He, L. Deep clustering: A comprehensive survey.
- [Rosenblatt, 1958] Rosenblatt, F. (1958). The perceptron: a probabilistic model for information storage and organization in the brain. *Psychological review*, 65(6):386.

- [Ross and Bagnell, 2010] Ross, S. and Bagnell, D. (2010). Efficient reductions for imitation learning. In Teh, Y. W. and Titterton, M., editors, Proceedings of the Thirteenth International Conference on Artificial Intelligence and Statistics, volume 9 of Proceedings of Machine Learning Research, pages 661–668, Chia Laguna Resort, Sardinia, Italy. PMLR. **Article**.
- [Ross et al., 2011] Ross, S., Gordon, G., and Bagnell, D. (2011). A reduction of imitation learning and structured prediction to no-regret online learning. In Gordon, G., Dunson, D., and Dudík, M., editors, Proceedings of the Fourteenth International Conference on Artificial Intelligence and Statistics, volume 15 of Proceedings of Machine Learning Research, pages 627–635, Fort Lauderdale, FL, USA. PMLR. **Article**.
- [Rudin, 2019] Rudin, C. (2019). Stop explaining black box machine learning models for high stakes decisions and use interpretable models instead. Nature Machine Intelligence, 1(5):206–215. **Article**.
- [Rumelhart et al., 1986] Rumelhart, D. E., Hinton, G. E., and Williams, R. J. (1986). Learning representations by back-propagating errors. nature, 323(6088):533–536.
- [Russell and Norvig, 2021] Russell, S. J. and Norvig, P. (2021). Artificial Intelligence: A Modern Approach. Pearson, 4th edition.
- [Samvelyan et al., 2019] Samvelyan, M., Rashid, T., de Witt, C. S., Farquhar, G., Nardelli, N., Rudner, T. G. J., Hung, C.-M., Torr, P. H. S., Foerster, J., and Whiteson, S. (2019). The starcraft multi-agent challenge.
- [Schulman et al., 2017] Schulman, J., Wolski, F., Dhariwal, P., Radford, A., and Klimov, O. (2017). Proximal policy optimization algorithms.
- [Sequeira and Gervasio, 2020] Sequeira, P. and Gervasio, M. (2020). Interestingness elements for explainable reinforcement learning: Understanding agents’ capabilities and limitations. Artificial Intelligence, 288:103367. **Article**.
- [Seshia et al., 2022] Seshia, S. A., Sadigh, D., and Sastry, S. S. (2022). Toward verified artificial intelligence. Communications of the ACM, 65(7):46–55. **Article**.
- [Shao et al., 2024] Shao, Z., Wang, P., Zhu, Q., Xu, R., Song, J., Bi, X., Zhang, H., Zhang, M., Li, Y. K., Wu, Y., and Guo, D. (2024). Deepseekmath: Pushing the limits of mathematical reasoning in open language models.
- [Shlens, 2014] Shlens, J. (2014). Notes on kullback-leibler divergence and likelihood. **Article**.
- [Siegel and Craver, 2024] Siegel, G. and Craver, C. F. (2024). Phenomenological laws and mechanistic explanations. Philosophy of Science, 91(1):132–150.
- [Sieusahai and Guzdial, 2021] Sieusahai, A. and Guzdial, M. (2021). Explaining deep reinforcement learning agents in the atari domain through a surrogate model. **Article**.

- [Silver et al., 2017a] Silver, D., Hubert, T., Schrittwieser, J., Antonoglou, I., Lai, M., Guez, A., Lanctot, M., Sifre, L., Kumaran, D., Graepel, T., et al. (2017a). Mastering chess and shogi by self-play with a general reinforcement learning algorithm. arXiv preprint arXiv:1712.01815.
- [Silver et al., 2017b] Silver, D., Hubert, T., Schrittwieser, J., Antonoglou, I., Lai, M., Guez, A., Lanctot, M., Sifre, L., Kumaran, D., Graepel, T., Lillicrap, T., Simonyan, K., and Hassabis, D. (2017b). Mastering chess and shogi by self-play with a general reinforcement learning algorithm.
- [Silver et al., 2021] Silver, D., Singh, S., Precup, D., and Sutton, R. S. (2021). Reward is enough. Artificial intelligence, 299:103535.
- [Simon, 1996] Simon, H. A. (1996). The Sciences of the Artificial. MIT Press, 3 edition.
- [Simonyan et al., 2014] Simonyan, K., Vedaldi, A., and Zisserman, A. (2014). Deep inside convolutional networks: Visualising image classification models and saliency maps. **Article. Code**.
- [Skinner, 1938] Skinner, B. F. (1938). The behavior of organisms: An experimental analysis. Appleton-Century.
- [Sutton, 1988] Sutton, R. S. (1988). Learning to predict by the methods of temporal differences. Machine Learning, 3:9–44.
- [Sutton and Barto, 1998] Sutton, R. S. and Barto, A. G. (1998). Reinforcement Learning: An Introduction. MIT Press, 1st edition.
- [Sutton and Barto, 2018] Sutton, R. S. and Barto, A. G. (2018). Reinforcement Learning: An Introduction. MIT Press. **Book**.
- [Telgarsky, 2016] Telgarsky, M. (2016). Benefits of depth in neural networks. In Conference on learning theory, pages 1517–1539. PMLR.
- [Templeton et al., 2024] Templeton, A., Conerly, T., Marcus, J., Lindsey, J., Bricken, T., Chen, B., et al. (2024). Scaling monosemanticity: Extracting interpretable features from claude 3 sonnet. Transformer Circuits Thread. **Web**.
- [Thorndike, 1911] Thorndike, E. L. (1911). Animal intelligence: Experimental studies. The Psychological Review Monograph Supplement, 8(4).
- [Topin et al., 2021] Topin, N., Milani, S., Fang, F., and Veloso, M. (2021). Iterative bounding mdps: Learning interpretable policies via non-interpretable methods. In Proceedings of the AAAI Conference on Artificial Intelligence, volume 35, pages 9923–9931. **Article**.
- [Towers et al., 2024] Towers, M., Kwiatkowski, A., Terry, J., Balis, J. U., De Cola, G., Deleu, T., Goulão, M., Kallinteris, A., Krimmel, M., KG, A., et al. (2024). Gymnasium: A standard interface for reinforcement learning environments.

- [Trivedi et al., 2022] Trivedi, D., Zhang, J., Sun, S.-H., and Lim, J. J. (2022). Learning to synthesize programs as interpretable and generalizable policies. **Article. Code.**
- [Tsitsiklis and Van Roy, 1996] Tsitsiklis, J. and Van Roy, B. (1996). Analysis of temporal-difference learning with function approximation. Advances in neural information processing systems, 9.
- [Turing, 1948] Turing, A. (1948). Intelligent machinery. National Physical Laboratory Report.
- [Vaswani et al., 2017] Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł., and Polosukhin, I. (2017). Attention is all you need. Advances in neural information processing systems, 30.
- [Vaswani et al., 2023] Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł., and Polosukhin, I. (2023). Attention is all you need. **Article.**
- [Voynov and Babenko, 2020] Voynov, A. and Babenko, A. (2020). Unsupervised discovery of interpretable directions in the gan latent space.
- [Watkins, 1989] Watkins, C. J. C. H. (1989). Learning from delayed rewards. PhD Thesis, University of Cambridge.
- [Watkins and Dayan, 1992] Watkins, C. J. C. H. and Dayan, P. (1992). Q-learning. Machine Learning, 8:279–292.
- [Wei et al.,] Wei, X., Zhang, Z., Huang, H., and Zhou, Y. An overview on deep clustering. 590:127761.
- [Weitkamp et al., 2019] Weitkamp, L., van der Pol, E., and Akata, Z. (2019). Visual rationalizations in deep reinforcement learning for atari games. **Article.**
- [Xie et al.,] Xie, J., Girshick, R., and Farhadi, A. Unsupervised deep embedding for clustering analysis.
- [Zahavy et al., 2017] Zahavy, T., Zrihem, N. B., and Mannor, S. (2017). Graying the black box: Understanding dqns. **Article.**
- [Zednik, 2019] Zednik, C. (2019). Solving the black box problem: A normative framework for explainable artificial intelligence. arXiv preprint arXiv:1903.04361.